The Frontier Method

Circuit lower bounds from graph layouts, with a formal proof

Sam McGuire Samuel Schlesinger Ryan Williams
Google MIT
sgschlesinger@gmail.com

Abstract

We prove that circuits of fan-in two over *any* basis need more than $(L-\varepsilon)\,n$ gates to decide explicit sets in P, where

$$L \;=\; 1+\frac{\pi}{3\arccos\frac{1+2\sqrt{2}}{4}} \;\approx\; 4.5625.$$

The best previous lower bound for an explicit function over the full binary basis was $(3.1-o(1))\,n$. The same coefficient holds for nondeterministic circuits, uniformly in the number of witness inputs; for circuits of any basis computing totally regular linear maps over finite fields; for arithmetic circuits over every field of characteristic zero computing the Cauchy map, counting additions and multiplications; and in the presence of $o(n)$ gates of unbounded fan-in computing AND, OR, parity, or fixed-modulus counts. In the average case, every circuit of size $(L-\varepsilon)n$ agrees with a sumset extractor of error b on at most a $1/2+b+2^{-\Omega_\varepsilon(n)}$ fraction of the inputs.

The method is not gate elimination. Lay the wiring graph of a circuit out on a line and cut it anywhere: whatever the inputs on one side tell the computation on the other must cross the cut. A circuit with s gates of fan-in r compiles into a constraint network of maximum degree $r+1$ and cycle rank about $(r-1)s-n$, and each layout of the network is an unambiguous fixed-order read-once program whose states are frontier assignments. Dense rectangle-free sets, such as the fibers of extractors for the sum of two sources, force some frontier to carry $n-o(n)$ values. A layout theorem $\mathrm{cw}(G)\le(A+o(1))\,\beta_1(G)+o(|V(G)|)$ supplies layouts whose frontiers carry about $A((r-1)s-n)$ values. Comparing demand and supply gives $(r-1)s\ge(1+1/A-o(1))\,n$.

We prove the layout theorem for subcubic graphs with $A=\frac{3}{\pi}\arccos\frac{1+2\sqrt{2}}{4}\approx 0.2807$, by listing the vertices of a cubic graph by the median of correlated Gaussian edge scores. In graph-theoretic terms, every cubic graph on h vertices has a linear layout of cutwidth at most $(p+o(1))\,h$, where $p=A/2=0.140350968635\ldots$. This improves the bisection bound of Monien and Preis and the pathwidth bound of Fomin and Høie from $h/6$, and is within $0.0006\,h$ of the best bound known for random cubic graphs. Gaussian vertex scores give coefficients in every fixed degree, including $7n/4-o(n)$ for ternary gates. Run forward as a dynamic program, the same frontiers count the satisfying assignments of a circuit of fan-in two with s gates deterministically in time $O^*(2^{(1/L+\eta)s})$ for every $\eta>0$, where $1/L\approx 0.2192$.

The layout theorem, the exact lower bounds, and the uniform average-case bound are formally verified in Lean 4. The explicit Boolean instantiations use cited sumset-extractor constructions; the counting algorithms are not formalized.

Contents

1 Introduction 1

2 Rectangles 8

3 Sweeps and the frontier counting lemma 10

4 Graphs and constraint networks 13

5 Compiling circuits into networks 15

6 Demand and supply 19

7 Layout bounds 26

8 Linear maps and arithmetic circuits 34

9 Gates of unbounded fan-in 41

10 Counting satisfying assignments 44

11 Decomposable representations 49

12 Design notes on the formalization 51

13 Open problems 53

A Index of formal statements 53

Acknowledgments 56

AI methodology statement 59

1 Introduction

1.1 Results

A *circuit* over a finite alphabet U is a straight-line program: each gate applies an operation $U^a \to U$ of some arity a to inputs and earlier gates. Its *fan-in* is the largest arity. Gates of arity zero are constants, and we never need to count them: we write s for the number of *inner* gates, those of positive arity. A circuit with a designated output *decides* a set $S \subseteq U^n$ if, for some set $\mathrm{Acc} \subseteq U$ of accepting values, an input lies in S exactly when the output lies in Acc. No assumption is made about the available operations: the basis may be any set of operations, including all of them.

A set $S \subseteq U^n$ is *K-rectangle-free* if, however the n coordinates are split into two groups, every combinatorial rectangle contained in S has a side with fewer than K elements (Definition 2.2). Our main theorem reduces circuit lower bounds for such sets to one fact from graph theory, a *layout bound*: graphs of maximum degree d can be laid out on a line so that every cut between an initial segment and the rest is crossed by at most $A\,\beta_1 + o(|V|)$ edges, where $\beta_1 = |E| - |V| + 1$ is the cycle rank (Hypothesis 4.1).

Theorem 1.1 (Main theorem)**.** *Let $r \geq 2$ and $A > 0$, and assume the layout bound $\mathrm{LB}_{r+1}(A)$. Let U be an alphabet with $q \geq 2$ symbols, and for every n let $S_n \subseteq U^n$ be K_n-rectangle-free with*

$$\log K_n = o(n) \qquad \text{and} \qquad \log|S_n| \geq n \log q - o(n).$$

Then for every $\varepsilon > 0$ and all sufficiently large n, every circuit with s inner gates of fan-in at most r that decides S_n, over any basis on U, satisfies

$$(r-1)\,s \;>\; \Big(1 + \frac{1}{A} - \varepsilon\Big)\,n.$$

`Frontier.lowerBound`

Theorem 1.2 (Gaussian layouts)**.** $\mathrm{LB}_3(A)$ *holds for $A = \frac{3}{\pi}\arccos\frac{1+2\sqrt{2}}{4} \approx 0.2807$, and hence for $A = 1/3$.* `Frontier.layoutBound_gaussian`

In graph-theoretic terms, Theorem 1.2 is a statement about cubic graphs alone.

Corollary 1.3 (Cubic graphs)**.** *Let $p = \frac{3}{2\pi}\arccos\frac{1+2\sqrt{2}}{4} \approx 0.14035$. For every $\xi > 0$ and all large h, every cubic graph on h vertices has a linear layout all of whose cuts have at most $(p+\xi)\,h$ edges. Consequently its cutwidth, and hence its pathwidth and its bisection width, are at most $(p+\xi)\,h$.* `Frontier.Gaussian.cubicLayoutBound_gaussian`

The formal statement is the layout; pathwidth is at most cutwidth, and the middle cut of a layout is a bisection. Table 1 places the constant among the known bounds. A loopless cubic multigraph satisfies the same bound through $\mathrm{LB}_3(2p)$, since its cycle rank is $h/2 + 1$.

We call $L = 1 + 1/A$ the *circuit coefficient* of the layout coefficient A. For fan-in two the bound is on the number of gates itself.

Corollary 1.4 (Fan-in two)**.** *Every circuit of fan-in at most two deciding S_n, over any basis, has more than $(L-\varepsilon)\,n$ inner gates for all large n, where $L = 1 + \pi/(3\arccos\frac{1+2\sqrt{2}}{4}) \approx 4.5625$; in particular more than $(4-\varepsilon)n$.* `Frontier.lowerBound_gaussian`

For every fixed fan-in $r \geq 2$, Gaussian vertex scores give

$$(r-1)s > \left(1 + \frac{1}{a_{r+1}} - \varepsilon\right) n, \qquad a_d = \frac{3d}{2d-3}\,\frac{\arccos(2\sqrt{d-1}/d)}{\pi} < \frac{3}{4}$$

Cubic graphs on h vertices	Previous bound	This paper
bisection width, every graph	$h/6$ [MP06]	$p\,h$
pathwidth, every graph	$h/6$ [FH06]	$p\,h$
bisection width, random graphs	$0.139822\,h$, and at least $0.103295\,h$ [LM23]	$p\,h$

Table 1: Width parameters of cubic graphs, up to $o(h)$, with the exact coefficient p of Corollary 1.3. The last row compares our bound, which holds for every cubic graph, with the stronger upper bound for random cubic graphs.

(Theorem 7.12 and Corollary 7.13). In particular, $a_4 = 2/5$ gives $s > (7/4 - \varepsilon)n$ for ternary gates. The median-edge coefficient above is sharper for binary gates. A spanning-tree argument also gives the simpler baseline $(r-1)s > (2-\varepsilon)n$ (Theorem 7.9 and Corollary 7.10).

Nondeterministic circuits. The same lower bound holds for verifiers $C(x, y)$ with n ordinary inputs and any number of existential witness inputs, counting all positive-arity verifier gates. The threshold for n is independent of the witness count (Theorem 6.5). In particular the explicit language below has the same nondeterministic lower bound. More precisely, a binary verifier of size $s \le n^D$, for fixed D, satisfies $s > (L-\varepsilon)n + r$, where r counts distinct witness inputs reachable from its output (Corollary 6.6).

Explicit sets. Over an alphabet with an addition, a rectangle is a sumset, so the fibers of a function that is never constant on the sum $A + B$ of two large sets of inputs are rectangle-free (Lemma 2.7). Extractors for the sum of two independent sources are such functions, and their fibers are dense. Explicit extractors for the sum of two sources were constructed by Chattopadhyay and Liao [CL22], and Li [Li23] brought their min-entropy down to $c \log n$, with polynomial-time evaluation.

Corollary 1.5 (An explicit language)**.** *There is a language in* P *such that for every $\varepsilon > 0$ and all large n, every circuit of fan-in two deciding it on inputs of length n, over any basis of Boolean operations, has more than $(L-\varepsilon)\,n$ inner gates, where $L \approx 4.5625$.*

`Frontier.lowerBound_sumsetDisperser`

The formal statement (Theorem 2.8) applies to every balanced family of sumset dispersers with $\log K_n = o(n)$; that Li's extractor is one is a citation, not part of the formalization.

Average-case hardness. If the hard function is a sumset extractor of error b_n, the weighted frontier lemma gives agreement at most $1/2 + b_n + 2^{-\Omega_\varepsilon(n)}$ under uniform inputs for every circuit of size at most $(L-\varepsilon)n$ (Theorem 6.8 and Corollary 6.9). This is a bound for deterministic circuits; its proof uses uniqueness of the compiled trace. The ordinary bias $2b_n$ corresponds to prediction advantage b_n. The polynomial-error extractor of Chattopadhyay and Liao gives an explicit language in P with agreement at most $1/2 + n^{-\Omega(1)}$ in this range (Corollary 6.10). Its polylogarithmic entropy requirement is still $o(n)$. The weighted form also handles arbitrary input distributions through their min-entropy and permits exceptional inputs charged to a shared measure (Theorem 6.7).

Linear maps and arithmetic circuits. Nothing in the method is specific to deciding a set. For a circuit computing a linear map $x \mapsto Mx$ the demand comes from linear algebra: if M is *totally regular*, that is, every square submatrix is nonsingular, every layout has a frontier whose values determine the input (Lemma 8.3).

Theorem 1.6 (Linear maps)**.** *Assume* $\mathrm{LB}_{r+1}(A)$*. For every* $\varepsilon > 0$ *and all large* N*, every circuit with* s *inner gates of fan-in at most* r *computing* $x \mapsto Mx$ *for a totally regular* $M \in \mathbb{F}^{N\times N}$ *has* $(r-1)s > (1 + 1/A - \varepsilon)N$*, if either* $\mathbb{F}$ *is finite and the basis is arbitrary, or* $\mathbb{F}$ *is infinite and the operations are polynomials, of any degree and with any coefficients. The threshold for* N *does not depend on* $\mathbb{F}$*.* `Frontier.lowerBound_linear_finite`, `Frontier.lowerBound_linear_polynomial`

Corollary 1.7 (Cauchy maps)**.** *For every* $\varepsilon > 0$ *and all sufficiently large* N*, uniformly in the field, the following holds. Over every field of characteristic zero, such as* $\mathbb{Q}$*,* $\mathbb{R}$*, or* $\mathbb{C}$*, every circuit over* $\{+, \times, \textit{constants}\}$ *computing* $x \mapsto \left(\sum_j x_j/(i - N - j)\right)_{i<N}$ *uses more than* $(L - \varepsilon)N$ *additions and multiplications, constants being free. Over* $\mathbb{Z}/q$*, for every prime* $q \geq 2N$*, every circuit of fan-in two over any basis computing the same map has more than* $(L - \varepsilon)N$ *inner gates.*
`Frontier.arith_cauchy_gaussian`

Subtraction and scalar-multiplication gates are also covered by Theorem 1.6, with each such gate counted as one positive-arity gate.

Gates of unbounded fan-in. A gate of fan-in k costs $k - 1$ in the cycle rank, so the method cannot afford gates of unbounded fan-in in general. A gate that *aggregates*, computing a function of a product of contributions of its arguments in a commutative monoid, can however be removed from the graph: guess its output, and check the guess against the product of the contributions, accumulated along the layout in one global register, a *ledger* (Section 9).

Theorem 1.8 (Aggregate gates)**.** *In Theorem 1.1, let the circuit have* s_{ord} *ordinary inner gates of fan-in at most* r *and* k *special gates of any fan-in that aggregate in a finite commutative monoid* T*, with* $k \log|T| = o(n)$*. Then* $(r-1)s_{\mathrm{ord}} > (1 + 1/A - \varepsilon)n$*. In particular* $o(n)$ *gates computing unbounded AND, OR, parity, or fixed-modulus counts are free.* `Frontier.lowerBound_aggregate`

Counting. Run forward as a dynamic program over frontier values, a layout of width w counts the satisfying assignments of a bounded-degree network in time $O^*(q^{w+O(1)})$ (Section 10). Here O^* suppresses polynomial factors in the number of inputs and the length of the circuit or network encoding. For each fixed accuracy parameter, the constants may depend on that parameter; the Gaussian cubic-layout construction has an absolute polynomial degree.

Theorem 1.9 (Counting satisfying assignments)**.** *For every* $\eta > 0$*, the number of satisfying assignments of a Boolean circuit of fan-in two with* s *inner gates can be computed deterministically in time and space* $O^*(2^{(A/(1+A)+\eta)s})$*, where* $A \approx 0.2807$ *is the coefficient of Theorem 1.2 and* $A/(1 + A) \approx 0.219178$*.*

The lower-bound coefficient and the counting exponent are reciprocals:

$$L = 1 + \frac{1}{A}, \qquad \alpha = \frac{A}{1+A} = \frac{1}{L}.$$

The stronger input-sensitive bound is $O^*(2^{\min\{m, A(s-m)\}+\eta s})$ for m reachable inputs (Theorem 10.8). With a finite ledger, the bound becomes $O^*(|M|^{O(1)} 2^{(1/L+\eta)s_{\mathrm{ord}}})$ (Theorem 10.11). The deterministic layout algorithm uses fixed-accuracy local discretization and conditional expectations (Lemma 10.7). Its time on an h-vertex cubic graph is $f(\xi)h^3(\log(h + 2))^{O(1)}$, with an absolute polynomial degree. Direct sampling also gives an expected-polynomial-time layout algorithm, with the same counting exponent. Using the earlier coefficient $A = 1/3$ gives the simpler deterministic exponent $1/4$.

Two further consequences. The frontier that determines the input of a totally regular linear map does so for every finite-alphabet map whose graph code is MDS, with nonlinear internal gates allowed and the same coefficients (Theorem 8.18). And the charging argument behind the counting lemma applies to decomposable DAGs: a smooth DNNF representing a K-rectangle-free set S has at least $|S|/(K-1)^3$ nodes and edges, hence size $2^{n-O(\log n)}$ for the explicit language above (Theorem 11.1).

1.2 Comparison with previous work

Circuit lower bounds for explicit functions. Over the full binary basis, the known lower bounds for explicit functions come from gate elimination: $2n$ by Schnorr [Sch74], $2.5n$ by Paul [Pau77], $3n-o(n)$ by Blum [Blu84], with a simpler proof for affine dispersers by Demenkov and Kulikov [DK11], $(3+1/86)n-o(n)$ by Find, Golovnev, Hirsch, and Kulikov [FGHK16], and $3.1n-o(n)$ by Li and Yang [LY22]. Over the basis U_2, which lacks parity, Iwama and Morizumi [IM02] proved $5n-o(n)$. Since the frontier method applies to every basis at once, its coefficient lies between the two: above every bound known for the full binary basis, below the U_2 bound. Gate elimination simplifies a circuit by fixing inputs, and charges each restriction to the gates it removes. The frontier method never simplifies the circuit: it charges a circuit for the cycle rank of its wiring graph and asks only one combinatorial property of the hard function, rectangle-freeness. It applies verbatim to every basis, and the coefficient is determined by graph theory alone.

Read-once programs and extractors. The counting lemma at the heart of the method (Lemma 3.2) is a lower bound for unambiguous read-once programs (Remark 3.3). For deterministic read-once branching programs, Andreev, Baskakov, Clementi, and Rolim [ABCR99] gave a function in P requiring size $2^{n-O(\log^2 n)}$; for nondeterministic read-k programs, Borodin, Razborov, and Smolensky [BRS93] proved exponential lower bounds. Sumset sources were introduced by Chattopadhyay and Li [CL16]; extractors for the sum of two sources were constructed by Chattopadhyay and Liao [CL22], building on the two-source extractors of Chattopadhyay and Zuckerman [CZ19], and brought to min-entropy $O(\log n)$ by Li [Li23]. Chattopadhyay and Liao [CL23] showed that sumset extractors are hard for strongly read-once linear branching programs, a model of Gryaznov, Pudlák, and Talebanfard [GPT22]; see also Li and Zhong [LZ24]. This includes average-case hardness: the extractor-to-hardness connection is already present in these works; see also [Li23, Theorem 7.27]. Our weighted extension uses disjoint rectangles in an unambiguous fixed-order program and the circuit-to-layout conversion to obtain a circuit-size bound.

Cuts, width, and layouts. Converting circuits into ordered decision diagrams through small cuts goes back at least to Berman [Ber91], who obtained at most $nm2^{w(C)}$ nodes for a circuit with n inputs, m outputs, and circuit width $w(C)$. This width is tied to an evaluation order. Morizumi [Mor23] proposed converting circuits into nondeterministic bounded-width circuits and branching programs as an approach to circuit lower bounds.

The closest prior compilation result is due to Amarilli, Capelli, Monet, and Senellart [ACMS19, Theorem 4.4]: given a circuit over $\{\wedge,\vee,\neg\}$ and a path decomposition of width k, they construct an equivalent complete unambiguous OBDD of width at most $2^{2(k+1)}$, in time $O(|P|f(k))$ with $f(k)=O(2^{(2+\epsilon)k})$ for every $\epsilon>0$. Their pathwidth formulation already permits orders unrelated to circuit evaluation. Our refinement in Proposition 5.3 records one alphabet value per frontier edge of a bounded-degree constraint network: a cut of size w has at most q^w states, and the program, retaining its guess layers, has at most $|V|q^{w+r+2}+nq$ edges. The compiler also gives the exact accounting $\beta_1 \le (r-1)s-m+1$, where m is the number of reachable inputs (Proposition 5.5).

Thus a cycle-rank layout bound gives frontier width $A((r-1)s-m)+o(s)$ for fixed fan-in. The frontier count and the credit for reachable inputs are the quantitative ingredients used here.

Relations between treewidth, pathwidth, and cutwidth are due to Korach and Solel [KS93]. Fomin and Høie [FH06] proved that cubic graphs on h vertices have pathwidth $(1/6+o(1))h$, through the bisection bound of Monien and Preis [MP06]; this gives the layout coefficient $A = 1/3$ and the circuit coefficient 4. Our Gaussian layouts improve the coefficient to $A \approx 0.2807$; their correlation $2\sqrt{2}/3$ is that of Gaussian waves on the 3-regular tree [CGHV15, Lyo17]. In graph-theoretic terms, Corollary 1.3 bounds the cutwidth of every cubic graph by $(p+o(1))h$, with the exact p defined there; Table 1 compares this with the earlier bounds.

Linear maps. Valiant [Val77] observed that the graph of a linear circuit computing a totally regular map is a superconcentrator. Lev and Valiant [LV83] proved that a superconcentrator of in-degree two needs $4N - o(N)$ computed nodes, excluding inputs and including computed outputs. The precise bound $4N - O(\sqrt{N \log N})$ is [Lev80, Theorem 3.2, p. 61]; Lev credits the result jointly to Valiant. Their separate bound of $5N - o(N)$ edges has no degree restriction [Lev80, Theorem 3.3]. The computed-node theorem gives the corresponding bound on binary operations in this model. Our coefficient is $L \approx 4.5625$, counting positive-arity gates, with arbitrary affine coefficients allowed and constants free. Superlinear lower bounds for unrestricted linear circuits are known for matrices whose entries generate number fields of large degree; see Lokam's survey [Lok09]. Lemma 8.3 plays the role of the superconcentrator property, at a single frontier of a layout rather than through vertex-disjoint paths, and the circuit is charged for its cycle rank rather than its edges.

Satisfiability algorithms. Circuit SAT over the full binary basis with s gates can be solved in time $2^{0.4058s}$, by an algorithm of Nurk [Nur09]. Theorem 1.9 counts satisfying assignments, over any basis, with exponent $1/L \approx 0.2192$; even the pathwidth bound of Fomin and Høie, applied through the compression of Section 7.1, gives exponent $1/4$. The constants hidden by O^* are large. Golovnev, Kulikov, Smal, and Tamaki [GKST16] gave a framework in which gate elimination yields both lower bounds and algorithms counting satisfying assignments; in particular, they count in time faster than 2^n for full-binary-basis circuits of size at most $(3-\varepsilon)n$ for every fixed $\varepsilon > 0$ [GKST16, Section 5.2, Remark 2]. This input-relative regime differs from a bound expressed only in the gate-count exponent. Our bound gives faster-than-2^n counting for every fixed gate/input ratio below L, with polynomial factors in the instance size and exponential space. Williams [Wil13] showed that improving exhaustive search implies superpolynomial lower bounds for larger classes. Theorem 1.9 is the algorithmic counterpart of the frontier method: the frontiers that are small in the lower bound are enumerated by the algorithm.

1.3 Technical overview

Three ideas carry the proof of Theorem 1.1.

Gates create local constraints. Forget that a circuit computes. Put a value on every wire and ask only that each gate's output value equal its operation applied to its input values. For a fixed input, the run of the circuit is the unique assignment satisfying all of these local equations. The equations are *relations*: a gate $z = g(x, y)$ is the set of triples (x, y, z) with $z = g(x, y)$, and nothing ever needs to invert g.

Fan-out creates topology. To make every constraint see a bounded number of values, a wire read by many gates is routed through a chain of copy vertices, one per reader. The resulting constraint

graph has maximum degree $r+1$, and its cycle rank is about $(r-1)s-n$ (Proposition 5.5). A constant-free read-once binary formula on n distinct inputs compiles to a tree. Reusing an input or an intermediate signal contributes to the topology measured by the cycle rank.

Frontier values permit splicing. Lay the constraint graph out on a line, in any order whatsoever, unrelated to the order of evaluation. After the first t vertices, the edges between the processed and the unprocessed vertices form the *frontier*. Once the values on the frontier are fixed, the two sides of the constraint system are independent (Lemma 4.3). The frontier assignment is a boundary condition on a globally consistent run; it may depend on inputs on either side. The resulting messages form a sweep. For the compiled network, local consistency also gives an unambiguous read-once program (Proposition 5.3); Fig. 1 illustrates a cut unrelated to evaluation order.

Demand. A set S with such a sweep cannot be both large and rectangle-free unless some message is long. The *frontier counting lemma* (Lemma 3.2) makes this exact: charge each input to the first time at which the inputs sharing its message have at least K distinct revealed parts. The rectangular hull of those inputs lies inside S, so they have fewer than K distinct unrevealed parts, while just before, they still had fewer than K revealed parts. An input is therefore determined by its transition and by two parts with fewer than K values each, so $|S| \le (K-1)^2$ times the number of transitions. For a dense set $\log_q |S| \ge n - o(n)$, so some frontier carries $n - o(n)$ values.

Supply, and the comparison. The layout bound applied to the constraint graph gives a layout whose frontiers have at most about $A\beta_1 \approx A((r-1)s-n)$ edges. Demand cannot exceed supply: $n \le A((r-1)s-n) + o(n)$, that is, $(r-1)s \ge (1+1/A-o(1))n$. One lemma (Lemma 6.1) makes this comparison, with every lower-order term, for all three lower bounds of this paper.

Layouts from Gaussian scores. Theorem 1.2 is proved in three steps (Section 7). Compressing a subcubic multigraph to its cubic core costs a factor two, since the core has at most $2\beta_1$ vertices. A cubic graph is laid out by giving every edge a real score and listing the vertices by the median of their three scores: every frontier of this *median layout* is bounded by the number of vertices whose scores straddle a threshold, plus a few edges near it (Lemma 7.3). The scores are correlated Gaussians: for a standard Gaussian vector ω, the score of an edge is $\langle y_e, \omega \rangle$ for unit vectors y_e built from distance kernels on the graph, whose neighbouring correlations approach $2\sqrt{2}/3$. By Sheppard's formula two scores with correlation ρ are separated by a threshold with probability at most $\arccos(\rho)/\pi$, and a vertex straddles a threshold with probability at most $\frac{3}{2\pi}\arccos\frac{1+2\sqrt{2}}{4} \approx 0.1404$. The counts concentrate, because every event depends on a bounded neighbourhood.

Other demands. For a linear map, the compiler gives every output a vertex of its own. Splicing the runs on two inputs with the same frontier values shows that the outputs on each side of the frontier do not see the inputs on the other side; for a totally regular map, at the moment when the inputs read and the outputs held add up to N, this forces the frontier values to determine the input, so the frontier carries N field elements (Lemma 8.3). Over a finite field this is counting; over an infinite field, where a single element can encode anything, it needs the operations to be polynomials, and then the circuit can be replaced by its linearization at the origin (Lemma 8.7). For aggregate gates, the global register is a single extra message, and the only new difficulty is that removing the gates may disconnect the graph: the counting lemma itself then finds one component that reads almost all inputs (Theorem 9.7).

1.4 Formal verification

The exact circuit lower bounds, including the nondeterministic extension, their arithmetic instantiations, the signal-capacity inequality, the layout theorem, and the uniform average-case bound from rectangle or sumset bias, including its randomized extension, are formalized in Lean 4 [dMU21] (toolchain `v4.35.0-rc3`), using Mathlib [The20] (commit `9a6fbe0`) and CSLib [The26] (commit `89dc6fb`), in about 16,400 lines including documentation, available at

`https://github.com/SamuelSchlesinger/the-frontier-method`.

The source base commit for this PDF is `1d0be16e93d7` (modified working tree). If marked as a modified working tree, this version also requires the accompanying local source changes; the commit link alone recovers only the base version. The circuit model is CSLib's: signatures, interpretations, straight-line programs, and circuits. Formalized results below are tagged with their Lean declarations. These declarations depend only on Lean's standard axioms (`propext`, `Classical.choice`, `Quot.sound`); there is no `sorry`. The formal statements make their circuit basis, alphabet, and family hypotheses explicit. The extractor constructions and their polynomial-time evaluation behind Corollaries 1.5 and 6.10 are cited, not formalized: Lean proves the bounds for every family satisfying the respective disperser or flat-source bias hypothesis. The weighted inequality for coherent sweeps (Proposition 3.5) and the uniform average-case circuit theorem (Theorem 6.8) are checked, as are the general-degree layout and higher-fan-in bounds of Section 7.5, the decomposable threshold bound of Theorem 11.1, and the mixed-fiber inequality and nonlinear MDS bound of Proposition 8.17 and Theorem 8.18. The mixed-Hessian frontier lemma and its scalar-polynomial lower-bound transfer in Section 8.4 are also checked, including their instantiation on compiled circuits, the all-input-order determinant demand, and the determinant bound of Corollary 8.16. The independent-randomization transfer and the circuit specialization in Section 6.6 are also checked; the general transfer permits arbitrary measurable correctness relations and input and seed probability spaces. The repository also contains checked refinements of the boundary capacity that this paper does not use; its file `RESEARCH.md` records them. The explicit branching-program model and rectangle partition, the packing and arbitrary-distribution circuit statements (Lemma 3.6 and Theorems 3.7 and 6.7), the random-set existence argument, the DNNF smoothing conversion, the cubic-tree sharpness calculation, and all algorithms and complexity bounds in Section 10, including derandomization, are outside the formalization. From the indicated snapshot, with any accompanying local source changes applied, the verification commands are

```
lake exe cache get
lake build --wfail
lake env lean scripts/Audit.lean
python3 scripts/comparator.py self-test
python3 scripts/comparator.py check
python3 scripts/check_algorithms.py
python3 scripts/check_communication.py
./scripts/build-paper.sh
```

The audit checks the exact axiom lists of the principal results; a changed list fails the check. The Comparator harness [Lea] additionally compares every public theorem statement, and the definitions it uses, against an immutable reference commit. It checks the axiom dependencies of all library theorem proofs, including private lemmas, and replays their exported terms through Lean's kernel. Candidate compilation is isolated from the reference sources. Its adversarial tests must reject weakened statements, altered definitions, forbidden axioms, and invalid proof terms. The

checked-in policy identifies the reference and tool revisions; the generated report records coverage and source hashes. This verifies the formal propositions relative to the reference and permitted axioms; reviewing their mathematical meaning remains necessary. The README lists further finite checks that run in continuous integration. The Python script checks the reference edge, signal, and ledger counting procedures against exhaustive evaluation on small circuits, checks local conditional moments against full enumeration, and checks rectangle peeling and threshold charges in an unambiguous program with guess layers, asymmetric thresholds, and nonuniform weights. These are finite correctness checks, not proofs of the asymptotic running times. The communication script additionally checks capped pruning on compiled traces, exhaustive codebook optimization, dependent conditional moments, and a shared-selector obstruction to marginal-entropy shortcuts. Section 12 records the design decisions behind the formal statements, and Section A is an index from the paper to the code.

1.5 Organization

Sections 2 and 3 contain the combinatorics of rectangles and the frontier counting lemma, with no graphs or circuits. Section 4 introduces constraint networks and sweeps along layouts, and Section 5 compiles circuits into networks. Section 6 compares demand and supply and proves Theorem 1.1, with its nondeterministic and average-case variants, and Section 7 proves the binary and higher-fan-in layout bounds. Sections 8 and 9 treat linear and MDS maps, scalar polynomials through mixed Hessians, and aggregate gates, Section 10 the counting algorithms, and Section 11 decomposable representations. Section 12 discusses the formalization and Section 13 open problems; Section A is an index from the paper to the code.

2 Rectangles

2.1 Rectangle-free sets

Throughout, ι is a finite set of *coordinates* and U a finite *alphabet* with $q = |U|$ symbols. An *input* is a function $x\colon \iota \to U$. For $X \subseteq \iota$ we write $x|_X$ for the restriction of x to X, its *part* on X, and X^c for $\iota \setminus X$. An input is determined by its two parts $x|_X$ and $x|_{X^c}$.

Definition 2.1 (Rectangles)**.** Let $X \subseteq \iota$, $A \subseteq U^X$ and $B \subseteq U^{X^c}$. The *rectangle* with sides A and B is

$$\mathcal{R}_X(A, B) = \{\, x \in U^\iota : x|_X \in A \text{ and } x|_{X^c} \in B \,\}.$$

`Frontier.rectangle`

Definition 2.2 (Rectangle-free sets)**.** A set $S \subseteq U^\iota$ is *K-rectangle-free* if for every $X \subseteq \iota$ and all $A \subseteq U^X$, $B \subseteq U^{X^c}$ with $\mathcal{R}_X(A, B) \subseteq S$, either $|A| < K$ or $|B| < K$. `Frontier.RectangleFree`

In the language of communication complexity: for every way of dividing the coordinates between two players, every rectangle on which the indicator of S is constantly 1 has a side with fewer than K rows or columns. Taking $A = B = \emptyset$ shows $K \geq 1$ (`Frontier.RectangleFree.pos`); if $S \neq \emptyset$, a single input is a rectangle with sides of size one, so $K \geq 2$.

Proposition 2.3 (Random sets; not formalized)**.** *A uniformly random set $S \subseteq \{0,1\}^n$ is $(n+2)$-rectangle-free with probability at least $1 - 2^{-n-4}$.*

Proof. A rectangle in S with both sides of size at least $K = n + 2$ contains one with both sides of size exactly K. There are 2^n choices of X, and at most $2^{|X|K}\, 2^{(n-|X|)K} = 2^{nK}$ choices of the two sides; each of the resulting K^2 inputs lies in S independently with probability $1/2$. The expected number of such rectangles is at most $2^{n+nK-K^2} = 2^{-n-4}$. □

The rest of the argument uses rectangle-freeness once, inside the counting lemma of Section 3, and once more in the following form: a dense rectangle-free set cannot ignore many coordinates. A set S *depends only on* $R \subseteq \iota$ if membership in S is determined by the part on R.

Lemma 2.4 (Ignored coordinates)**.** *Let S be K-rectangle-free and depend only on D^c, where $K \le q^{|D|}$. Then $|S| \le q^{|D|}(K-1)$.* `Frontier.RectangleFree.ncard_le_of_dependsOn`

Proof. Let $A = U^D$ be every assignment to D, and $B = \{x|_{D^c} : x \in S\}$. If x lies in $\mathcal{R}_D(A, B)$, then x agrees off D with some element of S, so $x \in S$. Since $|A| = q^{|D|} \ge K$, rectangle-freeness gives $|B| \le K - 1$. Finally $x \mapsto (x|_D, x|_{D^c})$ maps S injectively into $A \times B$. □

Lemma 2.5 (Support lemma)**.** *Let S be K-rectangle-free with $|S| \ge qK^2$, and suppose S depends only on R. Then $q^{|R^c|} < K$, that is, S depends on all but fewer than $\log_q K$ coordinates.*
`Frontier.RectangleFree.pow_ncard_compl_lt`

Proof. Suppose $q^{|R^c|} \ge K$, and let d be the least integer with $q^d \ge K$; then $d \le |R^c|$. Choose $D \subseteq R^c$ with $|D| = d$. Then S depends only on $D^c \supseteq R$, so Lemma 2.4 gives $|S| \le q^d(K-1)$. If $d = 0$ this is $K - 1 < qK^2$. If $d > 0$ then $q^{d-1} \le K - 1$ by minimality, so $|S| \le q(K-1)^2 < qK^2$. Either way this contradicts $|S| \ge qK^2$. □

2.2 Sumset dispersers

Let the alphabet U carry an addition with a two-sided identity 0. For $X \subseteq \iota$ and $a \in U^X$, let $\hat{a} \in U^\iota$ agree with a on X and vanish elsewhere. An input x with parts $a = x|_X$ and $b = x|_{X^c}$ is then the sum $\hat{a} + \hat{b}$, so the rectangle $\mathcal{R}_X(A, B)$ is the sumset $\hat{A} + \hat{B}$, where $|\hat{A}| = |A|$ and $|\hat{B}| = |B|$.

Definition 2.6 (Sumset dispersers)**.** A function $f\colon U^\iota \to \{0,1\}$ *disperses sumsets of size K* if it takes both values on $A + B$ whenever $A, B \subseteq U^\iota$ have at least K elements each.
`Frontier.SumsetDisperser`

Lemma 2.7 (Dispersers have rectangle-free fibers)**.** *If f disperses sumsets of size K, then $f^{-1}(0)$ and $f^{-1}(1)$ are K-rectangle-free.* `Frontier.SumsetDisperser.rectangleFree`

Proof. A rectangle with sides of size at least K inside a fiber is a sumset of two sets of size at least K on which f is constant. □

For the rest of this paragraph, let U be a finite additive group. A *sumset extractor* with error $\nu < 1/2$ for min-entropy k outputs a nearly uniform bit on $X + Y$ for independent random inputs X and Y of min-entropy at least k. The uniform distributions on two sets of size 2^k are such inputs, so an extractor disperses sumsets of size 2^k; and with X and Y uniform on all of U^ι, the sum $X + Y$ is uniform, so the fiber $f^{-1}(1)$ contains at least a $(1/2 - \nu)$ fraction of all inputs.

Theorem 2.8 (Lower bounds for sumset dispersers)**.** *Let $f_n\colon U^n \to \{0,1\}$ disperse sumsets of size K_n, with $\log K_n = o(n)$, and let $f_n^{-1}(1)$ contain at least a fixed fraction $\delta > 0$ of all inputs. Then for every $\varepsilon > 0$ and all large n, every circuit of fan-in two over any basis on U deciding $f_n^{-1}(1)$ has more than $(L - \varepsilon)\, n$ inner gates, with $L \approx 4.5625$.* `Frontier.lowerBound_sumsetDisperser`

Proof. The fibers are rectangle-free by Lemma 2.7, and dense: $n \log q - \log|f_n^{-1}(1)| \le \log(1/\delta)$ (`Frontier.isLittleO_of_card_le`). Apply Corollary 7.8. □

Proof of Corollary 1.5. Li [Li23, Theorem 7.13] constructs, for every $\nu > 0$, a constant c and a polynomial-time computable extractor $f_n \colon \{0,1\}^n \to \{0,1\}$ with error ν for the sum, modulo two, of two independent sources of min-entropy $c \log n$. With $\nu = 1/8$ it disperses sumsets of size $K_n = n^c$, and $f_n^{-1}(1)$ contains at least $3/8$ of all inputs. Theorem 2.8 applies to the language $\{x : f_{|x|}(x) = 1\}$. □

3 Sweeps and the frontier counting lemma

This section is the heart of the method, and it contains no graphs and no circuits.

Definition 3.1 (Sweeps)**.** Let $S \subseteq U^\iota$. A *sweep* of S of length T with messages in a set M consists of sets of *revealed* coordinates $X_0, X_1, \dots, X_T \subseteq \iota$ and *message* maps $\mu_0, \dots, \mu_T \colon U^\iota \to M$ such that

(i) $X_0 = \emptyset$ and $X_T = \iota$;

(ii) the final message μ_T is constant on S;

(iii) *(splicing)* if $x, y \in S$ and $\mu_t(x) = \mu_t(y)$, then the input that agrees with x on X_t and with y on X_t^c lies in S.

The *transition* taken by x at step $t < T$ is

$$\tau_t(x) = \big(\mu_t(x),\ \mu_{t+1}(x),\ x|_{X_{t+1} \setminus X_t}\big) :$$

the messages before and after the step, together with the coordinates revealed during it.

`Frontier.Sweep`

Splicing says that a common message permits recombining the revealed and unrevealed parts of accepted inputs. A message may depend on the whole input; it need not be computable from the revealed part alone. The revealed sets need not be nested; the proof below never uses it.

Lemma 3.2 (Frontier counting lemma)**.** *Let S be K-rectangle-free with $|S| \ge K$. Then for every sweep of S,*

$$|S| \ \le\ (K-1)^2 \sum_{t<T} |\tau_t(S)|.$$

`Frontier.Sweep.ncard_le`

Proof. For $x \in S$ and a time t, the *peers* of x at time t are the inputs $y \in S$ with $\mu_t(y) = \mu_t(x)$. Let $P_t(x)$ be the set of their parts on X_t (the *past side*) and $F_t(x)$ the set of their parts on X_t^c (the *future side*).

The rectangular hull of the peers lies in S. If $z|_{X_t} = y|_{X_t}$ and $z|_{X_t^c} = y'|_{X_t^c}$ for peers y, y' of x, then $\mu_t(y) = \mu_t(y')$, and splicing puts z in S. So $\mathcal{R}_{X_t}(P_t(x), F_t(x)) \subseteq S$, and

$$|P_t(x)| < K \quad \text{or} \quad |F_t(x)| < K. \tag{1}$$

The past side starts small and ends large. At $t = 0$ nothing is revealed and $|P_0(x)| \le 1 < K$ (note $K \ge 2$, as $S \ne \emptyset$). At $t = T$ every input of S is a peer, since μ_T is constant on S, and everything is revealed, so $|P_T(x)| = |S| \ge K$.

Charging. Charge x to the first time $c(x)$ with $|P_{c(x)}(x)| \geq K$; then $1 \leq c(x) \leq T$. Write $c(x) = t+1$. By minimality $|P_t(x)| \leq K-1$, and by (1) at time $t+1$, $|F_{t+1}(x)| \leq K-1$.

Few inputs share a charge and a transition. Fix t and a transition value θ, and let Q be the set of $y \in S$ with $c(y) = t+1$ and $\tau_t(y) = \theta$. Fix $x \in Q$. Every $y \in Q$ has $\mu_t(y) = \mu_t(x)$ and $\mu_{t+1}(y) = \mu_{t+1}(x)$, so $y|_{X_t} \in P_t(x)$ and $y|_{X_{t+1}^c} \in F_{t+1}(x)$. Moreover, every $y \in Q$ has the same part $y|_{X_{t+1} \setminus X_t}$ as x. The three sets X_t, $X_{t+1} \setminus X_t$ and X_{t+1}^c cover ι, so $y \mapsto (y|_{X_t}, y|_{X_{t+1}^c})$ is injective on Q, and $|Q| \leq |P_t(x)|\,|F_{t+1}(x)| \leq (K-1)^2$.

Summing over the charge time $t+1$ and the transition $\theta \in \tau_t(S)$ gives the bound. □

Remark 3.3 (Read-once programs). Consider a layered nondeterministic read-once program with a fixed variable order, a single accepting sink, and layers that either query the designated variable or guess. Choose an accepting path for each accepted input, let X_t be the variables already queried and $\mu_t(x)$ its node at layer t. Recombining accepting paths through a common node gives splicing, hence a sweep. Lemma 3.2 therefore bounds the number of edges of such a program below by $|S|/(K-1)^2$.

The converse fails for abstract sweeps, even if one coordinate is revealed at each step. For example, let $S = \{0,1\}^3 \setminus \{000\}$, with coordinates revealed in order and constant initial and final messages. At time one identify 011 and 101 in a message class a, and at time two identify 101 and 110 in a class b; all other classes are singletons. The two nontrivial rectangular hulls are contained in S, so splicing holds. But the union of the selected transitions has a path labelled 000: its successive edges are witnessed by 011, 101, and 110. Selecting one path per input does not rule out additional paths. For compiled circuits, a direct construction from local constraints gives an unambiguous program (Proposition 5.3). This discussion is not formalized.

Remark 3.4 (Why charge at a threshold). There is no a priori single cut at which both sides of every message class are small: the past side grows and the future side shrinks as t increases, at rates that differ from input to input. Charging each input at the first time its past side reaches K catches it at a moment when the previous past side and the current future side are both below K. The transition records exactly what is needed to pass between the two times.

3.1 A weighted approximate frontier lemma

For average-case bounds, deleting a message class must preserve splicing on exactly the remaining inputs. The abstract sweep axiom alone does not guarantee this. We call a sweep *coherent* if splicing x, y at time t, when their messages agree, also splices their whole message traces: the resulting input z has $\mu_u(z) = \mu_u(x)$ for $u \leq t$ and $\mu_u(z) = \mu_u(y)$ for $u \geq t$. Compiled circuits have this stronger property because their satisfying edge assignments are unique. The formal proof uses the following interface.

Proposition 3.5 (Weighted coherent sweeps)**.** *Let P be a coherent sweep of $S \subseteq U^n$ of positive length, with U finite. Fix $K_L, K_R \geq 2$, write $B = (K_L-1)(K_R-1)$, and let D be the sum of the numbers of used transitions over its steps. Call a coordinate rectangle* thick *when its two sides have at least K_L and K_R elements, respectively. Let $w : U^n \to \mathbb{R}$, $c : U^n \to [0,\infty)$, and $a \geq 0$ satisfy $|w(x)| \leq a$ and*

$$\left|\sum_{x \in R} w(x)\right| \leq \sum_{x \in R} c(x) \qquad \text{for every thick rectangle } R.$$

Then

$$\left|\sum_{x \in S} w(x)\right| \leq \sum_{x \in S} c(x) + aBD.$$

Proof. Coherence makes each message class equal to its rectangular hull: splicing two peers preserves their message at the splice time. Delete any thick class R at time t. The sweep restricted to $S \setminus R$ is still coherent. Indeed, splice two remaining inputs at time u. Its message at t comes from the first input if $t \le u$, and from the second if $u \le t$, so it avoids the deleted class. Restriction cannot increase D. Repeat; the deleted rectangles are disjoint.

In the residual sweep every class has past side smaller than K_L or future side smaller than K_R. If the residual set has at least K_L inputs, the threshold-charging proof of Lemma 3.2, with past threshold K_L and future threshold K_R, bounds its size by BD: at a charged transition the previous past side has at most $K_L - 1$ elements and the next future side at most $K_R - 1$. If it has fewer than K_L inputs, the same bound holds because a nonempty sweep of positive length has $D \ge 1$. Sum the bounds on the deleted rectangles and use $|w| \le a$ on the residue. Nonnegativity of c allows its sum over the deleted rectangles to be bounded by its sum over S. □

For example, taking $w(x) = f(x)\mu(x)$, $c(x) = \beta\mu(x) + \xi(x)$ and $a = \max_x \mu(x)$ handles an arbitrary input distribution and a shared nonnegative error measure ξ. The following program formulation also records the rectangle partition explicitly. Unambiguity makes state deletion remove exactly the inputs in its rectangle.

Lemma 3.6 (Weighted approximate frontier lemma; not formalized)**.** *Let P be an unambiguous layered read-once program with a fixed variable order, one accepting sink, and D edges. Every accepting path queries all $n \ge 1$ input coordinates, with additional guess layers allowed. Let $S \subseteq U^n$ be its accepted set, where U is a finite alphabet. Fix $K_L, K_R \ge 2$ and put $B = (K_L - 1)(K_R - 1)$. Call a coordinate rectangle* thick *if its past side has at least K_L elements and its future side has at least K_R elements. Then S is the disjoint union of thick rectangles, possibly at different coordinate splits, and a residual set of size at most BD.*

Consequently, if $w : U^n \to [-1, 1]$ and $\beta \ge 0$ satisfy

$$\left|\sum_{x \in R} w(x)\right| \le \beta|R|$$

for every thick coordinate rectangle R, then

$$\left|\sum_{x \in S} w(x)\right| \le \beta|S| + BD.$$

Proof. Discard states not lying on an accepting path. At a state v in layer t, let A_v be the set of input prefixes on paths from the start to v, and B_v the set of suffixes on paths from v to the sink. Fixed order makes the inputs having an accepting path through v exactly the rectangle $R_v = A_v \times B_v$, for the coordinate partition at that layer. If $|A_v| \ge K_L$ and $|B_v| \ge K_R$, record R_v and delete v and its incident edges. By unambiguity, no input of R_v has another accepting path avoiding v. Thus the remaining accepted set is exactly the old one minus R_v, and the remaining program is still unambiguous. Repeat until there is no such state. The recorded rectangles are disjoint, and deletion never increases the number of edges.

Let S' be the residual accepted set. If it is empty there is nothing to prove. Otherwise $D \ge 1$, since $n \ge 1$. If $|S'| < K_L$, it has at most $K_L - 1 \le BD$ elements. Otherwise, on the unique accepting path of each $x \in S'$, charge x to the edge $u \to v$ at which $|A_v|$ first reaches K_L. The start has one prefix and the sink has $|S'|$ prefixes, so the edge exists. Minimality gives $|A_u| < K_L$; termination

of the deletion process gives $|B_v| < K_R$. For a fixed charged edge, an input is determined by its prefix at u, its suffix at v, and the edge label (a queried symbol or a guess). Hence at most B inputs charge to it, exactly as in Lemma 3.2. This bounds $|S'|$ by BD in the second case. Finally sum the rectangle bounds over the disjoint pieces and use $|w(x)| \le 1$ on S'. □

The weight need not be Boolean. For a Boolean target f, the relevant weight is $w(x) = (-1)^{f(x)}$. If f has bias at most $2b$ on every large rectangle, apply the lemma with $K_L = K_R = K$ and $\beta = 2b$ separately to the two output classes of a circuit. This bounds prediction agreement by $1/2 + b$ plus half the residual fraction. A cover with overlapping rectangles would not justify this summation; the state-deletion argument supplies a partition.

Theorem 3.7 (Weighted distributions and a shared error budget; not formalized)**.** *Let $f, g : U^n \to \{-1, 1\}$ and let μ be any probability measure on U^n, where $q = |U| \ge 2$ and $n \ge 1$. Define*

$$\mathrm{Pack}_{K_L,K_R}(f,\mu) = \sup_{(R_j)} \sum_j \left| \int_{R_j} f \, d\mu \right|, \qquad h_q = -\log_q \max_x \mu(x),$$

where the supremum is over pairwise disjoint thick coordinate rectangles; their splits may differ. Since $|f| = 1$, this parameter lies in $[0, 1]$. Suppose each output class of g has an unambiguous program as in Lemma 3.6, with a total of D edges between the two programs. Then

$$|\mathbb{E}_\mu[fg]| \le \mathrm{Pack}_{K_L,K_R}(f,\mu) + (K_L - 1)(K_R - 1)Dq^{-h_q}.$$

In particular, if $\beta \ge 0$ and a nonnegative measure ξ of mass ζ satisfy

$$\left| \int_R f \, d\mu \right| \le \beta\mu(R) + \xi(R)$$

for every thick rectangle R, then the packing term is at most $\beta + \zeta$.

Proof. Peel the two output classes separately. All resulting rectangles are disjoint, including across classes, and g is constant on each. Their total absolute contribution is therefore at most the packing term. The leftover has at most $(K_L - 1)(K_R - 1)D$ points and mass at most its cardinality times $\max_x \mu(x) = q^{-h_q}$. For the last assertion, sum the rectangle inequality: disjointness bounds the total μ-mass by one and the total ξ-mass by ζ. □

No product structure is assumed for μ. For example, let E be a set of exceptional inputs with $\mu(E) = \zeta$. If every thick rectangle satisfies $|\int_{R \setminus E} f \, d\mu| \le \beta\mu(R \setminus E)$, take ξ to be the restriction of μ to E. The triangle inequality gives the hypothesis of the theorem, and the exceptional mass is charged only once over all peeled rectangles.

4 Graphs and constraint networks

4.1 Multigraphs, layouts, and the layout bound

A *multigraph* G has vertices V, edges E, and two endpoints for every edge; parallel edges are distinct, and the orientation of an edge plays no role (`Frontier.Multigraph`). The *cut* $\delta(L)$ of a set L of vertices is the set of edges with exactly one endpoint in L. A *layout* is a numbering $\pi\colon V \to \{0, \dots, |V|-1\}$; the first t vertices form the *initial segment* $P_t = \{v : \pi(v) < t\}$, and $C_t = \delta(P_t)$ is the *frontier* at time t. The *width* of π is $\max_t |C_t|$, and the *cutwidth* $\mathrm{cw}(G)$ is the least width of a layout. For a nonempty connected graph, the *cycle rank* is $\beta_1(G) = |E| - |V| + 1$, the number of independent

cycles. The formal quantity `Frontier.Multigraph.cycleRank` is $\max\{0, |E| - |V| + 1\}$ on all graphs; we use it only on connected graphs, where no truncation occurs (`Frontier.Multigraph.Connected.card_vertex_le`), apart from the harmless empty-graph convention. For disconnected graphs the ordinary cycle rank is the sum of the component ranks, $|E| - |V| + \#\text{components}$, not this formal quantity.

Hypothesis 4.1 (Layout bound)**.** Let $d \in \mathbb{N}$ and $A > 0$. The layout bound $\mathrm{LB}_d(A)$ asserts: for every $\eta > 0$ there is a constant C such that every connected, loopless multigraph G of maximum degree at most d has a layout whose frontiers all have at most

$$(A + \eta)\,\beta_1(G) + \eta\,|V(G)| + C$$

edges. `Frontier.LayoutBound`

Trees have cycle rank zero and, in bounded degree, cutwidth $\Theta(\log|V|)$ in the worst case, so an additive term sublinear in $|V|$ is unavoidable. Only the coefficient A matters: any sublinear error term is absorbed into $\eta|V| + C$.

4.2 Constraint networks

Definition 4.2 (Constraint networks)**.** A *constraint network* over the alphabet U with inputs ι consists of a multigraph (V, E), a partial map site: $\iota \rightharpoonup V$ assigning to some inputs the vertex at which they are read, and for every vertex v a *constraint* $\mathcal{C}_v \subseteq U^\iota \times U^E$ that is *local*: whether $(x, \alpha) \in \mathcal{C}_v$ depends only on the inputs read at v and on the values $\alpha(e)$ of the edges e incident to v. An assignment $\alpha\colon E \to U$ *satisfies* the network on input x if $(x, \alpha) \in \mathcal{C}_v$ for every v, and x is *accepted* if some assignment satisfies the network on x. `Frontier.Network`

Networks are computation with the direction removed: the constraints are arbitrary local relations. They are also nondeterministic, in that acceptance is existential.

Write $\mathrm{read}(L)$ for the inputs read at vertices of L. The next lemma is the exact sense in which the frontier separates the past from the future.

Lemma 4.3 (Separator lemma)**.** *Let α satisfy the network on input x, and β on input y, and suppose α and β agree on the cut $\delta(L)$. Let z agree with x on* $\mathrm{read}(L)$ *and with y elsewhere. Then the assignment γ that agrees with α on the edges touching L and with β on the other edges satisfies the network on z.* `Frontier.Network.Satisfies.splice`

Proof. At a vertex $v \in L$, every incident edge touches L, so γ agrees with α there, and z agrees with x on the inputs read at v; by locality, the constraint at v holds for (z, γ) because it holds for (x, α). At a vertex $v \notin L$, the inputs read at v are not in $\mathrm{read}(L)$, so z agrees with y there. An edge at v either does not touch L, and then $\gamma = \beta$ on it, or touches L, and then it lies in $\delta(L)$, where $\gamma = \alpha = \beta$. So the constraint at v holds for (z, γ) because it holds for (y, β). □

Taking L to be all vertices shows that acceptance depends only on the inputs that are read (`Frontier.Network.dependsOn_read`).

4.3 Sweeping along a layout

Fix a layout π of a network with N vertices. Process the vertices in the order of π; at time $t \le N$ the revealed inputs are $X_t = \mathrm{read}(P_t)$, those read by the first t vertices, and at time $N + 1$ all inputs are revealed, $X_{N+1} = \iota$, including those that no vertex reads. Messages pair an arbitrary *extra* message with the values on the frontier.

Proposition 4.4 (Sweeps along a layout)**.** *Let $S \subseteq U^\iota$. For every input x fix values $\alpha_x \colon E \to U$ on the edges, and an extra message $\epsilon_t(x)$ at each time $t \le N+1$, constant on S at time $N+1$. Suppose that whenever $x, y \in S$ have $\epsilon_t(x) = \epsilon_t(y)$ and $\alpha_x = \alpha_y$ on C_t, the input that agrees with x on X_t and with y elsewhere lies in S. Then the messages $\mu_t(x) = (\epsilon_t(x), \alpha_x|_{C_t})$ form a sweep of S of length $N+1$. At a step $t < N$, processing the vertex $v = \pi^{-1}(t)$,*

(a) the number of transitions is at most $|\epsilon| \cdot q^{|C_t \cup C_{t+1}| + |\mathrm{read}(v)|}$, where $|\epsilon|$ is the number of values of the pair $(\epsilon_t, \epsilon_{t+1})$ on S;

(b) if v lies in a set W of vertices whose edges contain $C_t \cup C_{t+1}$, and the values of α_x on the edges at W are determined by the pair $(\epsilon_t(x), \epsilon_{t+1}(x))$ and the inputs read in W, the number of transitions is at most $|\epsilon| \cdot q^{|\mathrm{read}(W)|}$.

At the last step it is at most $|\epsilon| \cdot q^{\#\textit{unread inputs}}$. `Frontier.Network.frontierSweep`

Proof. At time 0 nothing is read and at time $N+1$ everything is revealed; the final frontier $C_{N+1} = \delta(V)$ is empty and the final extra message is constant, so the final message is constant on S. Splicing at $t \le N$ is the hypothesis; at $t = N+1$ it is trivial. The transition at step t consists of the two messages and the inputs revealed during the step, which are read at v (`Frontier.Network.newly_subset_readAt`). The two messages are determined by the two extra messages and the values on $C_t \cup C_{t+1}$, which gives (a) (`Frontier.Network.ncard_transition_le`). Under the hypothesis of (b), the values on $C_t \cup C_{t+1}$ and the inputs read at v are determined by the extra messages and the inputs read in W (`Frontier.Network.ncard_transition_le_of_touching`). At the last step both frontiers are empty and the inputs revealed are the unread ones (`Frontier.Network.ncard_transition_last_le`). □

For a network, take for α_x a satisfying assignment on each accepted input, and no extra message at all. Splicing is then the separator lemma with $L = P_t$, and Proposition 4.4 gives *the sweep of the network* (`Frontier.Network.sweep`).

Theorem 4.5 (Frontier counting for networks)**.** *Let the accepted set S of a network be K-rectangle-free with $|S| \ge K$. Suppose a layout has all frontiers of size at most w, every vertex has degree at most d, and every vertex reads at most m inputs. Then*

$$|S| \;\le\; (K-1)^2 \left(|V|\, q^{\,w+d+m} \;+\; q^{\#\textit{unread inputs}} \right).$$

`Frontier.Network.ncard_accepted_le`

Proof. Apply Lemma 3.2 to the sweep of the network and bound the transitions by Proposition 4.4(a). Adding the vertex v changes the cut only at its own edges, so $C_{t+1} \subseteq C_t \cup E(v)$ and $|C_t \cup C_{t+1}| \le w + d$. □

The memory of the sweep is the frontier plus one local constraint: this is the source of the constant $d + m$ in the exponent.

5 Compiling circuits into networks

5.1 Circuits

We use CSLib's model [The26]. A *signature* σ is a set of operation symbols, each with an arity; an *interpretation* I assigns to every symbol g of arity a an operation $I(g) \colon U^a \to U$. A *straight-line*

program over σ with n inputs and s gates is a list of lines; line j names a symbol and, for each of its arguments, a *wire*: an input x_i or an earlier gate. A *circuit* is a program together with output wires; it has *fan-in at most* r if every symbol used has arity at most r, and its *inner gates* are those of positive arity (`Cslib.Circuits.Circuit.innerSize`).

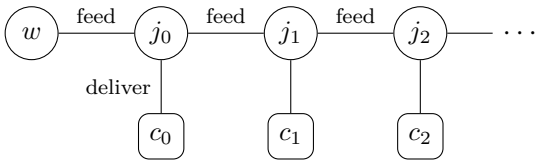
We compile a program together with a finite family of outputs $o \in O$, each with a set $\mathrm{Acc}_o \subseteq U$ of accepting values. A circuit with one output *decides* $S \subseteq U^n$ if $x \in S$ exactly when the output carries a value in Acc on input x (`Frontier.Decides`); a circuit *computing* a function accepts every value at every output. The basis is arbitrary: there may be infinitely many symbols, constants, operations without any algebraic structure, and the full basis of all operations of arity at most r. A Boolean circuit computing f decides $f^{-1}(1)$ with $\mathrm{Acc} = \{1\}$ (`Frontier.Decides.of_computes`).

5.2 The construction

Fix a program p, outputs $(\mathrm{out}_o)_{o \in O}$, I, and $(\mathrm{Acc}_o)_{o \in O}$.

Signals and consumers. A wire is *reachable* if it is an output or an argument of a reachable gate; the reachable wires are the *signals* (`Frontier.Compiler.Reach`). A *consumer* is an argument slot of a reachable gate, or an output, consumed by its own *output vertex*. Each consumer reads one signal, and every signal has at least one consumer.

Junction chains. For each signal w, number its consumers $c_0, c_1, \ldots$, and create one *junction* j_i per consumer. The junctions form a chain hanging off w:



Each junction has a *feed* edge from its predecessor (the signal vertex itself for j_0) and a *deliver* edge to the vertex owning its consumer: the gate vertex of the slot, or the output vertex.

The network. The vertices are the signals, the junctions, and the outputs; the edges are the feed and deliver edges, two per junction (`Frontier.Compiler.network`). The constraints are:

- at the vertex of an input x_i, which reads x_i: its outgoing edge carries x_i;
- at the vertex of a gate g: its outgoing edge carries $I(g)$ applied to the values on the deliver edges of its argument slots;
- at a junction: every outgoing edge carries the value of its feed edge;
- at the vertex of an output o: its incoming edge carries a value in Acc_o.

The design is uniform: a signal with a single consumer has a chain of length one, so there is no special case for fan-out one, and an output is just another consumer, so even a circuit whose output is an input wire compiles without a special case.

A complete example. Consider the Boolean circuit $a = x_1 \oplus x_2$, $b = a \oplus x_3$, $c = a \vee b$, accepting when $c = 1$. The signal a has two consumers; every other signal has one. Its network has six signal vertices, seven junctions, and one output vertex, with fourteen edges, hence cycle rank one (Fig. 1). Process c, j_c, o first. The two frontier edges carry a and b, and the processed constraints require $a \vee b = 1$. No input has yet been read: the accepting inputs 001 and 100 give frontier values $(0, 1)$ and $(1, 1)$, respectively. Thus a frontier assignment need not be computable from the revealed inputs alone. Completing the local constraints on the other side checks it against the input.

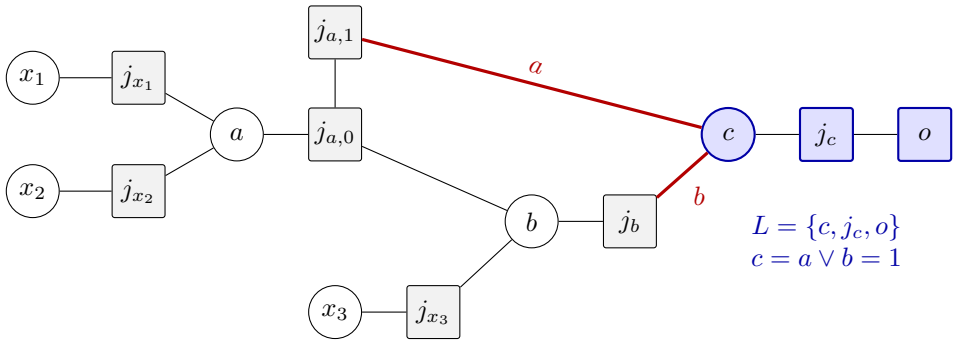


Figure 1: The complete compiled network for $a = x_1 \oplus x_2$, $b = a \oplus x_3$, $c = a \vee b$. Circles are signal vertices and rectangles are junctions or the output. The shaded vertices are a possible initial segment of a layout; the two thick edges are its frontier. This layout begins at the output, before reading any input.

5.3 Exactness

Theorem 5.1 (Exactness)**.** *The network accepts exactly the inputs on which every output o lies in* Acc_o*, and every satisfying assignment puts on each edge the value of its signal in the run of the program.* `Frontier.Compiler.accepted_network`

Proof. If every output lies in its accepting set, put on every edge the value of its signal in the run of the program; every constraint holds (`Frontier.Compiler.trace_satisfies`). Conversely, let α satisfy the network. The junction constraints force every edge of a signal to carry the value of the signal's first feed edge (`Frontier.Compiler.eq_head`). By induction along the program order, using that a gate reads only inputs and earlier gates (`Cslib.Circuits.Program.lines_wires_eq_gate_lt`), the gate constraints force that value to be the signal's value in the run (`Frontier.Compiler.eq_trace`). The constraints at the output vertices then say that every output lies in its accepting set. □

So the satisfying assignment is unique: the local guesses of a sweep of the network are forced globally. The lower bound for decision problems never uses uniqueness, but the extensions of Sections 8 and 9 do, through the following consequence of the separator lemma.

Lemma 5.2 (Outputs across a cut)**.** *Let the network accept every value at every output. Let the runs on inputs x and y agree on the cut of a set of vertices L, and let z agree with x on* read(L) *and with y elsewhere. Then every output of z whose vertex lies in L equals that output of x, and every other output of z equals that output of y.* `Frontier.Compiler.trace_out_of_cut`

Proof. The separator lemma gives an assignment γ satisfying the network on z, equal to the run of x on the edges touching L and to the run of y elsewhere. By Theorem 5.1, γ is the run of z. The edge delivering an output to its vertex touches L if that vertex lies in L; if it touches L although the vertex does not, it lies on the cut, where the two runs agree. □

Proposition 5.3 (Frontier programs; not formalized)**.** *A layout of width w of the compiled network of a one-output circuit of fan-in at most r gives an unambiguous fixed-order read-once program for its accepted set, with at most $|V|\, q^{w+r+2} + nq$ edges.*

Proof. At layer t the states are assignments to C_t. On processing v, query its input if it reads one, and guess values on the new frontier edges. A transition exists exactly when the old and new assignments agree on their common edges and satisfy the constraint at v with the queried value. The incident edges of v lie in $C_t \cup C_{t+1}$, whose size is at most $w + r + 1$, so there are at most q^{w+r+2} transitions at this layer. Finally query the unread inputs one at a time, adding at most nq edges.

Every complete path assigns a consistent value to each edge from its first endpoint to its second, and satisfies every local constraint. It therefore determines a satisfying assignment of the network on its input. Conversely, every satisfying assignment gives exactly one path: each transition records all new edge values and any input read at that step. By Theorem 5.1, an accepted input has exactly one satisfying assignment, and a rejected input has none. Thus the program decides the same set and is unambiguous. □

5.4 Shape and size

Proposition 5.4 (Shape)**.** *The network is loopless, and it is connected if some wire is upstream of every output, in particular if there is one output. If the fan-in is at most $r \geq 2$, every vertex has degree at most $r + 1$; in particular, for fan-in two the graph is subcubic.*

`Frontier.Compiler.maxDegreeLE`

Proof. A gate vertex has at most r incoming deliver edges and one outgoing feed edge; a junction has its feed, its deliver, and, if present, the feed of the next junction; an input vertex and an output vertex have one edge each. A junction is joined along its chain to its signal, and a signal through a consumer to that consumer's gate or output vertex, so every vertex is joined to the vertex of some output; a wire upstream of an output is joined to it (`Frontier.Compiler.connected_of_upstream`). No edge joins a vertex to itself (`Frontier.Compiler.loopless`). □

Proposition 5.5 (Counting)**.** *Let the fan-in be at most r, let s be the number of inner gates, and let* read *be the set of reachable inputs, which are the inputs read by the network. Then*

$$|E| + |\mathrm{read}| \leq |V| + (r-1)\, s \qquad \text{and} \qquad |V| \leq 2rs + 3|O|,$$

and if the network is connected, $\beta_1 + |\mathrm{read}| \leq (r-1)\, s + 1$.

`Frontier.Compiler.card_edge_add_ncard_read_le`

Proof. Let there be C consumers, W signals, G reachable inner gates, and Z slots. There are $2C$ edges and $W + C + |O|$ vertices, and $C = Z + |O|$. A gate of arity k has k slots, so $Z \leq rG$; constants have none. The signals include the reachable inputs and the reachable inner gates, so $W \geq |\mathrm{read}| + G$. Hence

$$|E| + |\mathrm{read}| = 2Z + 2|O| + |\mathrm{read}| \leq Z + 2|O| + W + (r-1)G = |V| + (r-1)G.$$

Every signal has a consumer, so $W \leq C$ and $|V| \leq 2C + |O| \leq 2rs + 3|O|$ (`Frontier.Compiler.card_vertex_le`). A connected graph has $|V| \leq |E| + 1$, which gives the bound on the cycle rank (`Frontier.Compiler.cycleRank_add_ncard_read_le`). □

For fan-in two the bound reads $\beta_1 \leq s + 1 - m$ for m reachable inputs. A constant-free read-once binary formula on m distinct inputs has $m - 1$ gates and compiles to a tree. If all reachable gates

are binary and there are no constants, the equality is $\beta_1 = s + 1 - m$: repeated occurrences of an input count as sharing too. For example, $x_1 \wedge (x_1 \vee x_2)$ has $s = m = 2$ and compiles to a graph of cycle rank one. Constant gates only lower the cycle rank, which is why they are free in every theorem.

5.5 Counting distinct signals

An edge records a copy of a circuit signal. Exactness identifies all copies on every accepting run, allowing a finer capacity measure. Write $\ell(e)$ for the signal carried by edge e, C_t for a frontier, and $L_t = \ell(C_t)$ for its set of signal labels. For the vertex v_t processed at step t, define the *signal transition budget*

$$b_t = |L_t \cup L_{t+1}| + |\mathrm{read}(v_t)|, \qquad b = \max_{t<|V|} b_t.$$

The union matters: the two frontiers share one assignment of values to their signal labels.

Proposition 5.6 (Signal capacity)**.** *For a compiled circuit over an alphabet of size q, with a K-rectangle-free accepted set S of size at least K, every layout satisfies*

$$|S| \le (K-1)^2(|V|q^b + q^{n-|\mathrm{read}|}).$$

Moreover $|L_t| \le |C_t|$ and, at maximum degree d with one input per vertex, $b \le w + d + 1$ for edge width w. `Frontier.Compiler.ncard_accepted_le_signals`

Proof. By exactness, the value on e is the trace value of $\ell(e)$. Thus the labels on $C_t \cup C_{t+1}$, together with the input read at v_t, determine the transition; there are at most q^{b_t} choices. Apply the frontier counting lemma and the usual last-step bound for unread inputs. The formal transition lemma works for any edge labels through which the chosen edge values factor (`Frontier.Network.ncard_transition_le_labels`). Finally, taking an image cannot increase cardinality, and $C_{t+1} \subseteq C_t \cup E(v_t)$. □

This can save arbitrarily many redundant copies at a particular frontier: process a signal's producer and all its junctions before its k consumers. Its k delivery edges cross the cut, but they carry one signal value. The proposition applies to dense rectangle-free families with the same $n - o(n)$ demand on b as on the former edge budget. The dynamic program of Section 10 can likewise keep one state bit per signal label and run in time $O^*(2^b)$.

A universal bound $b \le (A_{\mathrm{sig}} + o(1))\beta_1 + o(|V|)$ for compiled graphs would therefore give coefficient $1 + 1/A_{\mathrm{sig}}$, and, if constructive, counting exponent $A_{\mathrm{sig}}/(1 + A_{\mathrm{sig}})$. The present edge-layout theorem supplies $A_{\mathrm{sig}} = A$. A strict improvement $A_{\mathrm{sig}} < A$ is an additional layout problem; the capacity proposition alone does not establish it.

6 Demand and supply

6.1 The frontier inequality

Every lower bound in this paper ends with the same comparison. The *demand* of a problem is a lower bound on the width of every layout of the graph of a circuit for it, about n. The *supply* is the width of the layout that the layout bound provides, about A times the cycle rank, which is about $s - n$. The following lemma keeps every lower-order term.

Lemma 6.1 (Demand and supply)**.** *Let $A > 0$, $\varepsilon > 0$ and $\kappa \geq 0$. There is $\eta_0 > 0$ such that for every $\eta \in (0, \eta_0]$, every constant C, and every function $e(n) = o(n)$, for all large n the following holds. If real numbers $s, v \geq 0$ and w satisfy $v \leq \kappa(n + s + 1)$,*

$$\begin{aligned} n &\leq w + e(n) + \kappa \log(v+1) && \textit{(demand)},\\ w &\leq (A+\eta)(s - n + e(n)) + \eta\, v + C && \textit{(supply)}, \end{aligned}$$

then $s > (1 + 1/A - \varepsilon)\, n$. `Frontier.eventually_lt_of_demand`

In applications v is the number of vertices of the graph, w the width of a layout, and s is $(r-1)$ times the number of gates.

Proof. Let $B = 1/A + \kappa(2 + 1/A)$ and $\eta_0 = A\varepsilon/(2B+2)$, so that $\eta B \leq A\varepsilon/2$ for $\eta \leq \eta_0$, and let $\delta = A\varepsilon/10$. For large n we have $(A+\eta+1)|e(n)| \leq \delta n$, $|C| + \eta\kappa \leq \delta n$, and $(\kappa+1)\log(\kappa(2 + 1/A)n + \kappa + 1) \leq \delta n$. Suppose $s \leq (1 + 1/A - \varepsilon)n$. Then $s \leq (1 + 1/A)n$, so $v \leq \kappa(2 + 1/A)n + \kappa$ and $\kappa \log(v+1) \leq \delta n$. From $s - n \leq (1/A - \varepsilon)n$ we get $(A+\eta)(s-n) \leq (1 - A\varepsilon + \eta/A)\, n$, and $\eta v \leq \eta\kappa(2 + 1/A)\, n + \eta\kappa$. Adding demand and supply,

$$n \;\leq\; n - A\varepsilon n + \eta B n + (A+\eta+1)|e(n)| + |C| + \eta\kappa + \kappa\log(v+1) \;\leq\; n - \tfrac{A\varepsilon}{2} n + 3\delta n,$$

that is, $A\varepsilon n/2 \leq 3A\varepsilon n/10$, which is false. □

The order of the quantifiers matters: the slack η of the layout bound is chosen first, depending only on A, ε and κ, and then the layout bound provides the constant C.

6.2 The reduction theorem

Theorem 6.2 (Reduction)**.** *Let c be a circuit with s inner gates of fan-in at most $r \geq 2$, over any basis on U, deciding a K-rectangle-free set $S \subseteq U^n$ with $|S| \geq qK^2$. Then there is a connected, loopless multigraph G of maximum degree $r+1$ with*

$$|V(G)| \leq 2rs + 3 \qquad \textit{and} \qquad \beta_1(G) \leq (r-1)\, s - n + \log_q K + 1,$$

every layout of which has width at least

$$\log_q |S| \;-\; 3\log_q K \;-\; \log_q(|V(G)| + 1) \;-\; (r+2).$$

`Frontier.exists_graph_of_decides`

Proof. Let G be the constraint graph of c; its shape and size are given by Propositions 5.4 and 5.5. The accepted set is S (Theorem 5.1), which depends only on the read inputs, so the support lemma gives $q^{n-|\mathrm{read}|} < K$. Hence $|\mathrm{read}| > n - \log_q K$, and Proposition 5.5 gives the bound on β_1 (`Frontier.cycleRank_le`). For a layout of width w, Theorem 4.5 with $d = r+1$ and $m = 1$ gives

$$|S| \leq (K-1)^2(|V| q^{w+r+2} + q^{n-|\mathrm{read}|}) \leq K^2(|V|\, K q^{w+r+2} + K q^{w+r+2}) = K^3(|V|+1)\, q^{w+r+2},$$

using $q^{n-|\mathrm{read}|} < K$ once more; take logarithms (`Frontier.logb_ncard_le`). □

6.3 Proof of the main theorem

Proof of Theorem 1.1. Let s now denote $(r-1)$ times the number of inner gates, and set

$$e(n) = (n - \log_q |S_n|) + 4 \log_q K_n + r + 3, \qquad \kappa = 4 + 1/\log 2.$$

By hypothesis $e(n) = o(n)$, and for large n we have $|S_n| \geq qK_n^2$ (`Frontier.eventually_card_mul_sq_le_ncard`). Let $\eta = \eta_0$ be given by Lemma 6.1, and C by $\mathrm{LB}_{r+1}(A)$ for this η. Apply the layout bound to the graph G of Theorem 6.2, and let w be the width of the resulting layout. With s_0 inner gates, the graph has $v \leq 2rs_0 + 3 \leq 4s + 3 \leq \kappa(n + s + 1)$ vertices, as $r \geq 2$. The reduction theorem gives the demand, since $\log_q(v+1) \leq \kappa \log(v+1)$ and $n - \log_q |S_n|$ is part of $e(n)$; the layout bound gives the supply, since $\beta_1(G) \leq s - n + e(n)$. Lemma 6.1 concludes. □

Remark 6.3 (Unambiguous programs suffice; not formalized). The proof uses rectangle-freeness and density only to force exponentially many transitions, through Lemma 3.2. By Proposition 5.3, every layout of a compiled circuit gives an unambiguous fixed-order read-once program with at most $|V| q^{w+r+2} + nq$ edges. So the conclusion of Theorem 1.1 holds for every family S_n such that every unambiguous read-once program for S_n has $q^{n-o(n)}$ edges. The support lemma is replaced by the observation that a set depending only on R has an unambiguous program with $O(|R|\, q^{|R|} + nq)$ edges, so that $|\mathrm{read}| \geq n - o(n)$.

6.4 Existential witness inputs

Lemma 6.4 (Hiding inputs)**.** *Let N be a network with ordinary inputs $x \in U^n$ and witness inputs $y \in U^k$, over a nonempty alphabet. There is a network H on the same graph with ordinary inputs only and accepted set $\{x : \exists y,\ (x, y) \in \mathrm{accepted}(N)\}$. Each visible input keeps its original site; consequently the number of inputs read at any vertex, and in total, cannot increase.*

`Frontier.Network.accepted_hide`

Proof. At each vertex replace the old constraint $Q_v(x, y, a)$ by $\exists y\, Q_v(x, y, a)$. These constraints remain local on the visible inputs and incident edges. To assemble local witnesses, choose a witness assignment y_v for each satisfied local constraint, and set the global value of a witness coordinate to its value in y_v at its unique read site v. Unread coordinates get any fixed alphabet value. Locality implies that this single assignment satisfies every old constraint. The converse uses the same global witness at every vertex. □

Theorem 6.5 (Nondeterministic circuits)**.** *The conclusion of Theorem 1.1 holds for a verifier circuit $C(x, y)$ accepting $x \in S_n$ exactly when $C(x, y) \in \mathrm{Acc}$ for some $y \in U^k$. The size threshold is uniform in k: only the n ordinary inputs appear on the right-hand side, while all positive-arity verifier gates count towards s. In particular, at fan-in two the coefficient is $L = 4.562497\ldots$.*

`Frontier.lowerBound_nondeterministic`

Proof. Compile C and hide its witness inputs. The connected graph still has maximum degree $r+1$ and at most $2rs + 3$ vertices. If m ordinary inputs and m_y witness inputs were reachable, its cycle rank satisfies

$$\beta_1 \leq (r-1)s + 1 - m - m_y \leq (r-1)s + 1 - m.$$

The support lemma for H forces $m > n - \log_q K_n$. Its accepting assignments need not be unique, but the network frontier inequality uses only existence. Thus every layout has the same frontier demand as in Theorem 6.2; formally the finite reduction is `Frontier.exists_graph_of_nondeterministic_decides`. The identical demand–supply comparison finishes the proof. The Gaussian specialization is

`Frontier.lowerBound_nondeterministic_gaussian`. Applying this to Li's cited disperser family gives the explicit Boolean language of Corollary 1.5 the same nondeterministic lower bound. This does not give projected model counting: counting pairs (x, y) would count witnesses with multiplicity. □

Corollary 6.6 (Read witnesses cost gates)**.** *Let S_n be the dense rectangle-free family of Theorem 1.1. For every fixed $D \in \mathbb{N}$ and $\varepsilon > 0$, for all sufficiently large n, every binary verifier for S_n with $s \le n^D$ positive-arity gates satisfies*

$$s > (L - \varepsilon)n + r, \qquad L = 1 + \frac{\pi}{3\arccos((1+2\sqrt{2})/4)}.$$

Here r counts distinct witness inputs reachable from the output. Unused declared witnesses do not count, and the threshold is uniform in the declared witness length and gate basis.

`Frontier.lowerBound_nondeterministic_witnesses_gaussian`

Proof. Retain the witness term in the preceding proof. The resulting graph has at most $4s + 3$ vertices and

$$\beta_1 \le s - r - n + \log_q K_n + 1.$$

The finite statement is `Frontier.exists_graph_of_nondeterministic_decides_witnesses`. If $p > 0$ is a cubic layout coefficient, the compression proof in Proposition 7.2 gives, for every fixed $\eta > 0$, a layout of width at most $(2p + \eta)\beta_1 + 3\log_2|V| + C_\eta$. This logarithmic form is `Frontier.CubicLayoutBound.exists_layout_log`. Since $s \le n^D$, both this overhead and the logarithmic graph-size term in the frontier demand are $o(n)$. Together with density and $\log K_n = o(n)$, this yields

$$n \le (2p + \eta)(s - r - n) + o(n).$$

Choosing η sufficiently small in terms of ε gives the coefficient $1 + 1/(2p)$, as formalized by `Frontier.lowerBound_nondeterministic_witnesses`. The Gaussian cubic coefficient gives L. Keeping the logarithmic overhead is essential here: an error $\eta|V|$ need not be sublinear in n for polynomial-size verifiers. □

Consequently a verifier family of size $Ln + o(n)$ can read only $o(n)$ distinct witnesses. This constrains verifiers for these hard families; it gives no procedure for eliminating witnesses from arbitrary circuits. The same corollary applies to the explicit family in Corollary 1.5, using the cited extractor construction.

6.5 Average-case hardness from extractors

Theorem 6.7 (Circuit correlation under weighted inputs; not formalized)**.** *Fix a finite alphabet U of size $q \ge 2$, a fan-in bound $r \ge 2$, and assume $\mathrm{LB}_{r+1}(A)$. Put $\alpha = A/(1+A)$. Let C_n be single-output circuits with $s_n = O(n)$ positive-arity gates and m_n reachable inputs, and let $g_n : U^n \to \{-1, 1\}$ be any Boolean readout of their outputs. Write $s'_n = (r-1)s_n$. For arbitrary targets $f_n : U^n \to \{-1, 1\}$, probability measures μ_n, and thresholds $K_{L,n}, K_{R,n} \ge 2$, put $B_n = (K_{L,n} - 1)(K_{R,n} - 1)$ and $h_{q,n} = -\log_q \max_x \mu_n(x)$. Then*

$$\begin{aligned}|\mathbb{E}_{\mu_n}[f_n g_n]| &\le \mathrm{Pack}_{K_{L,n},K_{R,n}}(f_n, \mu_n) + B_n q^{-h_{q,n}+\min\{m_n, A(s'_n - m_n)\}+o(n)} \\ &\le \mathrm{Pack}_{K_{L,n},K_{R,n}}(f_n, \mu_n) + B_n q^{-h_{q,n}+\alpha s'_n+o(n)}.\end{aligned}$$

The lower-order term is uniform over the targets, measures, and thresholds. Under the shared-error hypothesis of Theorem 3.7, the packing term may be replaced by $\beta_n + \zeta_n$.

Proof. There are two ways to obtain unambiguous programs for the two output classes. First, enumerate the reachable inputs in a decision tree, then query each unread input along a chain before the accepting sink. Across the two classes this uses $q^{m_n+O(\log(n+1))}$ edges. Second, the compiler gives $\beta_1 \le s'_n - m_n + 1$ and $|V| \le 2rs_n + 3 \le 4s'_n + 3$. For each fixed $\eta > 0$, the layout bound and the direct construction of Proposition 5.3, including queries of unread inputs, give programs with total edge count D_n satisfying

$$\log_q D_n \le A(s'_n - m_n) + 5\eta s'_n + C_\eta + O_{q,r}(\log(n+1)),$$

where the constant absorbs the layout bound's additive term. Taking η to zero sufficiently slowly gives $\log_q D_n \le A(s'_n - m_n) + o(n)$ uniformly for $s_n = O(n)$. Both constructions apply to each circuit output class, so the two-program hypothesis requires no extra assumption on the circuit. Choose the smaller construction and apply Theorem 3.7. The second bound uses $\min\{m, A(s'-m)\} \le \alpha s'$. □

For uniform inputs $h_{q,n} = n$. If $\log_q K_{L,n} + \log_q K_{R,n} = o(n)$ and $s'_n \le (1 + 1/A - \varepsilon)n$ for a fixed $\varepsilon > 0$, the remainder is $q^{-\Omega_\varepsilon(n)}$. More generally, correlation at least the packing term plus a fixed $\delta > 0$ forces

$$s'_n \ge (1 + 1/A)(h_{q,n} - \log_q B_n + \log_q \delta) - o(n).$$

The next theorem specializes to uniform rectangle bias and improves the exponential remainder by using the unused coordinates directly.

Theorem 6.8 (Average-case circuit lower bound)**.** *Assume* $\mathrm{LB}_3(A)$ *for* $A > 0$ *and put* $L = 1 + 1/A$. *Let* $K_n \ge 2$ *with* $\log K_n = o(n)$, *and let* $f_n : \{0,1\}^n \to \{0,1\}$ *have bias at most* $2b_n$, *where* $0 \le b_n \le 1/2$, *on every coordinate rectangle whose two sides have at least* K_n *elements:*

$$\left|\mathbb{E}_{x \text{ uniform in } R}(-1)^{f_n(x)}\right| \le 2b_n.$$

For every fixed $0 < \varepsilon < L$ *and* $0 < \gamma < \min\{1, A\varepsilon\}$, *for all sufficiently large* n *every Boolean circuit* C *of fan-in at most two with at most* $(L - \varepsilon)n$ *positive-arity gates satisfies*

$$\Pr_{x \text{ uniform}}[C(x) = f_n(x)] \le \frac{1}{2} + b_n + 2^{-\gamma n}.$$

In fact, its absolute signed correlation with f_n *is at most* $2b_n + 2^{1-\gamma n}$. *With the Gaussian layout theorem* $L = 4.562497\ldots$.

`Frontier.averageCase`, `Frontier.averageCase_abs`, `Frontier.averageCase_gaussian`

Proof. Set $K = K_n$, $b = b_n$, $k = \lceil \log_2 K \rceil$, and let m be the number of reachable inputs of C. Write $S_a = \{x : C(x) = a\}$ and $w(x) = (-1)^{f_n(x)}$.

Many unread inputs. If $n - m \ge k$, choose k unread coordinates X. Each output class is the rectangle $S_a = \{0,1\}^X \times B_a$. Its first side has $K \le 2^k < 2K$ elements. If $|B_a| \ge K$, the bias hypothesis applies; otherwise $|S_a| \le 2K(K-1)$. Treat the small classes as a residual set of total size at most $4K^2$. It follows that

$$\left|2^{-n}\sum_x (-1)^{C(x)} w(x)\right| \le 2b + 4K^2\, 2^{-n}.$$

Since $k = o(n)$ and $\gamma < 1$, its contribution to agreement is at most $2^{-\gamma n}$ for large n.

Few unread inputs. Otherwise $m > n - k$. Compile C, obtaining a graph with $v \le 4s + 3$ vertices and $\beta_1 \le s + 1 - m$. For any fixed $\eta > 0$ the layout theorem gives width

$$w_0 \le (A + \eta)(s + 1 - m) + \eta(4s + 3) + C_\eta \le (1 - A\varepsilon + O_A(\eta))n + O_{A,\eta}(k+1) + C_\eta,$$

where $s \le (L-\varepsilon)n$. Choose η sufficiently small in terms of A, ε, γ. For each output value a, sweep the compiled network accepting S_a along this layout. This sweep is coherent: the separator lemma splices two satisfying edge assignments, and uniqueness identifies the result with the trace of the spliced input. Every earlier frontier then has the first input's values and every later frontier the second's. Its number of used transitions is at most

$$D = v2^{w_0+4} + 2^{n-m} < v2^{w_0+4} + K.$$

The final step reveals all unread coordinates; the adjacent-frontier union pays only $w_0 + 3$ edge values and at most one newly read input at each other step. Apply Proposition 3.5 with $c(x) = 2b$ and atom bound 1 to each output class. With $r = (K-1)^2 D$, this gives

$$\left| 2^{-n} \sum_x (-1)^{C(x)} w(x) \right| \le 2b + 2r\, 2^{-n}.$$

`Frontier.Network.sweep_coherent`, `Frontier.agreement_le_layout`, `Frontier.agreement_le_unused`

Now $r\, 2^{-n} \le 2^{-\gamma n}$ for large n: the two exponential rates in the remainder are at most $-A\varepsilon + O_A(\eta) + o(1)$ and $-1 + o(1)$, respectively. Finally, $\Pr[C(x) = f_n(x)] = (1 + \mathbb{E}(-1)^{C(x)+f_n(x)})/2$. The absolute-value bound also controls agreement after complementing the circuit output. □

Corollary 6.9 (Sumset extractors)**.** *The hypothesis of Theorem 6.8 holds if f_n is a one-bit sumset extractor with error b_n for independent sources of min-entropy at least $\log_2 K_n$. Thus every such family with $\log K_n = o(n)$ has agreement at most $1/2 + b_n + 2^{-\Omega_\varepsilon(n)}$ with circuits of size at most $(L-\varepsilon)n$. The formal hypothesis is just the flat-source guarantee: independent uniform sources on any two sets of size at least K_n have XOR output bias at most $2b_n$.*

`Frontier.FlatSumsetBias.rectangleBias`, `Frontier.averageCase_sumset`

Proof. Extend independent uniform samples from the two rectangle sides by zero on the opposite coordinates. Their sum is uniform on the rectangle and both sources have the required min-entropy. Error b_n for a single output bit is exactly bias at most $2b_n$. □

Corollary 6.10 (Explicit polynomially small agreement advantage)**.** *There are a language f in* P *and a constant $c > 0$ such that, for every fixed $\varepsilon > 0$ and all sufficiently large n, every Boolean circuit C of fan-in at most two with at most $(L-\varepsilon)n$ positive-arity gates satisfies*

$$\Pr_{x \sim U_n}[C(x) = f(x)] \le \frac{1}{2} + n^{-c}.$$

Proof. Chattopadhyay and Liao [CL22, Theorem 1] construct a polynomial-time extractor for the sum of two sources of min-entropy $k_n = (\log n)^{O(1)}$, with error $b_n = n^{-\Omega(1)}$. Take its first output bit; projection cannot increase statistical distance. Set $K_n = \lceil 2^{k_n} \rceil$. Then $\log K_n = o(n)$, so Corollary 6.9 applies. More precisely, if $b_n \le C_0 n^{-a}$ for some $a > 0$, its bound is eventually at most $1/2 + (C_0+1)n^{-a}$, which is at most $1/2 + n^{-a/2}$ for large n. The formal implication from the flat-source error bound, including absorption of the exponential remainder, is `Frontier.averageCase_sumset_polynomial`; the construction and its polynomial-time evaluation are external. □

Li's logarithmic-entropy construction still supplies the sharper polynomial rectangle threshold used for the DNNF bound, and gives constant-error average-case hardness for every fixed error [Li23]. The vanishing-error corollary instead uses the polynomial-error guarantee above. Both circuit agreement statements require a fixed gap below Ln; no exponentially small remainder is asserted exactly at $s = Ln$.

6.6 Independent randomization

The average-case bounds transfer to randomized computation by averaging over fixed seeds. This is the standard averaging direction of Yao's principle; see, for example, Blum and Blum [BB06, Section 5.4]. The formal transfer uses Mathlib's Tonelli theorem and is independent of the circuit model.

Theorem 6.11 (Randomization transfer)**.** *Let μ and ν be probability measures on input and seed spaces X and R, sampled independently, and let $b : R \to [0, \infty]$ be measurable. Let $h : X \times R \to [0, \infty]$ be measurable up to a $(\mu \times \nu)$-null set. If*

$$\int_X h(x, r)\, d\mu(x) \le b(r) \qquad \text{for } \nu\text{-almost every } r,$$

then

$$\int_X \int_R h(x, r)\, d\nu(r)\, d\mu(x) \le \int_R b(r)\, d\nu(r).$$

The displayed implication also holds for arbitrary s-finite measures. For probability measures and a constant bound b, some input x satisfies $\int_R h(x, r)\, d\nu(r) \le b$. If the double integral is finite, some seed preserves or improves that average payoff; the seed can be chosen to satisfy any prescribed almost-sure condition.

`Frontier.Randomized.average_le`
`Frontier.Randomized.exists_input_le`
`Frontier.Randomized.exists_seed`

Proof. Tonelli swaps the integrals. Integrating the seedwise inequality proves the bound. A measurable function on a probability space takes a value at most its average, and a nonnegative function of finite average takes a value at least its average outside any prescribed null set. Apply these two facts to the input and seed averages, respectively. □

In particular, let $\mathcal{C}$ be any class of deterministic maps $X \to Y$, with an arbitrary correctness relation on $X \times Y$. If every $g \in \mathcal{C}$ succeeds with μ-probability at most b, then any measurable random choice of such maps, independent of the input and lying in $\mathcal{C}$ almost surely, has average success at most b. Some input has success probability at most b. Apply the theorem to the correctness indicator. This covers decision, search, and approximation relations; neither finite support nor a uniform distribution is required.

`Frontier.Randomized.success_le_of_class`, `Frontier.Randomized.exists_input_success_le`

Corollary 6.12 (Randomized circuit lower bound)**.** *Under the hypotheses of Theorem 6.8, for all sufficiently large n, every circuit $C(x, y)$ of fan-in at most two and size at most $(L - \varepsilon)n$ satisfies*

$$\mathbb{E}_{x \sim U_n} \Pr_{y \sim \nu}[C(x, y) = f_n(x)] \le \frac{1}{2} + b_n + 2^{-\gamma n}.$$

Here y has any finite number of Boolean ports and any joint distribution ν independent of x; its coordinates may be biased and mutually dependent. Some input has success probability at most the same bound. Consequently, if $b_n \to 0$, any fixed success probability above $1/2$ on every input requires more than $(L - \varepsilon)n$ gates for all sufficiently large n. The Gaussian coefficient is $L = 4.562497\ldots$.

`Frontier.averageCase_randomized`
`Frontier.averageCase_randomized_gaussian`
`Frontier.lowerBound_randomized_gaussian`

Proof. For each fixed y, adjoin the two free nullary Boolean constants to the gate basis and wire the last input ports to the selected constants. This preserves the number of positive-arity gates exactly and preserves fan-in, while computing $C(x, y)$ on the ordinary inputs. Apply Theorem 6.8 to each fixed circuit, then Theorem 6.11 to the correctness indicator.

`Cslib.Circuits.Circuit.innerSize_fixInputs`
`Cslib.Circuits.Circuit.eval_fixInputs`
`Frontier.randomized_circuit_bound`

□

The polynomial-error instantiation retains the bound $1/2+(C_0+1)n^{-a}$, as checked by `Frontier.averageCase_randomized_sumset_polynomial`. Thus the externally constructed family of Corollary 6.10 also has randomized success at most $1/2+n^{-c}$ on some input below the same gate threshold. This consequence uses the average-case bias hypothesis, which is stronger than rectangle-freeness alone. It does not charge an additional gate for each coin port, and fixing a seed preserves average success rather than success simultaneously on all inputs.

7 Layout bounds

This section proves the layout bound for subcubic graphs that Theorem 1.1 takes as input. The coefficient is that of a random layout: score the edges of a cubic graph by correlated Gaussians, and list the vertices by the median of their scores.

Theorem 7.1 (Gaussian layouts)**.** *Let* $p = \frac{3}{2\pi}\arccos\frac{1+2\sqrt{2}}{4} \approx 0.14035$*. Then* $\mathrm{LB}_3(2p)$ *holds, and since* $2p \le 9/32$*, so does* $\mathrm{LB}_3(1/3)$*.* `Frontier.layoutBound_gaussian`

The proof has three parts. Section 7.1 reduces subcubic multigraphs to simple cubic graphs, at the cost of the factor two. Section 7.2 reduces laying out a cubic graph to scoring its edges. Section 7.3 chooses the scores.

7.1 Reduction to cubic graphs

Call a simple cubic graph on h vertices *c-layable* if for every $\xi > 0$ and all large h it has a layout whose frontiers have at most $(c+\xi)h + 2$ edges (`Frontier.CubicLayoutBound`).

Proposition 7.2 (Compression)**.** *If large cubic graphs are c-layable, then* $\mathrm{LB}_3(2c)$ *holds.*
`Frontier.LayoutBound.of_cubic`

Proof. Let G be connected, loopless, and subcubic, with N vertices and M edges, and write ∂X for the set of edges with exactly one endpoint in a set X of vertices. A *block* is a nonempty set of vertices listed in some order. A *compression* is a partition of the vertices into blocks such that

(i) $|\partial B| \le 3$ for every block B;

(ii) $|\partial P| \le 3\log_2|B|$ for every proper prefix P of a block B;

(iii) the number of blocks plus the number of edges with both endpoints in one block is at least N.

The singletons form a compression. If two distinct blocks B_1, B_2, with $|B_1| \ge |B_2|$, are joined by an edge and $|\partial(B_1 \cup B_2)| \le 3$, *merge* them into the block $B_1 \cup B_2$ that lists B_1 first and then B_2. The result is again a compression: (i) holds by assumption; (iii) holds because one block is lost and at least one edge moves inside a block; and a proper prefix of the merged block is a proper prefix

of B_1, or B_1 followed by a proper prefix P of B_2, whose boundary has at most $3 + 3\log_2|B_2| = 3\log_2(2|B_2|) \le 3\log_2|B_1 \cup B_2|$ edges, which gives (ii). Since $|\partial(B_1 \cup B_2)| = |\partial B_1| + |\partial B_2| - 2e$ for the number e of edges joining them, a merge is possible whenever one of the blocks has at most two edges leaving it, or two edges join them. Merging until no merge is possible gives a *final* compression (`Frontier.Multigraph.Compression.exists_final`).

If the final compression has a single block, list the vertices in its order: every frontier is the boundary of a proper prefix, of at most $3\log_2 N$ edges. Otherwise, by connectivity every block has a neighbouring block, so in a final compression every block has exactly three edges leaving it and two blocks are joined by at most one edge: the *quotient* H, whose vertices are the blocks, adjacent when an edge joins them, is a simple cubic graph, and its edges correspond to the edges of G between blocks (`Frontier.Multigraph.Compression.isRegularOfDegree`). With h blocks there are $3h/2$ edges between blocks, and (iii) gives $h + (M - 3h/2) \ge N$, that is, $h \le 2(M - N) = 2(\beta_1(G) - 1)$ (`Frontier.Multigraph.Compression.card_add_two_mul_le`).

Fix $\xi > 0$, let H have a layout whose frontiers have at most $(c + \xi)h + 2$ edges (for the finitely many small h, every frontier has at most $3h/2$ edges), and list the vertices of G block by block in the order of that layout, each block in its own order. A prefix of this layout is the union of a prefix Q of the layout of H and a proper prefix P of the next block, and its cut consists of edges of H crossing the frontier after Q and edges of ∂P. So every frontier has at most $(c + \xi)\,2\beta_1(G) + 3\log_2 N + O_\xi(1)$ edges, and since $3\log_2 N \le \eta N + O_\eta(1)$, this is $\mathrm{LB}_3(2c)$ (`Frontier.Multigraph.Compression.exists_layout`). □

7.2 Median layouts

Give each edge e of a cubic graph a real *score* $s(e)$. Each vertex has three edges and hence a *median* score $m(v)$: at least two of its edges score at most $m(v)$, and at least two score at least $m(v)$. A vertex *straddles* a threshold θ if one of its edges scores below θ and another at least θ. The *median layout* lists the vertices by median score, breaking ties arbitrarily.

Lemma 7.3 (Median layouts)**.** *Fix a grid $a, a+\delta, \dots, a+M\delta$, and let B be at least three times the number of edges scoring below a, at least three times the number scoring at least $a + M\delta$, and, for every $i < M$, at least*

$$\#\{v \text{ straddling } a + i\delta\} \;+\; 3\,\#\{e : a + i\delta \le s(e) < a + (i+1)\delta\}.$$

Then every frontier of the median layout has at most B edges. `Frontier.exists_key_of_edgeScore`

Proof. Let P be an initial segment of the layout and choose m with $m(u) \le m$ for $u \in P$ and $m(v) \ge m$ for $v \notin P$. An edge $e = uv$ leaving P, with $u \in P$, is charged as follows. If $s(e) < \theta \le m$ for a threshold θ, charge e to v: then $s(e) < m(v)$, and at most one edge at v scores below its median, so v is charged at most once. If $s(e) \ge \theta' > m$, charge e to u: then $s(e) > m(u)$, and at most one edge at u scores above its median.

If $a + i\delta \le m < a + (i+1)\delta$, take $\theta = a + i\delta$ and $\theta' = a + (i+1)\delta$. A vertex $v \notin P$ charged by an edge below θ has two edges scoring at least $m(v) \ge \theta$, so it straddles θ. A vertex $u \in P$ charged by an edge at least θ' has two edges scoring at most $m(u) < \theta'$: if one scores below θ, then u straddles θ; otherwise u is an endpoint of an edge scoring in the window $[\theta, \theta')$. The remaining edges leaving P score in the window. Since a window edge has two endpoints, at most $\#\{\text{straddlers}\} + 3\,\#\{\text{window edges}\}$ edges leave P. If $m < a$, every edge leaving P scores below a or is charged to a vertex of P with two edges below a, so at most three times the edges below a leave P; the case $m \ge a + M\delta$ is symmetric. □

Lemma 7.3 replaces the path decompositions through which such scores are usually turned into layouts: a frontier of the median layout is bounded directly by straddling counts.

7.3 Gaussian edge scores

Let H be a simple cubic graph with vertex set V. For a decay rate $0 \le \lambda < 1/\sqrt{2}$ and a radius R, the *kernel row* of a vertex v is the vector in $\mathbb{R}^V$ with entries $\lambda^{\mathrm{dist}(v,z)}$ for $\mathrm{dist}(v,z) \le R$ and 0 otherwise; let x_v be its normalization (`Frontier.Gaussian.unitKernel`). For an edge uv let y_{uv} be the unit vector in the direction of $x_u + x_v$. Draw a standard Gaussian vector $\omega \in \mathbb{R}^V$ and score each edge by $s(e) = \langle y_e, \omega \rangle$.

Figures 2 and 3 follow one realization through the construction on the hexagonal prism. The local medians give an order of the vertices; counting the edges crossing each prefix then gives the width of that order. Equal medians can occur because two endpoints may select the same edge score, even when the edge scores themselves are distinct.

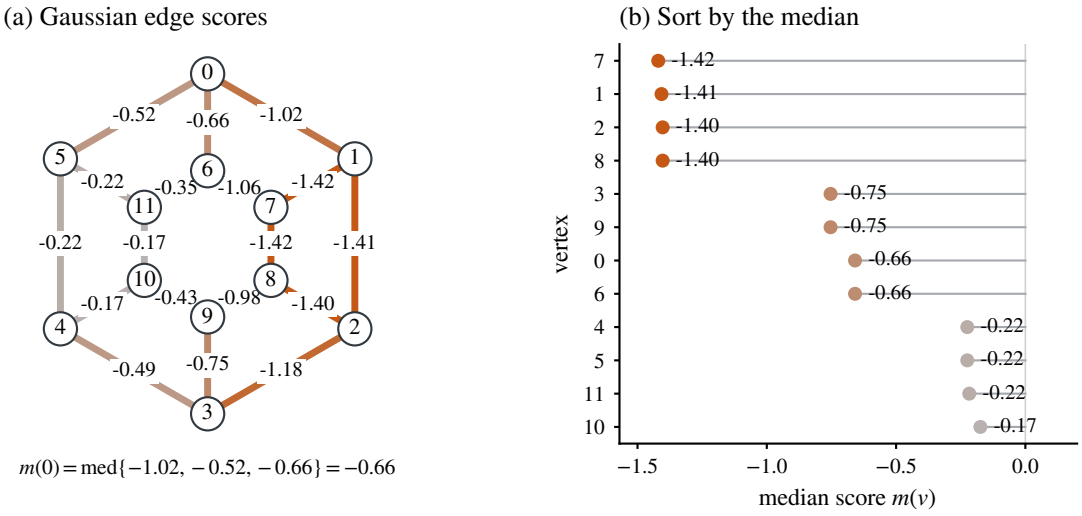


Figure 2: One Gaussian realization on the hexagonal prism: $\lambda = 0.65$, $R = 2$, seed zero. **(a)** Correlated edge scores. **(b)** Vertices sorted by their incident medians, using full precision and breaking ties by vertex label. Displayed scores are rounded. Each median is an edge score, so adjacent vertices can have equal medians.

Lemma 7.4 (Kernel correlation)**.** *In a graph of maximum degree three, adjacent unit kernel rows satisfy* $\langle x_u, x_v \rangle \ge 2\lambda/(1+\lambda^2) - 3(2\lambda^2)^R$. `Frontier.Gaussian.sum_unitKernel_mul_ge`

Proof. Let k_v be the kernel row of v, so $x_v = k_v/\|k_v\|$ and $\|k_v\| \ge 1$, since $k_v(v) = 1$. Let $\kappa = 2\lambda/(1+\lambda^2) \le 1$, and let u, v be adjacent. For every vertex z the distances $d_u = \mathrm{dist}(u,z)$ and $d_v = \mathrm{dist}(v,z)$ differ by at most one. If both are at most R, the entries $a = k_u(z)$ and $b = k_v(z)$ are equal or differ by the factor λ, and in either case $ab \ge \kappa(a^2+b^2)/2$; this needs no girth assumption. If exactly one is at most R, it equals R, so z lies on the sphere of radius R around u or around v; the nonzero entry is λ^R and the product is zero. In maximum degree three a vertex at distance $d+1$ has a neighbour at distance d, and a vertex at positive distance d has at most two neighbours at distance $d+1$, so a sphere of radius R has at most $3 \cdot 2^R$ vertices (`Frontier.Gaussian.card_sphere_le_degree`). There are thus at most $6 \cdot 2^R$ vertices of the

second kind, each costing at most $\lambda^{2R}/2$. Summing over z,

$$\langle k_u, k_v \rangle \;\geq\; \kappa\,\frac{\|k_u\|^2 + \|k_v\|^2}{2} - 3 \cdot 2^R \lambda^{2R} \;\geq\; \kappa\,\|k_u\|\,\|k_v\| - 3(2\lambda^2)^R,$$

and dividing by $\|k_u\|\,\|k_v\| \geq 1$ gives the claim. □

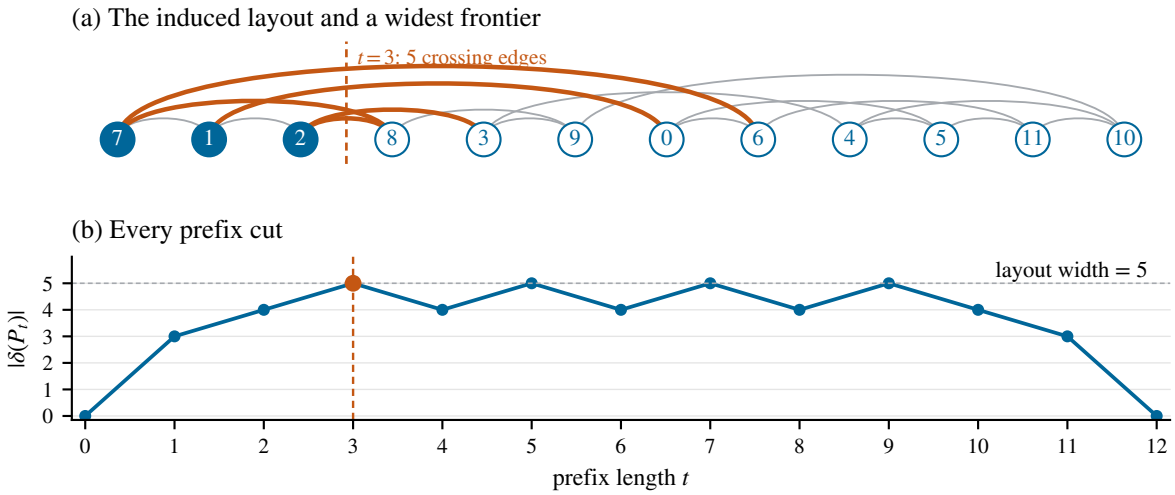


Figure 3: The layout from Fig. 2. **(a)** The median order from left to right, with the first 3 vertices filled and the 5 crossing edges highlighted. **(b)** All prefix cuts; this order has width 5. The finite sample illustrates the construction; the asymptotic coefficient is proved below. The generator and numerical data are included in the repository.

Lemma 7.5 (Sheppard's bound at every threshold)**.** *For unit vectors α, β with $\langle \alpha, \beta \rangle = \rho > -1$ and any threshold θ, the probability that θ separates $\langle \alpha, \omega \rangle$ and $\langle \beta, \omega \rangle$ is at most* $\arccos(\rho)/\pi$.

`Frontier.Gaussian.gaussPi_between_le_arccos`

Proof. The forms $S = \langle \alpha + \beta, \omega \rangle$ and $D = \langle \beta - \alpha, \omega \rangle$ are uncorrelated jointly Gaussian, hence independent, and a separation forces $|S - 2\theta| \leq |D|$. Conditionally on D, a centered Gaussian gives a symmetric interval at least the mass of any translate (Anderson's inequality on the line), so the probability is at most $\Pr[|S| \leq |D|]$. By the planar angle law this is $\frac{2}{\pi} \arctan \sqrt{(1-\rho)/(1+\rho)} = \arccos(\rho)/\pi$. □

Proposition 7.6 (One vertex)**.** *If adjacent kernel rows have correlation at least $\rho_0 \geq 17/32$, a vertex straddles any fixed threshold with probability at most* $\frac{3}{2\pi} \arccos \frac{1+3\rho_0}{4}$.

`Frontier.Gaussian.measureReal_straddleEvent_le`

The threshold $17/32$ is a convenience of the formal proof; the argument below needs only $\rho_0 \geq 0$.

Proof. A straddling vertex has at least four separated ordered pairs among its three edges, so the probability is at most a quarter of the sum, over the six ordered pairs, of the separation probabilities, each at most $\arccos(\rho')/\pi$ by Lemma 7.5. Each edge vector at v has correlation $\sqrt{(1+\rho)/2} \geq \kappa = \sqrt{(1+\rho_0)/2}$ with x_v. Since $\|\sum_e y_e\| \geq \langle \sum_e y_e, x_v \rangle \geq 3\kappa$, the pairwise correlations average at least $(3\kappa^2 - 1)/2 = (1 + 3\rho_0)/4$. Each pairwise correlation is at least $2\kappa^2 - 1 = \rho_0 \geq 0$, by the triangle inequality for angles, and as arccos is concave and decreasing on $[0, 1]$, the six angles

sum to at most $6 \arccos \frac{1+3\rho_0}{4}$ (`Frontier.Gaussian.sum_arccos_star_le`). So the probability is at most $\frac{1}{4} \cdot 6 \cdot \frac{1}{\pi} \arccos \frac{1+3\rho_0}{4}$. □

Proposition 7.7 (Sharpness of the local Gaussian estimate; not formalized)**.** *For the radial kernels on the infinite 3-regular tree, first letting the truncation radius tend to infinity and then $\lambda \uparrow 1/\sqrt{2}$, the probability that a vertex straddles zero tends to p. Thus the limiting constant in Proposition 7.6 is attained within this family of local score distributions.*

Proof. For fixed $0 < \lambda < 1/\sqrt{2}$, the untruncated row $k_v(z) = \lambda^{\mathrm{dist}(v,z)}$ is square summable, with $\|k_v\|^2 = (1+\lambda^2)/(1-2\lambda^2)$. Summing along the path between two vertices gives correlations between normalized rows at distances one and two

$$r_1 = \frac{2\lambda}{1+\lambda^2}, \qquad r_2 = \frac{\lambda^2(3-\lambda^2)}{1+\lambda^2}.$$

For example, for distance two the three path vertices contribute $3\lambda^2$ and their off-path branches contribute $5\lambda^4/(1-2\lambda^2)$. All pairs of the three incident edge scores therefore have correlation

$$c_\lambda = \frac{1+2r_1+r_2}{2+2r_1} \longrightarrow \frac{1+2\sqrt{2}}{4}.$$

Except on a null set, exactly two of the three unordered edge pairs have opposite signs when a vertex straddles zero, and none otherwise. The zero-threshold Gaussian angle formula gives probability $3 \arccos(c_\lambda)/(2\pi)$. Truncated rows converge in norm, so their local score distributions and straddling probabilities converge to these values. The limiting row correlations are those of the cubic-tree Gaussian wave of [CGHV15, Lyo17]. This is sharpness of the local probability estimate, not an optimality assertion for layouts or for the global cutwidth coefficient. □

Proof of Theorem 7.1. Let H be a simple cubic graph on h vertices, and let $\lambda \geq 0$, R, and $\rho_0 \geq 17/32$ satisfy $\rho_0 \leq 2\lambda/(1+\lambda^2) - 3(2\lambda^2)^R$, so that adjacent unit kernel rows have correlation at least ρ_0 (Lemma 7.4). Fix $T > 0$, an integer $M \geq 1$, $\delta = 2T/M$, $\varepsilon > 0$, and the thresholds $\theta_i = -T + i\delta$ for $i \leq M$. Consider the $2M+2$ counts

- the number Y_i of vertices straddling θ_i, for $i < M$;
- the number Z_i of edges scoring in the window $[\theta_i, \theta_{i+1})$, for $i < M$;
- the numbers Z_- and Z_+ of edges scoring below $-T$ and at least T.

Means. Every score is a standard Gaussian, since y_e is a unit vector. So $\mathbb{E}Y_i \leq \frac{3}{2\pi} \arccos \frac{1+3\rho_0}{4}\, h$ (Proposition 7.6), $\mathbb{E}Z_i \leq \frac{3h}{2} \cdot \frac{\delta}{\sqrt{2\pi}}$, and $\mathbb{E}Z_\pm \leq \frac{3h}{2} \cdot \frac{1}{T^2}$ by Chebyshev's inequality.

Concentration. The kernel row of v is supported on the ball of radius R around v, so the score of an edge uv depends only on the coordinates of ω in the ball of radius $R+1$ around u, and the event that a vertex straddles a threshold only on those in the ball of radius $R+1$ around it (`Frontier.Gaussian.dependsOn_straddleEvent`). Each such ball meets at most D others, where D depends only on R, since balls of radius $2R+2$ have boundedly many vertices (`Frontier.Gaussian.card_ball_le_degree`). Events depending on disjoint sets of independent coordinates are independent, so a count Y of F such events has $\mathrm{Var}\, Y \leq |F|\, D$, and by Chebyshev's inequality $\Pr[|Y - \mathbb{E}Y| \geq \varepsilon h] \leq |F|\, D/(\varepsilon h)^2$ (`Frontier.Gaussian.pi_deviation_le`). Every count has $|F| \leq 3h/2$ events, so all $2M+2$ counts lie within εh of their means except with probability at most $(3M+3)D/(\varepsilon^2 h)$, which is less than one for $h > (3M+3)D/\varepsilon^2$.

The layout. For such a sample, Lemma 7.3 with $a = -T$ bounds every frontier of the median layout by

$$h \cdot \max\Big(\tfrac{9}{2T^2} + 3\varepsilon,\ \tfrac{3}{2\pi}\arccos\tfrac{1+3\rho_0}{4} + \tfrac{9\delta}{2\sqrt{2\pi}} + 4\varepsilon\Big)$$

(`Frontier.Gaussian.exists_good_sample`).

The parameters. As $\lambda \uparrow 1/\sqrt{2}$ the correlation $2\lambda/(1+\lambda^2)$ increases to $2\sqrt{2}/3$, the neighbour correlation of Gaussian waves on the 3-regular tree [CGHV15, Lyo17], and for fixed $\lambda < 1/\sqrt{2}$ the error $3(2\lambda^2)^R$ tends to zero as $R \to \infty$ (`Frontier.Gaussian.exists_decay_radius`). So for every $\rho_0 < 2\sqrt{2}/3$ there are admissible λ and R. As $\rho_0 \uparrow 2\sqrt{2}/3$, $(1+3\rho_0)/4 \to (1+2\sqrt{2})/4$. Given $\xi > 0$, choose ρ_0 with $\frac{3}{2\pi}\arccos\frac{1+3\rho_0}{4} \le p + \xi/3$, then $T = 20$, $\varepsilon \le \xi/12$ and M large enough that the bound above is at most $(p+\xi)h$ (`Frontier.Gaussian.exists_parameters`). This shows that large cubic graphs are p-layable (`Frontier.Gaussian.cubicLayoutBound_gaussian`), and Proposition 7.2 gives $\mathrm{LB}_3(2p)$. Finally $\cos(3\pi/32) \le (1+2\sqrt{2})/4$, so $\arccos\frac{1+2\sqrt{2}}{4} \le 3\pi/32$ and $2p \le 9/32$ (`Frontier.Gaussian.two_mul_gaussianCoefficient_le`). □

Corollary 7.8 (The unconditional bounds)**.** *Under the hypotheses of Theorem 1.1 on S_n, every circuit of fan-in at most two deciding S_n, over any basis, has more than $(L-\varepsilon)\,n$ inner gates for all large n, where $L = 1 + \pi/(3\arccos\frac{1+2\sqrt{2}}{4}) \approx 4.5625$; in particular more than $(4-\varepsilon)\,n$.*

`Frontier.lowerBound_gaussian`

The coefficient $1/3$ was known earlier, from the cubic bisection bound of Monien and Preis [MP06] and the resulting pathwidth bound of Fomin and Høie [FH06]; here it is a consequence of Theorem 7.1.

7.4 Limits of edge layouts

Random cubic graphs on h vertices have bisection width at least $0.103295\,h$ with high probability [LM23], and the cutwidth of a graph is at least its bisection width, since the middle frontier of any layout is a bisection. A cubic graph on h vertices has cycle rank $h/2+1$, so $\mathrm{LB}_3(A)$ is false for $A < 0.20659$: with the layout bound as its only graph-theoretic input, the method cannot give a coefficient above $1 + 1/0.20659 \approx 5.8405$ for fan-in two. This is a limitation of the universal layout hypothesis. A sharper theorem for the particular graphs produced by circuits could evade it; other possibilities are a finer measure of frontiers or a different compiler.

7.5 Larger fan-in

The layout hypothesis has an unconditional instantiation in every fixed degree.

Theorem 7.9 (Every fixed degree)**.** *Let G be a nonempty connected multigraph on N vertices, of maximum degree at most d. There is a layout of width at most*

$$\beta_1(G) + d\lfloor \log_2 N \rfloor.$$

Consequently $\mathrm{LB}_d(1)$ *holds for every fixed d.*

`Frontier.Multigraph.exists_layout_cycleRank`, `Frontier.layoutBound_one`

Proof. Choose a spanning tree T. A tree of maximum degree d has cutwidth at most $d\lfloor \log_2 N \rfloor$: remove a centroid, recursively list the resulting components, each of size at most $N/2$, and put the centroid last. At a cut within one component, at most d edges to the centroid supplement the recursive cut. The recurrence terminates at single vertices, whose internal cuts are empty. A centroid exists by repeatedly moving into a component containing more than half the vertices, if there is one; its size strictly decreases at each move.

There are exactly $\beta_1(G)$ edges outside T. Restoring them adds at most that many edges to any cut. The logarithmic term is at most $\eta N + C_{d,\eta}$, proving the stated layout hypothesis. Lean obtains the same tree bound by the ordered-block merges of Proposition 7.2: in a tree the quotient has a block of boundary at most two, which can merge with a neighbour until only one block remains. □

Corollary 7.10 (All fixed fan-ins)**.** *Under the hypotheses on S_n in Theorem 1.1, for every fixed $r \geq 2$ and $\varepsilon > 0$, all sufficiently large n satisfy*

$$(r-1)s > (2-\varepsilon)n$$

for every circuit of fan-in at most r deciding S_n. `Frontier.lowerBound_all_fanIn`

Proof. Apply Theorem 1.1 with $A = 1$ and Theorem 7.9 at degree $r+1$. □

These are baseline consequences of the layout method. The following Gaussian construction sharpens them in every fixed degree. We make no claim that these higher-fan-in constants are optimal or historically best possible in their respective circuit models.

Theorem 7.11 (Gaussian vertex layouts in bounded degree)**.** *For every fixed integer $d \geq 2$ and $\xi > 0$, every sufficiently large loopless multigraph H of maximum degree at most d, with h vertices and m edges, has a layout of width at most*

$$\gamma_d m + \xi h, \qquad \rho_d = \frac{2\sqrt{d-1}}{d}, \qquad \gamma_d = \frac{\arccos \rho_d}{\pi}.$$

`Frontier.Gaussian.multigraph_vertexLayout_bound`

Proof. Use the normalized truncated distance kernels of Lemma 7.4 on the underlying simple graph, whose maximum degree is at most d, with $0 < \lambda < 1/\sqrt{d-1}$. A sphere of positive radius j has at most $d(d-1)^{j-1}$ vertices. The pointwise kernel defect proof is unchanged: adjacent distances differ by at most one, and a one-sided truncation error lies on a radius-R sphere. Thus adjacent unit rows satisfy

$$\langle x_u, x_v \rangle \geq \frac{2\lambda}{1+\lambda^2} - d\big((d-1)\lambda^2\big)^R.$$

This tends to ρ_d by first taking λ close to $1/\sqrt{d-1}$ from below and then taking R large. This estimate holds on every graph of the stated maximum degree, including graphs with short cycles (`Frontier.Gaussian.sum_unitKernel_mul_ge_degree`).

Give vertex v the score $Z_v = \langle x_v, \omega \rangle$ and order vertices by score, breaking ties by a fixed enumeration. If the adjacent correlation is at least ρ, Lemma 7.5 bounds the probability that an edge crosses any fixed threshold by $\arccos \rho/\pi$. The mean crossing count is therefore at most that quantity times m.

Here is the simultaneous estimate needed to obtain an ordering. Fix a grid of M windows covering $[-T, T]$, of length $\delta = 2T/M$. Count crossing edges at each grid threshold, vertices in each window, and vertices in either tail. Each indicator depends on a bounded ball of independent Gaussian coordinates. For fixed d, R, each support meets only boundedly many others, so the second-moment argument in the proof of Theorem 7.1 bounds every variance by $O_{d,R}(h)$. Parallel edges remain distinct summands: at most d edges are incident to any vertex, so multiplicity also respects the bounded-overlap estimate. No cut cardinality is replaced by a count in the underlying simple graph. For fixed M and $\epsilon > 0$, a union bound gives a sample in which all counts are at most their means plus ϵh, once h is large enough.

For a prefix ending at a score in a grid window, every crossing edge either crosses the window's left threshold or touches a vertex in the window. A window has mean at most $h\delta/\sqrt{2\pi}$ vertices, and each vertex touches at most d edges. A prefix ending in either tail is bounded by d times the corresponding tail count, whose mean is at most h/T^2. Consequently every prefix is bounded by

$$\frac{\arccos\rho}{\pi}m + h\left(\frac{d}{T^2} + \frac{d\delta}{\sqrt{2\pi}} + (d+1)\epsilon\right).$$

The window argument includes all tied vertices. Choose $\rho < \rho_d$ so that the crossing coefficient differs from γ_d by at most $\xi/(4d)$, then choose T, M, ϵ to make each of the three displayed error terms at most $\xi/4$. Since $m \le dh$, the result follows. The multigraph ordering and sampling steps are checked separately by `Frontier.Multigraph.exists_key_of_vertexScore` and `Frontier.Gaussian.exists_multi_vertex_sample`. □

Theorem 7.12 (Gaussian cycle-rank coefficients in every degree)**.** *For every fixed integer $d \ge 3$, the layout bound $\mathrm{LB}_d(a_d)$ holds, where*

$$a_d = \frac{3d}{2d-3}\,\frac{\arccos(2\sqrt{d-1}/d)}{\pi}.$$

Moreover, $0 < a_d < 3/4$ and $a_4 = 2/5$.

`Frontier.layoutBound_degree`, `Frontier.Gaussian.degreeCoefficient_lt_three_quarters`

Proof. Run the ordered-block compression of Proposition 7.2 with boundary budget d. The doubling invariant gives internal prefix boundary at most $d\log_2|B|$. If more than one block remains, form the loopless quotient multigraph H, keeping every edge between distinct blocks. Write $h = |V(H)|$ and $m = |E(H)|$. Adjacent blocks of degrees $a, b \le d$, joined by $e \ge 1$ edges, satisfy $a + b - 2e > d$ at termination. Thus $a + b \ge d + 3$ and every degree lies between 3 and d; connectivity ensures a neighbour for each block.

Charge weight $d + 3 - a$ to each incidence at a vertex of degree a. Its total contribution is

$$a(d+3-a) = 3d + (a-3)(d-a) \ge 3d.$$

Each edge receives weight $2(d+3) - a - b \le d + 3$, so $3dh \le (d+3)m$. If the original graph has N vertices and M edges, compression gives $m + N \le M + h$: every merge moves at least one edge inside a block. Combining these yields

$$(2d-3)m \le 3d(M-N) \le 3d\beta_1(G).$$

All counts retain multiplicity. The incidence and compression estimates are checked separately.

`Frontier.Multigraph.core_density`

`Frontier.Multigraph.Compression.Terminal.multi_edge_bound`

For a sufficiently large quotient, Theorem 7.11 gives width at most $\gamma_d m + \xi h$. Since $h \le m \le 3\beta_1(G)$, taking $\xi = \eta/3$ bounds this by $(a_d + \eta)\beta_1(G)$. Expanding the block lists adds at most $d\log_2 N + d$; absorb this in $\eta N + C_{d,\eta}$. Quotients below the fixed size threshold have boundedly many edges and contribute only to the constant. A single block is handled by the same internal-prefix invariant.

Finally, $\rho_d < 1$ gives positivity. The inequality $\sin t \le t$ applied to $t = \pi/2 - \arccos\rho_d$ gives $\arccos\rho_d \le \pi/2 - \rho_d$. Since $8\sqrt{d-1} > 3\pi$ for $d \ge 3$, substitution gives $a_d < 3/4$. At degree four, $\rho_4 = \sqrt{3}/2$ and $\gamma_4 = 1/6$, so $a_4 = 2/5$. □

Corollary 7.13 (Gaussian bounds for every fixed fan-in)**.** *Under the hypotheses on S_n in Theorem 1.1, for every fixed $r \geq 2$ and $\varepsilon > 0$, every circuit of fan-in at most r deciding S_n satisfies*

$$(r-1)s > \left(1 + \frac{1}{a_{r+1}} - \varepsilon\right) n$$

for all sufficiently large n. Since $a_{r+1} < 3/4$ by Theorem 7.12, this implies $(r-1)s > (7/3 - \varepsilon)n$.

`Frontier.lowerBound_degree`

Proof. Apply Theorem 1.1 with $A = a_{r+1}$. The same substitution gives the coefficients for totally regular maps over finite fields with arbitrary operations, over infinite fields with polynomial operations, and for finite-alphabet MDS maps (`Frontier.linear_finite_degree`, `Frontier.linear_polynomial_degree`, `Frontier.mds_degree`). □

For example, $a_5 = 15 \arccos(4/5)/(7\pi)$ gives a gate coefficient $(1 + 1/a_5)/3 \approx 1.092760448$ for fan-in four. These are improvements over the spanning-tree baseline within this method; at fan-in two the median-edge theorem is stronger.

Theorem 7.14 (The degree-four layout constant)**.** $\mathrm{LB}_4(2/5)$ *holds.*

`Frontier.layoutBound_two_fifths`

Proof. Specialize Theorem 7.12 to $d = 4$. In this case the terminal quotient is even simple: $a + b - 2e > 4$ with $a, b \leq 4$ forces $e = 1$. Its degrees are three or four, and no edge joins two degree-three vertices. The incidence estimate becomes $12h \leq 7m$, hence $5m \leq 12\beta_1(G)$; combining with $\gamma_4 = 1/6$ gives $2/5$. □

Corollary 7.15 (Ternary gates)**.** *Under the hypotheses on S_n in Theorem 1.1, every circuit of fan-in at most three deciding S_n has*

$$s > \left(\frac{7}{4} - \varepsilon\right) n$$

for every fixed $\varepsilon > 0$ and all sufficiently large n. `Frontier.lowerBound_fanInThree`

Proof. Apply Theorem 1.1 with $r = 3$ and $A = 2/5$: the bound is $2s > (7/2 - 2\varepsilon)n$. The same substitution in Theorem 8.6 proves the ternary bound for totally regular maps over finite fields with arbitrary bases, and over infinite fields with polynomial operations (`Frontier.linear_finite_fanInThree`, `Frontier.linear_polynomial_fanInThree`). □

8 Linear maps and arithmetic circuits

Let $\mathbb{F}$ be a field, $M \in \mathbb{F}^{N \times N}$, and let a circuit with N inputs and N outputs compute $x \mapsto Mx$. Compile it with every value accepted at every output, so that its network is satisfied on every input, by the run alone (Theorem 5.1). The graph has the shape and size of Section 5; what changes is the demand.

8.1 Totally regular maps

Definition 8.1 (Totally regular matrices)**.** A matrix is *totally regular* if every square submatrix, on distinct rows and distinct columns, is nonsingular. `Frontier.TotallyRegular`

Equivalently, for sets of rows R and columns C with $|R| = |C|$, a vector supported on C whose image vanishes on R is zero (`Frontier.TotallyRegular.eq_zero`). The standard examples are Cauchy matrices.

Lemma 8.2 (Cauchy matrices)**.** *Let $a_0, a_1, \ldots$ be distinct, $b_0, b_1, \ldots$ distinct, and $a_i \neq b_j$. The matrix $\left(1/(a_i - b_j)\right)_{i,j}$ is totally regular. In particular $M_N = \left(1/(i - N - j)\right)_{0 \leq i,j < N}$ is totally regular over every field of characteristic zero, and over $\mathbb{Z}/q$ for every prime $q \geq 2N$.*

`Frontier.totallyRegular_cauchy`

Proof. A square submatrix of a Cauchy matrix is a Cauchy matrix, so it suffices that square Cauchy matrices are nonsingular. Let $a_1, \ldots, a_k$ be distinct, $b_1, \ldots, b_k$ distinct, and $a_i \neq b_j$, and suppose that $\sum_j v_j/(a_i - b_j) = 0$ for every i. The polynomial $P(t) = \sum_j v_j \prod_{l \neq j}(t - b_l)$ has degree less than k and satisfies $P(t) = \prod_l (t - b_l) \sum_j v_j/(t - b_j)$ for $t \notin \{b_1, \ldots, b_k\}$, so it vanishes at the k points a_i. Hence $P = 0$, and $0 = P(b_j) = v_j \prod_{l \neq j}(b_j - b_l)$ gives $v_j = 0$ (`Frontier.det_cauchy_ne_zero`). For M_N, the nodes i and $N + j$ are distinct integers below $2N$, which remain distinct in characteristic zero and modulo a prime $q \geq 2N$. □

Lemma 8.3 (Some frontier determines the input)**.** *Let a circuit compute $x \mapsto Mx$ for a totally regular $M \in \mathbb{F}^{N \times N}$. Every layout of its network has a frontier whose values in the run determine the input.*

`Frontier.exists_injective_cut`

Proof. Every output depends on every input, since all entries of M are nonzero, so every input is read (`Frontier.upstream_of_totallyRegular`). For a time t, let X be the inputs read by the first t vertices and Y the outputs whose vertices are among them. The number $|X| + |Y|$ starts at 0, ends at $2N$, and grows by at most one per vertex, since a vertex reads at most one input and is at most one output vertex; so at some time $|X| + |Y| = N$, that is, $|Y| = |X^c|$ and $|Y^c| = |X|$.

Let the runs on x and x' agree on the frontier at that time. Splice them: by Lemma 5.2, the input z that agrees with x on X and with x' on X^c has the outputs of x in Y, so $M(z - x)$ vanishes on Y, and $z - x$ is supported on X^c. As M is totally regular and $|Y| = |X^c|$, $z = x$, that is, $x' = x$ on X^c. Symmetrically, the input that agrees with x' on X and with x on X^c has the outputs of x outside Y, and $|Y^c| = |X|$ gives $x' = x$ on X. □

The demand is exact: a frontier with N values, with no lower-order loss.

8.2 Capacity

How many values does a frontier of k edges carry? Over a finite field, k wires take at most $|\mathbb{F}|^k$ values. Over an infinite field there is no counting, and in fact no bound: a bijection $\mathbb{F} \times \mathbb{F} \to \mathbb{F}$ lets a single wire carry everything. The right notion is a property of the basis.

Definition 8.4 (Linear capacity)**.** A circuit over $\mathbb{F}$ with N inputs has *linear capacity* if whenever the values of k of its wires determine the input, $N \leq k$.

`Frontier.LinearCapacity`

Lemma 8.5 (Capacity)**.** *Every circuit over a finite field has linear capacity, whatever its basis. Every circuit over the affine basis, whose operations are $x \mapsto \sum_i a_i x_i + b$, has linear capacity over every field.*

`Frontier.linearCapacity_affine`

Proof. Over a finite field, an injective map from $\mathbb{F}^N$ to $\mathbb{F}^k$ forces $|\mathbb{F}|^N \leq |\mathbb{F}|^k$ (`Frontier.linearCapacity_of_finite`). Over the affine basis, the value of every wire is an affine function of the input (`Frontier.exists_affine_trace`), and an injective affine map from $\mathbb{F}^N$ to $\mathbb{F}^k$ has an injective linear part, so $N \leq k$. □

Theorem 8.6 (Linear maps)**.** *Assume $\mathrm{LB}_{r+1}(A)$. For every $\varepsilon > 0$ and all large N, every circuit with linear capacity, with s inner gates of fan-in at most r, over any field $\mathbb{F}$ and any basis, computing $x \mapsto Mx$ for a totally regular $M \in \mathbb{F}^{N \times N}$, has $(r - 1)s > (1 + 1/A - \varepsilon)\, N$.*

`Frontier.lowerBound_linear`

Proof. The network is connected, since all outputs depend on the first input, loopless and of maximum degree $r+1$; it reads all N inputs, so $\beta_1 \le (r-1)s+1-N$, and it has at most $2rs+3N$ vertices (Propositions 5.4 and 5.5). By Lemma 8.3 and linear capacity, every layout has a frontier of at least N edges (`Frontier.exists_graph_of_totallyRegular`). Apply Lemma 6.1 with $e(n)=1$ and $\kappa = 4$. □

8.3 Linearization

Lemma 8.7 (Linearization)**.** *Let $\mathbb{F}$ be infinite, and let a circuit whose operations are polynomials compute $x \mapsto Mx$. Replace each gate g, with polynomial P and arguments taking the values z on the zero input, by the affine operation $y \mapsto P(z) + \sum_a \partial_a P(z)\,(y_a - z_a)$. The resulting circuit over the affine basis has the same wires, the same inner gates and the same fan-in, and computes $x \mapsto Mx$.* `Frontier.exists_affine_circuit`

Proof. Evaluate the circuit over the dual numbers $\mathbb{F}[\epsilon]/(\epsilon^2)$ at the input ϵx. A polynomial satisfies $P(z+\epsilon v) = P(z) + \epsilon \sum_a \partial_a P(z)\, v_a$ (`Frontier.snd_aeval_dualNumber`), so by induction along the program, if a wire takes the value $a + \epsilon b$ there, the linearized circuit computes $a+b$ at x (`Frontier.trace_linearize`). At an output, compose the gate polynomials into a polynomial in x. It agrees with $\sum_j M_{ij} x_j$ as a function, hence as a polynomial, because $\mathbb{F}$ is infinite, so its value at ϵx is $\epsilon (Mx)_i$, and the linearized circuit computes $0 + (Mx)_i$. □

Over a finite field, functions do not determine polynomials ($x^q = x$), but there counting needs no linearization.

Corollary 8.8 (Finite fields, polynomial operations, arithmetic circuits)**.** *With $L \approx 4.5625$, for every $\varepsilon > 0$ and all large N:*

(a) over every finite field, every circuit of fan-in two over any basis computing a totally regular map has more than $(L-\varepsilon)N$ inner gates; in particular for M_N over $\mathbb{Z}/q$, uniformly in primes $q \ge 2N$;

(b) over every infinite field, the same holds for every basis of fan-in two whose operations are polynomials;

(c) over every field of characteristic zero, every arithmetic circuit computing $x \mapsto M_N x$ uses more than $(L-\varepsilon)N$ additions and multiplications, constants being free.

`Frontier.arith_cauchy_gaussian`

Proof. Combine Theorem 8.6 with Theorem 7.1, Lemmas 8.5 and 8.7, and Lemma 8.2. Arithmetic circuits, with addition and multiplication of arity two and a constant of arity zero for every element of the field, have fan-in two and polynomial operations, and their inner gates are the additions and multiplications. □

Remark 8.9. Over $\mathbb{R}$, continuous operations would also do, without derivatives: a frontier of k values that determines N inputs is a continuous injection from an open subset of $\mathbb{R}^N$ into $\mathbb{R}^k$, so $N \le k$ by invariance of domain. We have not formalized this variant.

8.4 Scalar polynomials and mixed Hessians

Mixed second derivatives give a demand for scalar polynomials, whose Jacobians have only one row. Derivative bounds on communication go back to Abelson [Abe78]; the scalar-Hessian route was suggested in [Fro]. Our proof uses compiler splicing and uniqueness over every field.

Throughout this subsection a circuit has polynomial operations and computes $f \in \mathbb{F}[x_1, \dots, x_N]$ *as a formal polynomial.* Over finite fields this is stronger than equality of the induced functions. There is one scalar output; fan-out and polynomial degrees are unrestricted, and constants are free. For $a \in \mathbb{F}^N$ and $S \subseteq [N]$, write

$$H_f(a)[S, S^c] = \left(\frac{\partial^2 f}{\partial x_i \partial x_j}(a) \right)_{i \in S,\ j \notin S}.$$

Lemma 8.10 (A bilinear rank bound)**.** *Let $B : X \times Y \to \mathbb{F}$ be bilinear, and let $L : X \to W$ and $R : Y \to W$ be linear maps between finite-dimensional spaces. If $\operatorname{im} L \cap \operatorname{im} R = \{0\}$ and $B(\ker L, \ker R) = 0$, then*

$$\operatorname{rank} B \le \operatorname{rank} L + \operatorname{rank} R \le \dim W.$$

`Frontier.Arithmetic.bilinear_rank_le_boundary`

Proof. Viewed as a map $X \to Y^*$, B sends $\ker L$ into the annihilator of $\ker R$, which has dimension $\operatorname{rank} R$. The remaining quotient $X/\ker L$ has dimension $\operatorname{rank} L$. The disjoint-image hypothesis gives the second inequality. □

Theorem 8.11 (Mixed-Hessian frontier lemma)**.** *Let P be any set of vertices of the compiled network, let S be the inputs read in P, and put $w = |\delta_G(P)|$. For every $a \in \mathbb{F}^N$,*

$$\operatorname{rank} H_f(a)[S, S^c] \le w.$$

The output vertex may lie on either side of the cut.

`Frontier.Arithmetic.rank_mixedHessian_le_cut`

Proof. Let $q = (q_e)_{e \in \delta_G(P)}$ be the vector of formal polynomials carried by the cut edges. At a, restrict its differential to directions on the two input sides:

$$L : \mathbb{F}^S \to \mathbb{F}^w, \qquad R : \mathbb{F}^{S^c} \to \mathbb{F}^w.$$

We extend such directions by zero outside their indicated side.

First, $Lu + Rv = 0$ implies $Lu = Rv = 0$. Evaluate the circuit over $\mathbb{F}[\epsilon]/(\epsilon^2)$ on $a + \epsilon(u + v)$ and on a. Their cut values agree. Splicing the first run on P with the second off P gives a satisfying assignment on $a + \epsilon u$. By uniqueness (Theorem 5.1), this is the actual run, and it retains the common cut values. Taking the coefficient of ϵ gives $Lu = 0$, and then $Rv = 0$. Applying this to $Lu - Rv = 0$ proves that the two images intersect trivially. This is the reason their dimensions can be added: disjoint sets of input coordinates alone would not imply disjoint images in the space of cut values. `Frontier.Compiler.trace_cut_of_cut`, `Frontier.Arithmetic.Splices.split`

Next suppose $Lu = 0$ and $Rv = 0$. Work over

$$J = \mathbb{F}[\epsilon, \delta]/(\epsilon^2, \delta^2).$$

The runs on $a + \epsilon u$ and $a + \delta v$ both carry $q(a)$ on the cut. Splice them to obtain the run on $a + \epsilon u + \delta v$. Its output is $f(a + \epsilon u)$ if the output vertex lies in P, and $f(a + \delta v)$ otherwise, by Lemma 5.2. In either case its $\epsilon\delta$ coefficient is zero. But polynomial expansion identifies this coefficient with

$$\sum_{i \in S} \sum_{j \notin S} \frac{\partial^2 f}{\partial x_i \partial x_j}(a) u_i v_j.$$

Thus the mixed-Hessian form vanishes on $\ker L \times \ker R$. Lemma 8.10 proves the result. No factorial is divided out: the two square-zero directions are independent. Lean uses the iterated dual-number algebra $\mathbb{F}[\epsilon][\delta]$ and checks the polynomial expansion in `Frontier.Arithmetic.jet_mixed`.

`Frontier.Arithmetic.Splices.mixed_zero`

□

Definition 8.12 (Mixed-Hessian demand)**.** A polynomial f has *mixed-Hessian demand at least h* if, for every ordering π of its N input coordinates, some prefix S and some point $a \in \mathbb{F}^N$ satisfy $\operatorname{rank} H_f(a)[S, S^c] \geq h$. The point may depend on the ordering and the prefix. A bound at one fixed point for all orderings is sufficient, but is not required. `Frontier.Arithmetic.HessianDemand`

The Lean definition uses injective positions in $\mathbb{N}$, allowing gaps. This gives the same input orders and also applies directly to the positions of input vertices inside a circuit layout.

Theorem 8.13 (Scalar-polynomial lower-bound transfer)**.** *Assume* $\mathrm{LB}_3(A)$ *with* $A > 0$. *For every* $\eta > 0$ *there is a constant* C_η *such that every fan-in-two polynomial circuit computing a polynomial* f *in which all* N *variables occur, with mixed-Hessian demand at least* h *and* s *inner gates, satisfies*

$$(A + \eta)N + h \leq (A + 5\eta)s + A + 4\eta + C_\eta. \tag{2}$$

Consequently, for each fixed $\rho \geq 0$ *and* $\varepsilon > 0$*, for all sufficiently large* N*, demand at least* ρN *implies*

$$s > \left(1 + \frac{\rho}{A} - \varepsilon\right) N.$$

The size threshold is uniform over fields and polynomial gate bases.

`Frontier.Arithmetic.hessian_gate_bound`, `Frontier.Arithmetic.lowerBound_hessian`

Proof. Every variable occurring formally in the output is upstream of that output, so all inputs are read. With one output the compiled graph is connected, and the compiler gives $\beta_1(G) + N \leq s + 1$ and $|V| \leq 4s + 3$. Each graph layout induces an input ordering. Its demanded prefix is realized by a vertex cut, and Theorem 8.11 makes that cut have at least h edges. Apply $\mathrm{LB}_3(A)$ and obtain

$$h \leq (A + \eta)\beta_1(G) + \eta|V| + C_\eta \leq (A + \eta)(s + 1 - N) + \eta(4s + 3) + C_\eta.$$

This is (2). For $h \geq \rho N$, first choose η small enough that $(A + \eta + \rho)/(A + 5\eta) > 1 + \rho/A - \varepsilon$, then absorb the constant term by taking N large.

`Frontier.Arithmetic.upstream_of_mem_vars`, `Frontier.Arithmetic.exists_cut_of_hessianDemand`

□

In particular, Theorem 7.1 supplies the checked Gaussian value $A \approx 0.280701937$ in this theorem (`Frontier.Arithmetic.lowerBound_hessian_gaussian`). We next verify the input-order demand for the determinant, following the permutation-matching argument proposed in [Fro].

Lemma 8.14 (Permutation matchings across a partition)**.** *For* $S \subseteq [n] \times [n]$ *and a permutation* $\pi \in \mathfrak{S}_n$*, put*

$$c_\pi(S) = |\{(i, j) : i \neq j,\ (i, \pi(j)) \in S,\ (j, \pi(i)) \notin S\}|.$$

There is a permutation π *such that*

$$|S|\,|S^c| \leq n^2 c_\pi(S) + 2n^3.$$

Each split pair is counted once, from its endpoint in S*.*

`Frontier.Arithmetic.DeterminantMatching.exists_crossing`

Proof. Let G count pairs $(x, y) \in S \times S^c$ whose entries lie in distinct rows and distinct columns. For each x, at most n choices of y share its row and at most n share its column. Hence

$$G \geq |S|\,|S^c| - 2n|S| \geq |S|\,|S^c| - 2n^3.$$

For $n \geq 2$, each such pair occurs in the matching of a uniform permutation with probability $1/(n(n-1))$: two distinct rows must be sent to two prescribed distinct columns. Therefore

$$\mathbb{E}_\pi c_\pi(S) = \frac{G}{n(n-1)} \geq \frac{G}{n^2},$$

and some permutation attains at least the average. The cases $n \leq 1$ are immediate. The formal proof uses finite double counting: permutations act transitively on ordered pairs of distinct indices, and all arithmetic is in $\mathbb{N}$ before the final rank estimate.

`Frontier.Arithmetic.DeterminantMatching.sum_pair_action`

□

Theorem 8.15 (Determinant demand)**.** *Over every field, the formal determinant polynomial in $N = n^2$ variables has mixed-Hessian demand at least*

$$\frac{n^2}{4} - 3n.$$

More precisely, for every partition $S \subseteq [n] \times [n]$ with $|S| = \lfloor n^2/2 \rfloor$, there is a permutation matrix P_π such that

$$\operatorname{rank} H_{\det}(P_\pi)[S, S^c] \geq \frac{n^2}{4} - 3n.$$

`Frontier.Arithmetic.hessianDemand_determinant`

Proof. Take $(P_\pi)_{r,c} = 1$ when $c = \pi(r)$ and 0 otherwise. Off its permutation diagonal, the Hessian is a signed matching: for $i \neq j$,

$$H_{\det}(P_\pi)_{(i,\pi(j)),(k,\pi(l))} = \begin{cases} -\operatorname{sgn}(\pi), & i = l \text{ and } j = k, \\ 0, & \text{otherwise.} \end{cases}$$

One algebraic verification takes the $\epsilon\delta$ coefficient of $\det(I + \epsilon E_{ij} + \delta E_{kl})$, then permutes the columns. The identity $\det(I + UV) = \det(I + VU)$ reduces this to a 2×2 determinant. Evaluation in two independent square-zero directions identifies the coefficient with the formal mixed derivative, without any characteristic restriction.

`Frontier.Arithmetic.det_two_entries`, `Frontier.Arithmetic.hessian_determinant_permutation`

For each pair counted by $c_\pi(S)$, select its S endpoint as a row and its S^c endpoint as a column of the mixed Hessian. These endpoints are distinct, and the displayed identity makes the selected submatrix diagonal with entries $-\operatorname{sgn}(\pi) \neq 0$. Thus

$$\operatorname{rank} H_{\det}(P_\pi)[S, S^c] \geq c_\pi(S).$$

Apply Lemma 8.14 to a balanced S. Since $4|S|\,|S^c| \geq n^4 - 1$, for $n \geq 1$ the resulting permutation satisfies

$$c_\pi(S) \geq \frac{n^2}{4} - 2n - \frac{1}{4n^2} \geq \frac{n^2}{4} - 3n.$$

The empty case is immediate. Every input ordering has a prefix containing exactly $\lfloor n^2/2 \rfloor$ entries, proving the demand statement.

`Frontier.Arithmetic.crossing_le_mixedRank`, `Frontier.Arithmetic.exists_prefix_card`

□

The permutation may depend on the input partition. The proof establishes the lower demand needed by the circuit theorem; it does not require an upper bound on determinant Hessian width or a point that works for all input orderings.

Corollary 8.16 (Determinant circuits)**.** *Assume* $\mathrm{LB}_3(A)$ *with* $A > 0$*. For every* $\varepsilon > 0$*, all sufficiently large* n*, every field* $\mathbb{F}$*, and every fan-in-two polynomial circuit computing* $\det_n$ *formally with* s *inner gates,*

$$s > \left(1 + \frac{1}{4A} - \varepsilon\right) n^2.$$

The threshold is uniform over fields and polynomial gate bases; constants are free and sharing is unrestricted. With the Gaussian layout coefficient,

$$s > (1.8906244197\ldots - \varepsilon)n^2.$$

`Frontier.Arithmetic.lowerBound_determinant_gaussian`

Proof. For $n \geq 2$, every determinant variable occurs formally. For example, for a given coordinate, choose a permutation making it an endpoint of one of the nonzero Hessian pairs above. Its first formal derivative therefore cannot vanish. Substituting $N = n^2$ and $h = n^2/4 - 3n$ in Theorem 8.13 gives, for every $\eta > 0$,

$$(A + \eta)n^2 + \frac{n^2}{4} - 3n \leq (A + 5\eta)s + A + 4\eta + C_\eta.$$

First choose η so that $(A + \eta + 1/4)/(A + 5\eta)$ exceeds $1 + 1/(4A) - \varepsilon$, then absorb the linear and constant terms by taking n large. Finally apply Theorem 7.1.

`Frontier.Arithmetic.mem_vars_determinant`, `Frontier.Arithmetic.determinant_gate_bound`

□

The polynomial-multiplication application proposed in [Fro] remains outside this development.

8.5 Mixed input–output recovery

Linearity is one way to make a boundary determine its input. The more general property is small ambiguity after a mixed observation. Let $F : U^N \to U^N$, with $q = |U| \geq 2$, and set

$$\Lambda_F(X, Y) = \max_{a \in U^X,\, b \in U^Y} |\{x : x|_X = a,\ F(x)|_Y = b\}|.$$

This definition involves the input and output coordinates, not the circuit's intermediate values. In Lean the maximum is indexed by an input realizing the observation; every nonempty fiber has such a representative.

Proposition 8.17 (Mixed-fiber demand)**.** *At any cut of the compiled network for* F*, let* X *be the inputs read on one side,* Y *the outputs on that side, and* L *the distinct signals crossing the cut. Then*

$$q^N \leq q^{|L|} \Lambda_F(X, Y) \Lambda_F(\bar{X}, \bar{Y}).$$

Equivalently,

$$|L| \geq N - \log_q \Lambda_F(X, Y) - \log_q \Lambda_F(\bar{X}, \bar{Y}).$$

The cardinal inequality is the checked statement.

`Frontier.Compiler.pow_le_cutSignals_mul_fibers`

Proof. Fix a class of inputs with the same frontier values and a representative x^0 in that class. For every x in the class, form the two splices

$$z = (x^0|_X, x|_{\bar{X}}), \qquad z' = (x|_X, x^0|_{\bar{X}}).$$

By Lemma 5.2, z agrees with x^0 on the mixed observation (X, Y), and z' agrees with x^0 on $(\bar{X}, \bar{Y})$. The pair (z, z') determines x. Hence the class has at most $\Lambda_F(X, Y)\Lambda_F(\bar{X}, \bar{Y})$ members. There are at most $q^{|L|}$ classes. This also proves the more general interface `Frontier.OutputSplicing.card_le`, with the number of used boundary messages in place of $q^{|L|}$. □

Call the graph code $\{(x, F(x)) : x \in U^N\}$ *MDS* when every N of its $2N$ coordinates determine the codeword. Equivalently, $\Lambda_F(X, Y) = 1$ whenever $|X| + |Y| = N$.

Theorem 8.18 (Nonlinear MDS maps)**.** *Suppose* $\mathrm{LB}_{r+1}(A)$ *holds with* $A > 0$ *and fixed* $r \geq 2$*. For every* $\varepsilon > 0$ *and all sufficiently large* N*, every circuit of fan-in at most* r *computing an MDS map* $F : U^N \to U^N$ *over any finite alphabet* U *of size at least two, with an arbitrary basis, satisfies*

$$(r - 1)s > (1 + 1/A - \varepsilon)N.$$

The size threshold is uniform in the alphabet and in F*. In particular the binary coefficient is* L*, and the ternary coefficient is* $7/4$*.* `Frontier.lowerBound_mds`, `Frontier.mds_gaussian`

Proof. Every layout has a cut with $|X| + |Y| = N$: the count starts at zero, ends at least at N because all outputs are present, and increases by at most one when a vertex is processed. Input sites and output sites are distinct. At this cut the complementary observation also contains N terminals, so both mixed fibers are singletons. Proposition 8.17 gives a frontier of at least N signals, hence at least N edges.

Moreover, every output depends on every input. Otherwise fixing the other $N - 1$ inputs and that output would fail to determine the remaining input, contrary to the MDS property. Thus every input reaches every output, so all inputs are read and the compiled graph is connected. Its cycle rank is at most $(r - 1)s + 1 - N$ and it has at most $2rs + 3N$ vertices. Comparing demand and supply now gives the bound exactly as in Theorem 8.6. □

Totally regular matrices satisfy this MDS property: after fixing X, the observations in Y determine the remaining inputs using the square nonsingular block $M_{Y,\bar{X}}$. This implication is checked by `Frontier.TotallyRegular.isMDS`. Applying arbitrary coordinatewise alphabet permutations before and after an MDS map preserves the property (`Frontier.IsMDS.permute`), giving nonlinear variants of the finite-field Cauchy maps. As with the linear examples, the alphabet may need to grow with N. These variants do not assert a new one-output Boolean construction.

9 Gates of unbounded fan-in

9.1 Ledgers

Mark some gates of a circuit as *special*; they may have any fan-in.

Definition 9.1 (Ledgers)**.** A *ledger* for the special gates in a commutative monoid M assigns to each wire w and each value u a *contribution* $\phi_w(u) \in M$, and to each element of M and each special gate a *readout*, such that for every assignment v of values to the wires and every special gate g,

$$\mathrm{readout}_g\Big(\prod_w \phi_w(v_w)\Big) = g(v_{w_1}, \ldots, v_{w_k}),$$

where $w_1, \ldots, w_k$ are the arguments of g. `Frontier.Ledger`

The identity is required for every assignment, not only for runs of the circuit: that is what lets a guess be checked. An operation of arity k *aggregates* in T if it has the form $x \mapsto \psi(\prod_i \phi_i(x_i))$.

Proposition 9.2 (Aggregate gates)**.** *If every special gate aggregates in T, the special gates have a ledger in T^k, where k is their number: the contribution of a wire to a special gate is the product of its contributions to the slots of that gate that read it. Unbounded AND and OR aggregate in $\mathbb{Z}/2$ under multiplication, and parity and counts modulo m in $\mathbb{Z}/m$ under addition.*

`Frontier.exists_ledger_of_aggregates`

So k gates computing AND, OR, parity, or counts modulo a fixed modulus have a ledger with $\log|M| = O(k)$ (`Frontier.aggregates_xor` and its siblings). Mixed fixed operations can use the product of their monoids, which still has constant size. For varying moduli $m_1, \dots, m_k$, the product ledger instead costs $\sum_j \log m_j$. A majority of d arguments aggregates in the counts $\{0, \dots, \lceil d/2 \rceil\}$ under saturating addition, so it costs $O(\log d)$.

9.2 Guess and check

Replace every special gate by a constant *placeholder*, numbered like the gate (`Frontier.erase`). The resulting program has the fan-in of the ordinary gates. To run it, *guess* the final product $m \in M$, and let each placeholder read its value from m (`Frontier.Ledger.run`). The guess is *consistent* if the product of the contributions of the run is m.

Lemma 9.3 (Guess and check)**.** *A consistent guess yields the true run of the circuit, and the product of the contributions of the true run is a consistent guess.* `Frontier.Ledger.run_eq_trace`

Proof. If the guess m is consistent, the ledger identity applied to the run says that each placeholder holds the value of its gate applied to its arguments in the run; so the run satisfies every gate equation of the original program, and is its trace. Conversely, the trace satisfies the equations of the erased program with the guess $\prod_w \phi_w(\mathrm{trace}_w)$, by the same identity (`Frontier.Ledger.run_total_trace`). □

A wire of the erased program is *live* if it is an input or a gate of positive arity; the other wires, the constants and the placeholders, take values that depend on the guess but not on the input. Compile the erased program with outputs the output of the circuit, accepting Acc, and every live wire, accepting every value, so that every live wire is a signal and carries its contribution at its vertex. For each guess m this gives a network; the graph and the sites do not depend on m, only the constraints at the placeholders do. The network has at most $2rs + 3(n + s + 1)$ vertices, where s counts the inner gates of the erased program (Proposition 5.5).

9.3 The ledger sweep

Lay out this graph. For an input x of S, let α_x put on every edge the value of its signal in the true run, and let the extra message at time t be the pair $(m_x, \lambda_t(x))$ of the consistent guess m_x and the product $\lambda_t(x)$ of the contributions of the signals whose vertices are among the first t (the *ledger so far*); after the last vertex the extra message is empty.

Proposition 9.4 (The ledger sweep)**.** *These data satisfy the hypothesis of Proposition 4.4, and so define a sweep of S.* `Frontier.Ledger.sweep`

Proof. Let $x, y \in S$ have the same extra message at time t and the same values on the frontier, and let z agree with x on the inputs read by the first t vertices and with y elsewhere. The runs of x

and y satisfy the network of the common guess m (Lemma 9.3), and they agree on the frontier, so the separator lemma gives an assignment satisfying it on z, which is the run of z with the guess m (Theorem 5.1). On the signals among the first t vertices it agrees with the run of x, on the other signals with the run of y, and on the remaining wires, which are constants, with both. So its ledger is $\lambda_t(x) = \lambda_t(y)$ times the rest of the ledger of y, which is m: the guess is consistent for z, the run is the true run of z by Lemma 9.3, and its output lies in Acc (`Frontier.Ledger.mem_of_splice`). □

The pair of extra messages before and after a step takes at most $|M|^3$ values, so by Proposition 4.4(a) there are at most $|M|^3 q^{w+d+1}$ transitions at a step with frontiers of size at most w (`Frontier.Ledger.ncard_transition_le_width`). The cost of the ledger is the term $3 \log_q |M|$.

9.4 Components

The erased program has lost the edges into the special gates, and its graph may fall apart into components. The sum of their cycle ranks has one additive unit for every component, so it may be much larger than $(r-1)s - n$. Here we mean the ordinary component-sum rank, not `Frontier.Multigraph.cycleRank` evaluated on the disconnected graph. The remedy is to look at one component at a time.

A set W of vertices is *closed* if no edge leaves it. Write $E(W)$ for the edges at W and $\beta_1(W) = \max\{0, |E(W)| - |W| + 1\}$ for the quantity `Frontier.Multigraph.cycleRankOn`. For a nonempty connected component this is its ordinary cycle rank. We also use this notation in the next counting lemma for closed sets, without claiming additivity over components.

Lemma 9.5 (Layouts of disconnected graphs)**.** *Under* $\mathrm{LB}_d(A)$*, every loopless graph of maximum degree* d *has a layout with the following property. At each step, the vertex processed lies in a closed set* W *(a connected component) with at least* $|W| - 1$ *edges, whose edges contain both frontiers of the step, and both frontiers have at most* $(A + \eta)\,\beta_1(W) + \eta|V| + C$ *edges.*

`Frontier.LayoutBound.exists_layout_componentwise`

Proof. The connected component of a vertex is closed, and the subgraph it induces is connected, loopless, of maximum degree d, and has cycle rank $\beta_1(W)$; a connected graph has at least $|W| - 1$ edges. Lay out the components one after another, each by the layout bound, which applies with the same constant C to every component, and $\eta|W| \le \eta|V|$. A prefix of the resulting layout is the union of some whole components, which are closed, and a prefix of the current component; since no edge leaves a closed set, its cut is the cut of that prefix inside the current component (`Frontier.Multigraph.cut_union_of_cut_eq_empty`). □

Lemma 9.6 (Counting inside a closed set)**.** *In the network of a program with fan-in at most* r*, let* s_W *count the inner gate vertices in a closed set* W*. If* W *has at least* $|W| - 1$ *edges, then* $\beta_1(W) + |\mathrm{read}(W)| \le (r-1)s_W + 1$*. In particular this is at most* $(r-1)s + 1$ *for the total number* s *of inner gates.*

`Frontier.Compiler.cycleRankOn_add_ncard_readIn_le_local`

Proof. The proof of Proposition 5.5, restricted to W: the edges at W are those of the junctions in W, which are the junctions of the slots of the gates in W and of the outputs in W (`Frontier.Compiler.card_touching_add_ncard_readIn_le_local`). □

Inside a closed set the run depends only on the guess and on the inputs read there, by the separator lemma with an empty cut (`Frontier.Ledger.run_eq_of_eqOn_readIn`). So Proposition 4.4(b) applies at a step inside W: there are at most $|M|^3 q^{|\mathrm{read}(W)|}$ transitions (`Frontier.Ledger.ncard_transition_le_closed`). A component that reads few inputs has few transitions, however wide its layout.

Theorem 9.7 (Ledgers)**.** *Assume* $\mathrm{LB}_{r+1}(A)$*, let* S_n *be as in Theorem 1.1, and let* $\beta(n) = o(n)$*. For every* $\varepsilon > 0$ *and all large* n*, every circuit deciding* S_n *whose gates have fan-in at most* r*, except for special gates of any fan-in with a ledger in a monoid* M *with* $\log|M| \le \beta(n)$*, has* $(r-1)s_{\mathrm{ord}} > (1 + 1/A - \varepsilon)\, n$*, where* s_{ord} *counts only the positive-arity gates that are not special.*

`Frontier.lowerBound_ledger`

Proof. Write $s = (r-1)s_{\mathrm{ord}}$, and lay out the network of the erased program by Lemma 9.5. By the two transition bounds, the bound $|M|^3$ at the last step, and Lemma 3.2, some step t inside a closed set W has

$$|S| \le K^2\,(|V|+1)\,|M|^3\,q^{\min(a,\ |\mathrm{read}(W)|)},$$

where $a \le w + r + 2$ for the frontier w of the step (`Frontier.Ledger.exists_ncard_le`). Write $D = n - \log_q|S|$ and $\Lambda = 2\log_q K + 3\log_q|M| + \log_q(|V|+1)$, so that $n \le D + \Lambda + \min(a, |\mathrm{read}(W)|)$. Then the component W reads almost all inputs, $n - |\mathrm{read}(W)| \le D + \Lambda$, and its layout is wide, $n \le D + \Lambda + w + r + 2$. By Lemma 9.6, $\beta_1(W) \le s + 1 - |\mathrm{read}(W)| \le s - n + 1 + D + \Lambda$. Let $B = (A+\eta)\beta_1(W) + \eta|V| + C \ge w$ be the bound of Lemma 9.5, and apply Lemma 6.1 to $w' = B - (A+\eta)\log_q(|V|+1)$, with

$$e(n) = D + 2\log_q K_n + 3\beta(n)/\log 2 + r + 3, \qquad \kappa = 7 + (A+2)/\log 2,$$

and $\eta \le 1$: the supply holds for w' by the bound on $\beta_1(W)$, the demand since $n \le D + \Lambda + B + r + 2$ and $(A+\eta+1)\log_q(|V|+1) \le \kappa\log(|V|+1)$, and $|V| \le \kappa(n+s+1)$. □

Theorem 1.8 is the special case of aggregate gates (Proposition 9.2), and with Theorem 7.1 the bound is $(L-\varepsilon)n$ for fan-in two (`Frontier.lowerBound_aggregate_gaussian`).

10 Counting satisfying assignments

This section concerns running times, which are not formalized. Its correctness rests on the exactness of the compiler (Theorem 5.1) and on the layout theorems of Section 7, which are. We also prove that the Gaussian layout construction can be derandomized without changing its leading coefficient. Throughout, O^* suppresses polynomial factors in $n + |\mathrm{encoding}(C)|$; the polynomial degree may depend on the fixed accuracy.

10.1 Counting along a layout

Proposition 10.1 (Counting along a layout)**.** *Given a loopless constraint network over* U *with a layout of width* w*, in which every vertex has degree at most* d *and reads at most* m *inputs, and the constraints can be tested in polynomial time, the number of pairs* (x, α) *of an input and an assignment satisfying the network can be computed in time* q^{w+d+m} *times a polynomial in the full network encoding and input size, using space* q^w *times such a polynomial, for fixed* q*.*

Proof. For $t \le |V|$ and an assignment σ to the frontier C_t, let $c_t(\sigma)$ count the pairs of an assignment to the inputs read by the first t vertices and an assignment to the edges touching them that satisfy the constraints of the first t vertices and agree with σ on C_t. Then $c_0(\emptyset) = 1$. The vertex v processed at step t sees only its own edges and inputs, and its edges lie in $C_t \cup C_{t+1}$; so c_{t+1} is obtained from c_t by extending each σ with the at most q^{d+m} values on the edges of v and the inputs read at v that are consistent with σ and satisfy the constraint of v, and restricting to C_{t+1}. The answer is $c_{|V|}(\emptyset)$ times q to the number of unread inputs. All counts are at most $q^{|\iota|+|E|}$, so the arithmetic is on integers of polynomially many bits. Only the current and next tables need be stored. □

For the network of a circuit with one output, the satisfying assignments on an accepted input form a single run (Theorem 5.1), so Proposition 10.1 counts the accepted inputs.

10.2 Computing layouts

Say that $\mathrm{LB}_3(A)$ is *constructive* if for every $\eta > 0$ there are a constant C and a polynomial-time algorithm that, given a connected, loopless, subcubic multigraph G, outputs a layout whose frontiers have at most $(A+\eta)\beta_1(G) + \eta|V(G)| + C$ edges; and *randomized constructive* if the algorithm is randomized and runs in expected polynomial time.

Lemma 10.2 (Constructive compression)**.** *If for every $\xi > 0$ there is a (randomized) polynomial-time algorithm that lays out every simple cubic graph on $h \geq h_0(\xi)$ vertices with frontiers of at most $(c+\xi)h + O(1)$ edges, then $\mathrm{LB}_3(2c)$ is (randomized) constructive.*

Proof. The compression of Proposition 7.2 performs fewer than $|V|$ merges, each found by testing pairs of adjacent blocks, so the final compression and its quotient H are computed in polynomial time. If H has fewer than $h_0(\xi)$ vertices, an optimal layout of H is found by exhaustive search in time depending only on ξ; otherwise the given algorithm lays out H. The expansion is computed in linear time, and the bound of Proposition 7.2 applies. □

Lemma 10.3 (Layouts from path decompositions)**.** *A cubic graph with a path decomposition of width p has a layout, computable in polynomial time from the decomposition, whose frontiers have at most $p+4$ edges.*

Proof. Let $X_1, \dots, X_\ell$ be the bags. Choose for every edge e an index $j(e)$ with both endpoints in $X_{j(e)}$, number the edges $e_1, \dots, e_m$ so that $j(e_k)$ is nondecreasing, and give e_k the score k. Apply Lemma 7.3 with $a = 1/2$, $\delta = 1$, and $M = m$: no edge scores below a or at least $a + M\delta$, and every window contains a single edge. A vertex v straddling the threshold $k + 1/2$ has edges e_i and $e_{i'}$ with $i \leq k < i'$; it lies in $X_{j(e_i)}$ and $X_{j(e_{i'})}$, hence in $X_{j(e_k)}$, since the bags containing v are consecutive and $j(e_i) \leq j(e_k) \leq j(e_{i'})$. So at most $p+1$ vertices straddle each threshold, and every frontier of the median layout has at most $p+1+3$ edges. □

Fomin and Høie [FH06] show that for every $\xi > 0$ every cubic graph on $h \geq h_0(\xi)$ vertices has a path decomposition of width at most $(1/6+\xi)h$, constructed in polynomial time. With Lemmas 10.2 and 10.3, $\mathrm{LB}_3(1/3)$ is constructive.

Lemma 10.4 (Sampling Gaussian layouts)**.** *For every $\xi > 0$ there is a randomized algorithm that lays out every simple cubic graph on $h \geq h_0(\xi)$ vertices with frontiers of at most $(p+\xi)h$ edges, in expected polynomial time, where $p = \frac{3}{2\pi}\arccos\frac{1+2\sqrt{2}}{4}$. Hence $\mathrm{LB}_3(2p)$ is randomized constructive.*

Proof. Fix the parameters $\lambda, R, T, M, \varepsilon$ of the proof of Theorem 7.1 for $\xi/4$. The real parameters may be chosen rational, with slack in their inequalities. A Gaussian sample ω then satisfies the bound $(p+\xi/4)h$, through Lemma 7.3, with probability at least $1 - O_\xi(1/h)$. Each edge vector y_e is supported on at most $D_0 = D_0(R)$ coordinates. The algorithm computes rational approximations $\widetilde{y}_e$ on exactly these supports, samples a rational approximation ω' of ω (below), and lists the vertices by the median of $s'(e) = \langle \widetilde{y}_e, \omega' \rangle$, breaking ties by vertex index. Fix a rational a with $p + \xi/2 < a < p + \xi$. The algorithm checks the actual cutwidth of this layout and repeats if it exceeds ah; no exact comparison with p is needed.

Perturbation. Suppose $|s'(e) - s(e)| \leq \zeta$ for every edge, where $s(e) = \langle y_e, \omega \rangle$, and let Z be the number of edges e with $|s(e) - \theta| \leq \zeta$ for some threshold θ of the grid. A vertex straddling a threshold under s' but not under s, an edge in a window or a tail under s' but not under s, both

involve one of these Z edges. Each individual count changes in absolute value by at most $2Z$, and every quantity bounded in Lemma 7.3 grows by at most $5Z$. Every $s(e)$ is a standard Gaussian, so $\mathbb{E}Z \le \frac{3h}{2}(M+1)\frac{2\zeta}{\sqrt{2\pi}} \le 1$ for $\zeta = 1/(3h(M+1))$, and $5Z \le \xi h/4$ except with probability $O_\xi(1/h)$.

Sampling. Let U_z be independent uniform variables on $[0,1]$ and $\omega_z = \Phi^{-1}(U_z)$, where Φ is the standard normal distribution function. The algorithm reads the first b bits of each U_z and computes ω'_z, the value of Φ^{-1} at the midpoint of the dyadic interval of length 2^{-b} containing U_z, to within 2^{-b}. With $\tau = \sqrt{6 \ln h}$, all $|\omega_z| \le \tau$ except with probability at most $h \cdot e^{-\tau^2/2} = h^{-2}$, and then, as the density of a standard Gaussian is at least h^{-4} on $[-\tau - 1, \tau + 1]$ for large h, $|\omega'_z - \omega_z| \le 2^{-b}h^4 + 2^{-b}$. Approximate each nonzero coordinate of y_e within 2^{-b}, keeping all other coordinates exactly zero. Then $|s'(e) - s(e)| \le 2D_0(\tau+2)(h^4+2)2^{-b}$, which is at most ζ for $b = O_\xi(\log h)$. The kernel coefficients use only rational arithmetic and square roots. The inverse normal at a dyadic midpoint can also be approximated in polynomial time in b: bisect on an interval of radius $O(\sqrt{b})$, evaluating the Gaussian integral to $O(b)$ bits. On this interval the density is at least $2^{-O(b)}$, so these error bounds suffice; integrating the Taylor series for $\exp(-t^2/2)$ with $O(b)$ terms and guard bits gives the required integral approximation. Thus each coordinate is a deterministic function of its own b-bit word, computable in polynomial time. Each trial succeeds with probability $1 - O_\xi(1/h) \ge 1/2$ for large h. Lemma 10.2 concludes. □

Lemma 10.5 (Fixed-accuracy local discretization)**.** *Fix the kernel and grid parameters in the proof of Theorem 7.1. For every $\gamma > 0$ there are a finite rational alphabet Ω, a uniform random variable W on Ω, and rational edge coefficients, all with precision independent of h, such that replacing the independent Gaussian coordinates by independent copies of W changes the expectation of each individual straddling, window, or tail indicator by at most γ. The supports of the coefficients are unchanged. Each discretized indicator depends on at most D coordinates and its support meets at most B others in any one count, for constants D, B depending only on the fixed parameters and not on the graph.*

Proof. Every test uses at most three edge scores and at most D Gaussian coordinates, for a uniform constant D supplied by the degree and radius bounds. Couple the Gaussian coordinates to finite rational samples by rounding their inverse-normal quantiles at dyadic midpoints, as in Lemma 10.4, but choose a constant number of bits. First choose T_0 so large that the probability that any of the at most D coordinates in one test exceeds T_0 in absolute value is below $\gamma/2$. On $[-T_0 - 1, T_0 + 1]$ the Gaussian density has a positive lower bound. Consequently, sufficiently fine constant quantile and coefficient precision makes the perturbation of every score in this test at most ζ, outside that exceptional event. The normal coefficients are unit vectors; their rational approximations retain all exact zeros, and the error in the dot product is bounded by D times the coordinate and coefficient errors, with a factor depending only on T_0.

Each original edge score is standard normal. A test changes only if one of its at most three scores lies within ζ of one of its defining thresholds. The probability of this event is at most a fixed constant times ζ, by the upper bound $1/\sqrt{2\pi}$ on the normal density. Choose ζ to make it below $\gamma/2$. This bounds the probability of a changed indicator, and hence the change in its mean, by γ. The rational midpoint approximations form a uniform finite alphabet; repeated alphabet values may instead be indexed by their sampling words. The bounded support overlap is exactly the radius argument in Theorem 7.1. All choices are effective and independent of h. This is a coupling for each local test; no simultaneous approximation of all coordinates is required. □

Lemma 10.6 (Conditional expectations for local counts)**.** *Let $C_1, \dots, C_J$ each be a sum of at most ch indicators of independent uniform words in a fixed finite alphabet of size Q. Suppose that each indicator depends on at most D words and overlaps at most B other indicators within its count. For*

fixed $J, c, Q, D, B, \varepsilon > 0$ and sufficiently large h, one can deterministically choose the h words so that $|C_j - \mathbb{E}C_j| < \varepsilon h$ for every j. The running time is $f(J, c, Q, D, B, \varepsilon)h^3(\log(h+2))^{O(1)}$, provided each local test can be evaluated in constant time depending on these parameters.

Proof. Put $\mu_j = \mathbb{E}C_j$, computed exactly by enumerating each local support, and consider

$$F = \sum_{j=1}^{J} \frac{(C_j - \mu_j)^2}{(\varepsilon h)^2}.$$

Disjoint supports are independent, so $\operatorname{Var} C_j \le ch(B+1)$ and $\mathbb{E}F \le Jc(B+1)/(\varepsilon^2 h) < 1$ for large h. Fix the words one at a time, choosing a value whose conditional expectation of F is no larger than the current one. Such a value exists by averaging. At the end $F < 1$, which gives all the asserted inequalities. This is the method of conditional expectations; see Vadhan [Vad12, Section 3.4].

To compute each conditional expectation, expand the squares. There are $O(Jh^2)$ terms, each a product of at most two local indicators or a constant multiple of one. Each term has at most $2D$ unfixed coordinates, so its conditional mean is an exact average over at most Q^{2D} possibilities. Testing Q choices at each of h steps gives the stated bound. The rational numbers have $O(\log(h+2))$ bits plus a parameter-dependent constant: the local probabilities have denominators dividing Q^{2D}, and the normalization adds a factor h^2. For an arbitrary real requested tolerance use a smaller positive rational tolerance. □

Lemma 10.7 (Deterministic Gaussian layouts)**.** *There are a computable function f and an absolute constant c such that, for every fixed rational $\xi > 0$, a simple cubic graph on $h \ge h_0(\xi)$ vertices has a layout of cutwidth at most $(p+\xi)h$ that can be found deterministically in time $f(\xi)h^3(\log(h+2))^c$. Consequently $\mathrm{LB}_3(2p)$ is constructive.*

Proof. Choose the rational kernel and grid parameters in the proof of Theorem 7.1 with enough slack that allowing every count to exceed its Gaussian mean by $3\varepsilon h$ still gives cutwidth at most $(p+\xi)h$, for some rational $\varepsilon > 0$. This is possible by first choosing the limiting mean bounds strictly below the target and then decreasing ε. Apply Lemma 10.5 with $\gamma = \varepsilon/2$. A count has at most $3h/2$ indicators, so its discretized mean differs from its Gaussian mean by less than εh. Apply Lemma 10.6 to the $2M+2$ discretized counts. Their realized values are at most their Gaussian means plus $2\varepsilon h$. The deterministic median lemma therefore gives the claimed width. Computing the fixed-precision coefficients, evaluating the local tests, and sorting the rational median scores fit in the stated running time. All dependence on the accuracy lies in f and h_0, not in the polynomial degree. These constants can be very large; the assertion is an asymptotic construction. Lemma 10.2 then gives the constructive layout bound. □

10.3 The algorithm

Theorem 10.8 (Counting satisfying assignments)**.** *Let $A > 0$ and let $\mathrm{LB}_3(A)$ be constructive, or randomized constructive. For every fixed $\eta > 0$, the number of satisfying assignments of a Boolean circuit of fan-in two with n inputs, s positive-arity gates, and m inputs having a path to the output can be computed in*

$$O^*\left(2^{\min\{m,\, A(s-m)\}+\eta s}\right) \;\le\; O^*\left(2^{(A/(1+A)+\eta)s}\right)$$

time, or expected time, respectively. The deterministic algorithm uses at most the same space. The hidden multiplicative constant may depend on A and η.

Proof. Let R be the set of inputs with a path to the output; the count is $2^{n-|R|}$ times the count over the inputs in R. One algorithm enumerates the assignments to R, in time $O^*(2^{|R|})$. The other compiles the circuit into its network, which has at most $4s+3$ vertices and cycle rank at most $s+1-|R|$ (Proposition 5.5), computes a layout of width $w \le (A+\eta)(s+1-|R|)+\eta(4s+3)+C$, and applies Proposition 10.1, in time and space $O^*(2^w)$, in addition to the polynomial cost of finding the layout (expected polynomial cost in the randomized case). Run the algorithm with the smaller bound. Since $\min(|R|, A(s-|R|)) \le \frac{A}{1+A}\,s$ (if $|R| > \frac{A}{1+A}s$ then $A(s-|R|) < \frac{A}{1+A}s$), the time is first $O^*(2^{\min\{|R|,A(s-|R|)\}+O(\eta)s})$ and then $O^*(2^{\frac{A}{1+A}s+O(\eta)s})$; rescale η. The factors hidden by O^* cover reading the circuit and writing the answer, including the factor $2^{n-|R|}$ for unused inputs. □

By Lemma 10.7, $A = 2p$ is constructive. This proves Theorem 1.9 with the deterministic exponent

$$\frac{2p}{1+2p} = \frac{1}{L} = 0.2191781936\ldots.$$

Using Lemma 10.4 instead gives a randomized algorithm with the same exponent and an expected-polynomial-time layout search. The pathwidth construction gives the simpler deterministic exponent $\frac{1/3}{1+1/3} = \frac{1}{4}$.

Corollary 10.9 (The input-relative exponent)**.** *If all n inputs have a path to the output and $s = cn$ for a fixed c, the deterministic running time is $O^*(2^{(\min\{1,A(c-1)\}+\varepsilon)n})$ for every fixed $\varepsilon > 0$. With the Gaussian coefficient, $c = 3$ gives exponent $0.5614038745\ldots+\varepsilon$, and $c = 4$ gives $0.8421058118\ldots+\varepsilon$. The cutoff for a constant improvement over 2^n remains $c < L$. Reachability here is syntactic; no test for semantic dependence is used.*

Corollary 10.10 (Counting at every fixed fan-in; not formalized)**.** *For every fixed $r \ge 2$ and $\eta > 0$, satisfying assignments of a Boolean circuit of fan-in at most r can be counted deterministically in time and space*

$$O^*\left(2^{\min\{m,\,a_{r+1}((r-1)s-m)\}+\eta s}\right) \le O^*\left(2^{\left(\frac{(r-1)a_{r+1}}{1+a_{r+1}}+\eta\right)s}\right),$$

where s counts positive-arity gates and m inputs reach the output. In particular, the ternary exponent is $4/7+\eta$. For binary circuits, Theorem 10.8 is sharper.

Proof. The degree-$(r+1)$ compression is constructive by scanning adjacent block pairs and making eligible merges. Its quotient layout is constructible by the same local discretization and conditional-expectation argument as Lemma 10.7. The counts are now the vertex-score windows and tails and the edge-crossing tests in Theorem 7.11. Each test still depends on only a bounded neighborhood for fixed accuracy. Each vertex score is a unit Gaussian form, so the same tail truncation and one-dimensional anti-concentration estimates bound changes under finite-precision perturbations. The discretized tests therefore meet the hypotheses of Lemma 10.6; their first and second moments can be evaluated by enumerating their bounded supports. Conditional expectations select all coordinates in $f(\xi)h^3 \operatorname{polylog} h$ time, and Theorem 7.11's grid-to-order argument verifies the resulting layout.

The compiler has cycle rank at most $(r-1)s+1-m$ and $O_r(s+1)$ vertices. Repeat the proof of Theorem 10.8 with $A = a_{r+1}$. The two exponents meet at $m = A((r-1)s-m)$, namely $m = (r-1)As/(1+A)$. Constants may depend on the fixed fan-in and accuracy; polynomial factors include the full circuit encoding and number of inputs. □

10.4 Counting with a ledger

Theorem 10.11 (Counting with aggregate gates)**.** *Assume* $\mathrm{LB}_3(A)$ *is constructive and put* $\alpha = A/(1+A)$*. Consider a Boolean circuit with* s_{ord} *ordinary positive-arity gates of fan-in at most two and special gates admitting a finite ledger* M*. Suppose that* M *can be enumerated and its multiplication, contributions, readouts, and gate operations can be evaluated in time polynomial in the full instance encoding and* $\log|M|$*. Then for every fixed* $\eta > 0$ *its satisfying assignments can be counted deterministically in time and space*

$$O^*\left(|M|^{O(1)} 2^{(\alpha+\eta)s_{\mathrm{ord}}}\right).$$

For k *unbounded AND, OR, parity, or fixed-modulus count gates this becomes* $O^*(2^{(\alpha+\eta)s_{\mathrm{ord}}+O(k)})$*.*

Proof. Fix a ledger guess $a \in M$, erase the special gates, and compile the designated output and every live wire, exactly as in Section 9. The graph is independent of a. Contributions of nonlive wires (constants and special-gate placeholders) are known from a; let their product be $b(a)$. Associate the contribution of each live wire with its unique producer vertex, so each is multiplied exactly once.

For a component W_i, let m_i be the number of input vertices, s_i the number of ordinary inner gate vertices, and o_i the number of output vertices in it. The local form of Lemma 9.6 gives

$$\beta_1(W_i) + m_i \le s_i + 1.$$

The compiler count restricted to this component gives $|W_i| \le 4s_i + 3o_i$: every signal has a consumer in the same component, and there are at most $2s_i + o_i$ consumers. There is one auxiliary output per live wire and one designated output, hence $o_i \le m_i + s_i + 1$ and $|W_i| \le 10s_i + 6$. In particular, components with no ordinary gate have bounded size.

Compute a table $T_i(z)$ counting the satisfying assignments in W_i whose live-wire contributions multiply to $z \in M$. One method enumerates its 2^{m_i} input assignments; for the fixed guess the component run depends only on these inputs, by the closed-set locality lemma. Another applies Proposition 10.1, adding a ledger accumulator to the frontier state, at cost $|M|\,O^*(2^{w_i})$. All auxiliary outputs accept every value; the designated output, in its own component, enforces acceptance. Exactness of the compiler makes these tables count inputs, not multiple traces. The layout bound and the component size estimate give, for any fixed $\delta > 0$,

$$w_i \le A(s_i - m_i) + O(\delta)s_i + O_{A,\delta}(1).$$

Choose the cheaper method in each component. Since $\min\{m_i, A(s_i - m_i)\} \le \alpha s_i$, their total work is bounded by $|M|\,O^*(2^{(\alpha+\eta)s_{\mathrm{ord}}})$, after rescaling δ; $\sum_i s_i = s_{\mathrm{ord}}$. The times add over components.

Combine the tables by monoid convolution: $(T * T')(z) = \sum_{uv=z} T(u)T'(v)$, computed by enumerating all pairs $(u, v) \in M^2$. Sum the resulting entries with $b(a)z = a$, and finally sum over guesses a. No cancellation or inverse is required. A guess is consistent if and only if $a = \ell(v(x))$: soundness identifies its run with the true trace and completeness supplies that guess (`Frontier.Ledger.total_run_eq_iff`). Thus every accepted input is counted exactly once. The convolutions and the outer enumeration add only a polynomial factor in $|M|$. All integers have polynomially many bits in the full encoding and input size. Finally, fixed-monoid aggregate gates have $\log|M| = O(k)$ by Proposition 9.2. □

11 Decomposable representations

The charging argument of Lemma 3.2 is not specific to sweeps. This section applies it to the DAGs of knowledge compilation. A smooth decomposable representation is an acyclic DAG whose nodes

denote sets of partial assignments. A union combines sets on the same coordinate scope; a binary product combines sets on disjoint scopes. The root's scope is $[n]$. Leaves represent individual coordinate values, or constants on the empty scope, so each leaf has at most one assignment. In the Boolean setting this is smooth DNNF. Union branches may overlap, and different branches may decompose the coordinates differently.

Theorem 11.1 (Decomposable threshold counting)**.** *Let S be K-rectangle-free, where $K \geq 2$ and $|S| \geq K$. If a smooth decomposable representation of S has $e_\cup$ outgoing union edges and $v_\times$ binary product nodes, then*

$$|S| \leq (K-1)^2 e_\cup + (K-1)^3 v_\times.$$

In particular, if D counts all vertices and edges, then $|S| \leq (K-1)^3 D$.

`Frontier.Decomposable.ncard_le`, `Frontier.Decomposable.ncard_le_size`

Proof. For a node v with scope I_v, let A_v be its denotation and let C_v contain the assignments outside I_v supplied by accepting certificates through v. The standard certificate-context construction for DNNFs [BCMS16] gives

$$A_v \times C_v \subseteq S.$$

For completeness, substitute any assignment in A_v at its occurrence in such a certificate and ascend toward the root. At a union, membership in the selected child implies membership in the parent. At a product, the sibling uses disjoint coordinates and its assignment is unchanged. The substituted assignment is therefore accepted at the root. This proves the rectangle inclusion even when nodes are shared or union branches overlap (`Frontier.Decomposable.Occurs.substitute`). Rectangle-freeness gives $|C_v| \leq K-1$ whenever $|A_v| \geq K$.

For each $x \in S$, choose an accepting certificate and descend from the root, maintaining $|A_v| \geq K$. At a union, follow a selected accepting child u. If $|A_u| < K$, stop and charge x to the edge (v, u); smoothness gives the same scope, so there are at most $(K-1)^2$ possibilities in $A_u \times C_v$. At a product, descend into a child whose denotation has at least K assignments, if there is one. If both children are smaller, stop and charge x to the product node. The two child denotations and the parent context give at most $(K-1)^3$ possibilities. A decreasing DAG rank guarantees termination, since leaves have at most one assignment. Summing the charges proves the result. Neither deterministic unions nor uniqueness of certificates was used. □

Thus $|S_n| = 2^{n-o(n)}$ and $\log K_n = o(n)$ imply smooth DNNF size $2^{n-o(n)}$. For the polynomial threshold and constant density supplied by Corollary 1.5, the bound is $2^{n-O(\log n)}$. An ordinary Boolean DNNF can be smoothed with $O(n)$ overhead: on each union edge, conjoin the child with one tautology $x_i \vee \neg x_i$ for each coordinate missing from its parent's scope; implement this conjunction by binary products. These added scopes are disjoint from the child's scope. Missing root variables are treated in the same way. Binarizing product gates costs linear overhead in the number of edges, so the same asymptotic bounds hold for ordinary DNNFs. This syntactic conversion and the cited explicit extractor instantiation are outside the formalization; the finite threshold theorem itself is checked.

The theorem does not supply a weighted average-case bound for DNNFs: certificates may overlap, so the disjoint peeling of Proposition 3.5 does not apply. Nor does it improve the circuit coefficient by itself. A compiler producing a tree of regions instead of a linear sweep would charge, at a split $W = W_0 \dot\cup W_1$, the joint boundary

$$|\delta(W_0) \cup \delta(W_1)| = \frac{|\delta(W_0)| + |\delta(W_1)| + |\delta(W)|}{2},$$

which can exceed the largest individual boundary; no universal bound on this quantity in terms of the cycle rank is known.

12 Design notes on the formalization

The formal statements are meant to be read. We record the choices behind them.

Faithful, minimal statements. The main theorems quantify over every signature, interpretation, acceptance set, and circuit at once, and their hypotheses are exactly those of the paper: rectangle-freeness, $\log K_n = o(n)$, density, and $\mathrm{LB}_{r+1}(A)$. The density hypothesis is stated as $n \log q - \log|S_n| = o(n)$ and the threshold hypothesis as $\log K_n = o(n)$, with Mathlib's little-o. The linear theorems quantify over every field inside the eventuality, so the threshold for N is uniform in the field.

One comparison. The asymptotic arithmetic of the exact lower bounds is Lemma 6.1, proved once, for every sufficiently small slack. Each application supplies a graph, a demand, and a bound on the cycle rank, at a fixed length, and the lemma does the rest. The average-case variant instead bounds the logarithm of the peeling residue; `eventually_frontier_log_error` records the slack needed for every $\gamma < \min\{1, A\varepsilon\}$.

Sets and cardinalities. Sets of inputs, rectangles, cuts, and edge neighbourhoods are `Set`s, and sizes are `Set.ncard` and `Nat.card`. No statement mentions decidability, and finiteness enters only as `Finite` instances on coordinates, alphabet, vertices, and edges. Proofs pass to `Finset` when they need its counting lemmas.

The sweep lemma is abstract. Lemma 3.2 is stated for an abstract sweep (`Frontier.Sweep`) and proved without reference to graphs. Messages are arbitrary functions into an arbitrary type, and the revealed sets need not be nested.

One sweep along a layout. The initial network sweeps are built by a single constructor (`Frontier.Network.frontierSweep`), whose messages pair an extra message with the values on the frontier, encoded as functions $E \to \mathrm{Option}\ U$ that are `none` off the frontier. The sweep of a network has no extra message, and the ledger sweep carries the guess and the ledger so far. The transition bounds of Proposition 4.4 are proved once, for the constructor.

Inputs and sites. Each input is read by at most one vertex, so the site map is $\iota \to \mathrm{Option}\ V$. Inputs that no vertex reads are revealed in one extra step at the end of the sweep. This keeps the network definition honest, since an unread input is simply not read, and places the cost of unread inputs exactly where the support lemma pays for it.

Locality as an axiom of the structure. A constraint is a predicate on a whole input and a whole edge assignment, together with a field asserting that it depends only on the incident edges and the inputs read at the vertex. Encoding constraints as predicates on dependent function types $U^{E(v)}$ would make locality definitional, but at the cost of transport along the incidence relation in every proof.

Outputs and junction chains. The compiler takes a family of outputs indexed by any finite type, each with its own accepting set. Deciding a set is the case of one output; computing a function accepts every value; the ledger compiles the output of the circuit together with every live wire. One junction per consumer, with the junctions of a signal numbered by an equivalence between

consumers and pairs (signal, position), removes every special case at the cost of one extra vertex and one extra edge per signal, which changes neither the cycle rank nor the theorems.

Inner gates. The count of the compiler charges only gates of positive arity, so constants are free in every theorem. This is what makes arithmetic circuits a special case: no separate translation that absorbs constants into other gates is needed.

Witnesses and uniqueness. The sweep of a network chooses a satisfying assignment for each accepted input with Hilbert's choice operator, and the exact lower bound for decision problems never uses its uniqueness, so it applies verbatim to nondeterministic constraint systems. The lower bounds for linear maps and ledgers do use uniqueness, in the form of Lemma 5.2 and of the run of a closed set, because there the spliced assignment must be recognized as the run of the spliced input. The average-case proof uses it for coherence of the full frontier trace. It then deletes message classes through `Sweep.erase`; the remaining sweep stays coherent, and the weighted induction never assumes the false converse from arbitrary sweeps to programs.

Linearization through dual numbers. Mathlib, at the pinned version, has partial derivatives of multivariate polynomials but no chain rule for substitution. Evaluating the circuit over the dual numbers makes the chain rule a consequence of the fact that evaluation commutes with algebra homomorphisms, and reduces the analytic content to one Taylor identity, proved by induction on the polynomial.

Placeholders. The erased program replaces special gates by constant placeholders indexed by the number of the gate, not by the guess, so the erased program, and hence the graph, its layout, and its sweep, do not depend on the guess; only the interpretation of the placeholders does.

Components through lists. Lemma 9.5 is proved by induction over closed sets, splitting off one component at a time, with layouts represented as lists without repetition; concatenation of lists is then trivial, and the list is converted to an equivalence with $\mathrm{Fin}\,|V|$ at the end.

The layout hypothesis. $\mathrm{LB}_d(A)$ quantifies over graphs whose vertex and edge types live in `Type`, with `Finite` instances, and asks for a layout as an equivalence $V \simeq \mathrm{Fin}\,|V|$. The error term $\eta|V| + C$ is the weakest form that suffices, so the hypothesis is as weak as possible; in the proof of Theorem 7.1 it absorbs the logarithmic boundaries of the compression blocks.

Median layouts instead of path decompositions. Edge scores are usually turned into vertex layouts through path decompositions: list the edges by score, keep each vertex in the bags between its first and last edge, and order the vertices by the median of their bags. Lemma 7.3 goes directly from scores to frontiers, which removes path decompositions from the development.

Upstreaming. Facts about CSLib programs (`Frontier/ForCslib/Program.lean`): a gate reads only earlier gates, a fan-in bound bounds every line's arity, a gate's value is its operation applied to its arguments' values, the value of a wire depends only on the inputs upstream of it, and the number of inner gates. The constraint network and the compiler would fit naturally in CSLib next to its circuit model.

13 Open problems

Better layouts. Any improvement of $\mathrm{LB}_3(A)$ below $A \approx 0.2807$ improves all the binary-circuit coefficients and the running time of Theorem 10.8, subject to the obstruction in Section 7.4. What is the least A for which $\mathrm{LB}_3(A)$ holds?

Beyond edge counts. Proposition 5.6 supplies a concrete alternative capacity: distinct signals on adjacent frontiers. A bound with $A_{\mathrm{sig}} < A$ would improve both headline constants. The random-cubic bisection obstruction concerns raw edge counts and does not by itself bound signal capacity. What universal signal-layout theorem holds for compiled graphs? For the present Gaussian process, Proposition 7.7 shows that sharpening the local straddling estimate alone cannot improve its limiting constant. A different score process, a stronger global layout argument, or signal compression would be needed.

Larger fan-in. Theorem 7.12 proves $\mathrm{LB}_d(a_d)$ in every fixed degree $d \geq 3$, retaining parallel edges in both compression and concentration. Can one improve these coefficients by exploiting the joint degree and correlation profile, rather than bounding all adjacent correlations by ρ_d? Another route is to replace higher-degree vertices by subcubic trees and control the congestion when projecting a cubic layout back to the original graph.

Other capacities. The repository contains checked inequalities for several finer capacities than edge counts: transition codes and exact linear syndrome states, pruned transition sets with a tail budget, distribution-sensitive transition masses, trees of regions with joint merge boundaries, and an independent-set median update of Gaussian vertex keys that never increases a threshold cut. None of them comes with a supply theorem, and none changes a coefficient of this paper; the file `RESEARCH.md` records them and what each would need. Dimension for continuous operations over the reals (Remark 8.9) is another possible capacity, provided it supports the required splicing or injectivity theorem.

Explicit hard sets in the formalization. The formalization takes the hard sets as hypotheses. A formal construction of a sumset disperser, or of any dense rectangle-free family in P with $\log K_n = o(n)$, would make Corollary 1.5 unconditional in Lean. A polynomial-error sumset extractor would also formalize the explicit instantiation in Corollary 6.10, whose conditional average-case implication is already checked.

A Index of formal statements

All declarations are in the namespace `Frontier`, in the files under `Frontier/` named in each group of the table, except for the upstreaming candidates in `Frontier/ForMathlib` and `Frontier/ForCslib`. For Corollary 6.10, the indexed declaration proves the implication from polynomially small extractor error; the explicit construction is cited externally.

Paper	Lean declaration
Introduction	
Thm. 1.1	`lowerBound`
Thm. 1.2	`layoutBound_gaussian`, `layoutBound_one_third`
Cor. 1.3	`Gaussian.cubicLayoutBound_gaussian`

Paper	Lean declaration
Cor. 1.4	`lowerBound_gaussian`, `lowerBound_four`
Cor. 1.5	`lowerBound_sumsetDisperser`
Thm. 1.6	`lowerBound_linear_finite`, `lowerBound_linear_polynomial`
Cor. 1.7	`arith_cauchy_gaussian`, `cauchyZMod_gaussian`
Thm. 1.8	`lowerBound_aggregate`, `lowerBound_aggregate_gaussian`
Rectangles (`Rectangle.lean`, `Extractor.lean`)	
Def. 2.1	`rectangle`
Def. 2.2	`RectangleFree`
Lem. 2.4	`RectangleFree.ncard_le_of_dependsOn`
Lem. 2.5	`RectangleFree.pow_ncard_compl_lt`
Def. 2.6	`SumsetDisperser`
Lem. 2.7	`SumsetDisperser.rectangleFree`
Thm. 2.8	`lowerBound_sumsetDisperser`
Sweeps (`Sweep.lean`)	
Def. 3.1	`Sweep`, `Sweep.transition`
Lem. 3.2	`Sweep.ncard_le`
Graphs and networks (`Multigraph.lean`, `Network.lean`)	
Hyp. 4.1	`LayoutBound`
Def. 4.2	`Network`, `Network.accepted`
Lem. 4.3	`Network.Satisfies.splice`
Prop. 4.4	`Network.frontierSweep`, `Network.ncard_transition_le`
	`Network.ncard_transition_le_of_touching`, `Network.sweep`
Thm. 4.5	`Network.ncard_accepted_le`
Compiler and signals (`Compiler.lean`, `Signals.lean`)	
Thm. 5.1	`Compiler.accepted_network`, `Compiler.eq_trace`
Lem. 5.2	`Compiler.trace_out_of_cut`
Prop. 5.4	`Compiler.maxDegreeLE`, `Compiler.loopless`
	`Compiler.connected_of_upstream`, `Compiler.connected`
Prop. 5.5	`Compiler.card_edge_add_ncard_read_le`
	`Compiler.cycleRank_add_ncard_read_le`, `Compiler.card_vertex_le`
Prop. 5.6	`Compiler.ncard_accepted_le_signals`
Demand and supply (`Demand.lean`, `Reduction.lean`, `LowerBound.lean`)	
Lem. 6.1	`eventually_lt_of_demand`
Thm. 6.2	`exists_graph_of_decides`
Existential inputs (`Projection.lean`, `Nondeterministic.lean`)	
Lem. 6.4	`Network.accepted_hide`
Thm. 6.5	`lowerBound_nondeterministic`, `lowerBound_nondeterministic_gaussian`
Cor. 6.6	`lowerBound_nondeterministic_witnesses`
	`lowerBound_nondeterministic_witnesses_gaussian`
Average-case bounds (`AverageCase/*.lean`)	
Prop. 3.5	`Sweep.Coherent`, `Sweep.abs_sumOn_le_of_budget`
Thm. 6.8	`Network.sweep_coherent`, `agreement_le_layout`
	`agreement_le_unused`, `averageCase`, `averageCase_abs`
	`averageCase_gaussian`
Cor. 6.9	`FlatSumsetBias.rectangleBias`, `averageCase_sumset`
Cor. 6.10	`averageCase_sumset_polynomial`
Randomization (`Randomized.lean`, `Randomized/Basic.lean`)	
Thm. 6.11	`Randomized.average_le`, `Randomized.exists_seed`
	`Randomized.exists_input_success_le`, `Randomized.success_le_of_class`

Paper	Lean declaration
Cor. 6.12	`randomized_circuit_bound`, `size_lt_of_randomized_correct`
	`averageCase_randomized`, `averageCase_randomized_gaussian`
	`lowerBound_randomized_gaussian`
	`averageCase_randomized_sumset_polynomial`
Layouts (`Layouts/*.lean`, `Layouts.lean`, `Main.lean`)	
Thm. 7.1	`layoutBound_gaussian`, `Gaussian.cubicLayoutBound_gaussian`
Prop. 7.2	`CubicLayoutBound.exists_layout_log`, `LayoutBound.of_cubic`
Lem. 7.3	`exists_key_of_edgeScore`
Lem. 7.4	`Gaussian.sum_unitKernel_mul_ge`
Lem. 7.5	`Gaussian.gaussPi_between_le_arccos`
Prop. 7.6	`Gaussian.measureReal_straddleEvent_le`
Cor. 7.8	`lowerBound_gaussian`, `lowerBound_four`
Thm. 7.9	`Multigraph.exists_layout_cycleRank`, `layoutBound_one`
Cor. 7.10	`lowerBound_all_fanIn`
Thm. 7.11	`Gaussian.multigraph_vertexLayout_bound`
	`Gaussian.sum_unitKernel_mul_ge_degree`
	`Gaussian.exists_multi_vertex_sample`
	`Multigraph.exists_key_of_vertexScore`
Thm. 7.12	`layoutBound_degree`, `Multigraph.core_density`
	`Gaussian.degreeCoefficient_lt_three_quarters`
	`Multigraph.Compression.Terminal.multi_edge_bound`
Cor. 7.13	`lowerBound_degree`, `mds_degree`
	`linear_finite_degree`, `linear_polynomial_degree`
Thm. 7.14	`layoutBound_two_fifths`
Cor. 7.15	`lowerBound_fanInThree`
	`linear_finite_fanInThree`, `linear_polynomial_fanInThree`
Decomposable representations (`Decomposable.lean`)	
Thm. 11.1	`Decomposable.ncard_le`, `Decomposable.ncard_le_size`
	`Decomposable.Occurs.substitute`
Linear maps (`Linear/*.lean`)	
Def. 8.1	`TotallyRegular`, `TotallyRegular.eq_zero`
Lem. 8.2	`totallyRegular_cauchy`, `totallyRegular_cauchyNat`
Lem. 8.3	`exists_injective_cut`
Def. 8.4, Lem. 8.5	`LinearCapacity`, `linearCapacity_of_finite`, `linearCapacity_affine`
Thm. 8.6	`lowerBound_linear`
Lem. 8.7	`exists_affine_circuit`
Cor. 8.8	`linear_finite_gaussian`, `linear_polynomial_gaussian`
Scalar polynomials (`Arithmetic/*.lean`)	
Lem. 8.10	`Arithmetic.bilinear_rank_le_boundary`
Thm. 8.11	`Arithmetic.rank_mixedHessian_le_cut`
	`Arithmetic.jet_mixed`, `Arithmetic.cut_splices`
Def. 8.12	`Arithmetic.HessianDemand`
Thm. 8.13	`Arithmetic.hessian_gate_bound`, `Arithmetic.lowerBound_hessian`
	`Arithmetic.lowerBound_hessian_gaussian`
Lem. 8.14	`Arithmetic.DeterminantMatching.exists_crossing`
Thm. 8.15	`Arithmetic.determinant_balanced_rank`
	`Arithmetic.hessianDemand_determinant`
	`Arithmetic.hessian_determinant_permutation`
	`Arithmetic.crossing_le_mixedRank`
Cor. 8.16	`Arithmetic.determinant_gate_bound`
	`Arithmetic.lowerBound_determinant`

Paper	Lean declaration
	`Arithmetic.lowerBound_determinant_gaussian`
Mixed recovery (`Boundary/Fiber.lean`, `Boundary/MDS.lean`)	
Prop. 8.17	`OutputSplicing.card_le`, `Compiler.pow_le_cutSignals_mul_fibers`
Thm. 8.18	`lowerBound_mds`, `mds_gaussian`
	`TotallyRegular.isMDS`, `IsMDS.permute`
Ledgers (`Ledger/*.lean`, `Components.lean`)	
Def. 9.1	`Ledger`
Prop. 9.2	`exists_ledger_of_aggregates`, `aggregates_and`
Lem. 9.3	`Ledger.run_eq_trace`, `Ledger.run_total_trace`
Prop. 9.4	`Ledger.sweep`, `Ledger.mem_of_splice`
Lem. 9.5	`LayoutBound.exists_layout_componentwise`
Lem. 9.6	`Compiler.cycleRankOn_add_ncard_readIn_le_local`
Thm. 9.7	`lowerBound_ledger`, `Ledger.exists_ncard_le`

Acknowledgments

Sam McGuire contributed the weighted average-case argument based on disjoint rectangle peeling, including asymmetric thresholds and a shared exceptional-measure bound.

References

[ABCR99] Alexander E. Andreev, Juri L. Baskakov, Andrea E. F. Clementi, and José D. P. Rolim. Small pseudo-random sets yield hard functions: New tight explicit lower bounds for branching programs. In *Automata, Languages and Programming (ICALP 1999)*, volume 1644 of *Lecture Notes in Computer Science*, pages 179–189. Springer, 1999.

[Abe78] Harold Abelson. Lower bounds on information transfer in distributed computations. Technical Report MIT-LCS-TM-102, Massachusetts Institute of Technology, Laboratory for Computer Science, April 1978.

[ACMS19] Antoine Amarilli, Florent Capelli, Mikaël Monet, and Pierre Senellart. Connecting knowledge compilation classes and width parameters. arXiv:1811.02944v2, 2019. Theorem numbering follows this version.

[BB06] Avrim Blum and Manuel Blum. 15-451 algorithms, fall 2006: Lectures 1–10. Carnegie Mellon University lecture notes, 2006. Section 5.4: Lower bounds for randomized algorithms.

[BCMS16] Simone Bova, Florent Capelli, Stefan Mengel, and Friedrich Slivovsky. Knowledge compilation meets communication complexity. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI 2016)*, pages 1008–1014, 2016.

[Ber91] C. Leonard Berman. Circuit width, register allocation, and ordered binary decision diagrams. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 10(8):1059–1066, 1991.

[Blu84] Norbert Blum. A Boolean function requiring $3n$ network size. *Theoretical Computer Science*, 28(3):337–345, 1984.

[BRS93] Allan Borodin, Alexander Razborov, and Roman Smolensky. On lower bounds for read-k-times branching programs. *Computational Complexity*, 3:1–18, 1993.

[CGHV15] Endre Csóka, Balázs Gerencsér, Viktor Harangi, and Bálint Virág. Invariant Gaussian processes and independent sets on regular graphs of large girth. *Random Structures & Algorithms*, 47(2):284–303, 2015.

[CL16] Eshan Chattopadhyay and Xin Li. Extractors for sumset sources. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2016)*, pages 299–311. ACM, 2016.

[CL22] Eshan Chattopadhyay and Jyun-Jie Liao. Extractors for sum of two sources. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*, pages 1584–1597. ACM, 2022.

[CL23] Eshan Chattopadhyay and Jyun-Jie Liao. Hardness against linear branching programs and more. In *38th Computational Complexity Conference (CCC 2023)*, volume 264 of *LIPIcs*, pages 9:1–9:27, 2023.

[CZ19] Eshan Chattopadhyay and David Zuckerman. Explicit two-source extractors and resilient functions. *Annals of Mathematics*, 189(3):653–705, 2019.

[DK11] Evgeny Demenkov and Alexander S. Kulikov. An elementary proof of a $3n - o(n)$ lower bound on the circuit complexity of affine dispersers. In *Mathematical Foundations of Computer Science (MFCS 2011)*, volume 6907 of *Lecture Notes in Computer Science*, pages 256–265. Springer, 2011.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[FGHK16] Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch, and Alexander S. Kulikov. A better-than-3n lower bound for the circuit complexity of an explicit function. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2016)*, pages 89–98. IEEE, 2016.

[FH06] Fedor V. Fomin and Kjartan Høie. Pathwidth of cubic graphs and exact algorithms. *Information Processing Letters*, 97(5):191–196, 2006.

[Fro] The frontier method: Three lower-bound chains. Unpublished working note. Rectangle-free sets, Jacobian sweep rank, and mixed Hessians; undated.

[GKST16] Alexander Golovnev, Alexander S. Kulikov, Alexander V. Smal, and Suguru Tamaki. Circuit size lower bounds and #SAT upper bounds through a general framework. In *41st International Symposium on Mathematical Foundations of Computer Science (MFCS 2016)*, volume 58 of *LIPIcs*, pages 45:1–45:16, 2016.

[GPT22] Svyatoslav Gryaznov, Pavel Pudlák, and Navid Talebanfard. Linear branching programs and directional affine extractors. In *37th Computational Complexity Conference (CCC 2022)*, volume 234 of *LIPIcs*, pages 4:1–4:16, 2022.

[IM02] Kazuo Iwama and Hiroki Morizumi. An explicit lower bound of $5n - o(n)$ for Boolean circuits. In *Mathematical Foundations of Computer Science (MFCS 2002)*, volume 2420 of *Lecture Notes in Computer Science*, pages 353–364. Springer, 2002.

[KS93] Ephraim Korach and Nir Solel. Tree-width, path-width, and cutwidth. *Discrete Applied Mathematics*, 43(1):97–101, 1993.

[Lea] Lean FRO and contributors. Comparator: a trustworthy judge for Lean proofs. Software. https://github.com/leanprover/comparator. Revision fd5d5bc; lean4export revision 66f1fb4.

[Lev80] Gavriela Lev. *Size Bounds and Parallel Algorithms for Networks*. PhD thesis, University of Edinburgh, 1980.

[Li23] Xin Li. Two source extractors for asymptotically optimal entropy, and (many) more. In *Proceedings of the 64th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2023)*, pages 1271–1281. IEEE, 2023. arXiv:2303.06802.

[LM23] Lyuben Lichev and Dieter Mitsche. On the minimum bisection of random 3-regular graphs. *Electronic Journal of Combinatorics*, 30(2):P2.40, 2023.

[Lok09] Satyanarayana V. Lokam. *Complexity Lower Bounds using Linear Algebra*, volume 4 of *Foundations and Trends in Theoretical Computer Science*. now Publishers, 2009.

[LV83] Gavriela Lev and Leslie G. Valiant. Size bounds for superconcentrators. *Theoretical Computer Science*, 22(3):233–251, 1983.

[LY22] Jiatu Li and Tianqi Yang. $3.1n - o(n)$ circuit lower bounds for explicit functions. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*, pages 1180–1193. ACM, 2022.

[Lyo17] Russell Lyons. Factors of IID on trees. *Combinatorics, Probability and Computing*, 26(2):285–300, 2017.

[LZ24] Xin Li and Yan Zhong. Explicit directional affine extractors and improved hardness for linear branching programs. In *39th Computational Complexity Conference (CCC 2024)*, LIPIcs, pages 10:1–10:14, 2024.

[Mor23] Hiroki Morizumi. An approach to circuit lower bounds via bounded width circuits. arXiv:2305.00794, 2023.

[MP06] Burkhard Monien and Robert Preis. Upper bounds on the bisection width of 3- and 4-regular graphs. *Journal of Discrete Algorithms*, 4(3):475–498, 2006.

[Nur09] Sergey Nurk. An $O(2^{0.4058m})$ upper bound for Circuit SAT. PDMI preprint 10/2009, 2009.

[Pau77] Wolfgang J. Paul. A $2.5n$-lower bound on the combinational complexity of Boolean functions. *SIAM Journal on Computing*, 6(3):427–443, 1977.

[Sch74] Claus-Peter Schnorr. Zwei lineare untere Schranken für die Komplexität Boolescher Funktionen. *Computing*, 13(2):155–171, 1974.

[The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[The26] The CSLib Contributors. CSLib: a Lean library for computer science. https://github.com/leanprover/cslib, 2026. Commit 89dc6fb.

[Vad12] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1–3):1–336, 2012.

[Val77] Leslie G. Valiant. Graph-theoretic arguments in low-level complexity. In *Mathematical Foundations of Computer Science (MFCS 1977)*, volume 53 of *Lecture Notes in Computer Science*, pages 162–176. Springer, 1977.

[Wil13] Ryan Williams. Improving exhaustive search implies superpolynomial lower bounds. *SIAM Journal on Computing*, 42(3):1218–1244, 2013.

AI methodology statement

Large language models (Claude, by Anthropic, and ChatGPT, by OpenAI) were used extensively in this work: to write the Lean formalization, and to draft, revise, and proofread the manuscript, including checking its references. Every result marked with a Lean declaration is verified by the Lean kernel, using only the standard axioms; the statements of those declarations were read against the paper. The results that are not formalized are marked in the text: Propositions 2.3 and 5.3, the remarks, all results in Section 10 including conditional expectations and ledger counting, the program partition and packing statements of Lemma 3.6 and Theorems 3.7 and 6.7, the tree-limit calculation of Proposition 7.7, the DNNF smoothing conversion, and the cited extractor constructions. The authors take responsibility for all statements, proofs, and text in this paper.